

Repaso

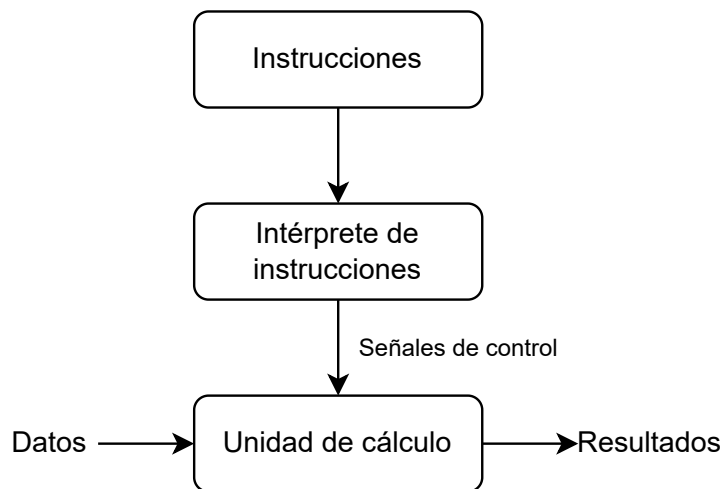
Concepto de programa

Se puede considerar como antecedente más cercano de los sistemas de cómputo a las máquinas mecánicas de calcular, capaces de realizar solo una operación a la vez. Si el cálculo a resolver requería más de una operación, se requería modificar la máquina. Para esto se necesitaba de una persona (operador).

El modelo en que un operador tiene que actuar sobre la máquina, se puede considerar como un modelo de **programación en hardware**. Ya que al cambiar de tarea, se requiere cambiar el medio físico que la ejecuta.

En un sistema de cómputo, la secuencia de tareas está almacenada en la máquina como instrucciones, que son decodificadas o interpretadas por una unidad de control (CU) y ejecutadas por una unidad de proceso.

Este nuevo modelo, donde las operaciones se definen en instrucciones almacenadas en memoria, lo consideramos **programación en software**. Ya que cambiar de tarea consiste en modificar las instrucciones y no el medio físico.



La información requerida para cada paso de la tarea es proporcionada por la secuencia de instrucciones, que son convertidas en señales y enviadas a las unidades funcionales.

Un programa:

- Es una secuencia de instrucciones almacenadas.
- Cada acción es una instrucción (de máquina).
- Cada instrucción es convertida en señales.
- Diferentes señales de control son necesarias para cada operación.

La primera generación de computadoras surgen en la década de los 40, con el fin de resolver cálculos matemáticos complejos en un menor tiempo, con fines militares y científicos. Usando dos tipos de tecnologías: válvulas electrónicas o relés.

Sobresale por novedosa la IAS, desarrollada por **Jonh Von Neumann**:

- Operaba en binario en vez de decimal.
- Única memoria para instrucciones y datos.
- Repertorio de 21 instrucciones de 20 bits.
- Los registros del CPU:
 - Registro de buffer de memoria. (MBR)
 - Registro de direcciones de memoria. (MAR)
 - Registros de instrucción y buffer de instrucción.
 - Registro contador de programa. (PC)

El modelo Von Neumann está basado en 5 subsistemas:

- Unidad aritmético-lógica (ALU).
- Unidad de control (CU).
- Unidad de memoria.
- Unidad de entrada de datos.
- Unidad de salida de datos.

Pudiendo ser simplificado en un modelo de 3 subsistemas que perdura aún hoy en día:

- Unidad de procesamiento central:
 - UC
 - ALU
 - Registros
- Unidad de entrada/salida: para introducir y extraer datos e instrucciones.
- Unidad de memoria: para almacenar temporalmente instrucciones.

Lenguajes

Llamamos **lenguaje máquina** al que el sistema de computo es capaz de "entender", es del tipo binario. En este se codifican las instrucciones, cuyo conjunto de bits están almacenados en palabras^[1] de memoria (1 o más).

Ya que para un ser humano es casi imposible trabajar con las representaciones en binario, se utiliza, en el nivel de abstracción contiguo, **lenguaje ensamblador** (assembly). Donde cada campo que forma una instrucción se puede representar con texto y números.

Las instrucciones máquina varían en complejidad. Las más complejas requieren de múltiples pasos o acciones para ser llevadas a cabo.

Cada instrucción máquina está organizada en **campos**:

- Campo de identificación de la instrucción (codop): especifica la acción a realizar.
- Campos de referencia a operandos: establece la información de referencia a el o los operandos (si se requieren). Esta información varía dependiendo el direccionamiento.
- Campo de referencia del resultado: establece donde almacenar el resultado. Pudiendo ser uno o más resultados.

- Campo de referencia a la siguiente instrucción: provee información a la CPU sobre donde se encuentra la siguiente instrucción a ejecutar. Usualmente esto se da de manera secuencial (contiguas en memoria).

Referimos a la **taxonomía basada en el formato de instrucción**, cuando hablamos de la clasificación de las máquinas según su formato de instrucción. La clasificación identifica la cantidad de campos que tendrán direcciones de memoria:

- Máquinas de 4 direcciones:

COP	DirRes	DirOp1	DirOp2	DirPróxIns
-----	--------	--------	--------	------------
- Máquinas de 3 direcciones:

COP	DirRes	DirOp1	DirOp2
-----	--------	--------	--------
- Máquinas de 2 direcciones:

COP	DirOp1	DirOp2
-----	--------	--------
- Máquinas de 1 direcciones:

COP	DirOp/DirRes
-----	--------------
- Máquinas de 0 direcciones:

COP

En las instrucciones de 3, 2, 1 y 0 direcciones, la dirección de la siguiente instrucción está implícita. En instrucciones de 1 dirección, se hace uso del registro de uso general acumulador. Con instrucciones de múltiples direcciones se suelen disponer múltiples registros. Las máquinas con instrucciones de 0 direcciones hacen uso de la pila.

Nº direcciones	Representación simbólica	Interpretación
3	OP A, B, C	$A \leftarrow B \text{ OP } C$
2	OP A, B	$A \leftarrow A \text{ OP } B$
1	OP A	$AC \leftarrow AC \text{ OP } A$
0	OP	$T \leftarrow (T - 1) \text{ OP } T$

Otra forma de clasificación de las máquinas es la **taxonomía basada en los tipos de almacenamientos de datos**. Los datos pueden estar almacenados en tres lugares distintos:

- Memoria: cache, principal, virtual.
- Registros del CPU: la referencia es un número de registro.
- Dispositivos E/S: la referencia es un número de registro E/S.

ISA

Diseño

Formato de instrucción

El formato de instrucción refiere a la estructura que tendrán las instrucciones.

Formato fijo o variable:

- Formato fijo: tienen todas una misma cantidad de bits. La captación y ejecución es más sencilla, pero aumenta la cantidad de instrucciones de los programas.
- Formato variable: la longitud de las instrucciones varía con el tipo de instrucción y modo de direccionamiento. La captación y ejecución es más compleja, pero disminuye la cantidad de instrucciones de los programas.

Cantidad de campos de direcciones:

- A mayor cantidad de referencias de direcciones, se tienen instrucciones más complejas y largas, menos instrucciones por programa, y captación y ejecución de las instrucciones más lenta.
- A menor cantidad de referencias de direcciones, se tienen instrucciones más simples y cortas, más instrucciones por programa y una captación y ejecución de las instrucciones más rápida.

Tipos de datos

Las categorías generales más importantes son:

- **Direcciones:** en muchos casos debe realizarse algún cálculo sobre la referencia a un operando de una instrucción a fin de determinar la dirección de memoria.
- **Números:** Todos los lenguajes máquina incluyen tipos de datos numéricos. Incluso en el procesamiento de datos no numéricos se necesitan números que actúen como contadores, longitudes de campos, etc. Por ej. enteros, en coma fija, en coma flotante, en decimal (BCD), con signo, sin signo, etc.
- Texto o **secuencia de caracteres:** se representan mediante secuencias de bits, utilizando una codificación en particular. Por ej. ASCII o EBCDIC.
- **Datos lógicos:** a veces es útil considerar una unidad de n bits como n elementos o datos de un bit, donde cada elemento tiene un valor 1 o 0. Cuando los datos son vistos de esta manera, se consideran datos lógicos.

Repertorio de operaciones

El número de códigos de operación (**codops**) diferentes varía ampliamente de una máquina a otra. Sin embargo, en todas las máquinas podemos encontrar los mismos tipos generales de operaciones:

- Transferencia de datos: transfiere datos de una posición a otra.
- Aritméticas: Puede implicar transferencias de datos antes y/o después. Realiza operaciones en la ALU. Actualiza códigos e indicadores de condición.
- Lógicas: lo mismo que una aritmética.
- De conversión: similar a la aritmética y lógica. Puede implicar a lógica especial para realizar la conversión.
- De E/S: cursa una orden a un módulo de E/S.
- De control del sistema: por lo general instrucciones privilegiadas que pueden ejecutarse solo mientras el procesador está en un estado privilegiado concreto. Normalmente, están reservadas para que las use el sistema operativo.
- De control de flujo: Actualiza el contador de programa. En el caso de llamadas y retornos de subrutinas, gestiona la transferencia y enlace de parámetros.

Administración y tipo de registros

Administración y tipo de registros: dado que son un recurso escaso y tienen un fuerte impacto en la velocidad de ejecución de los programas, deben ser diseñados con un enfoque en:

- Tamaño y cantidad de registros.
- Tipos de registros: cual será su uso y a que instrucciones se van a aplicar.

Modos de direccionamiento

Modos de direccionamiento: las decisiones respecto a los modos de direccionamiento se orientan a:

- Cantidad de modos de direccionamiento.
- Tipos de modos de direccionamiento.

Estrategias de diseño

Los diseños de las arquitecturas se han orientado hacia dos estrategias:

- **CISC** (Complex Instruction Set Computing)
 - Repertorio de instrucciones muy amplio.
 - Muchos modos de direccionamiento.
 - Formato de instrucciones variable.
- **RISC** (Reduced Instruction Set Computing)
 - Repertorio de instrucciones simplificado.
 - Pocos modos de direccionamiento.
 - Formato de instrucción fijo.

Técnicas de almacenamiento

Existen dos formas de almacenar un mismo dato en múltiples palabras de memoria. De acuerdo al orden en que se almacena cada parte del dato podemos discernir:

- **Big-endian:** el byte más significativo se almacena en la dirección más baja.
- **Little-endian:** el byte más significativo se almacena en la dirección más alta.

Las 2 formas son incompatibles entre sí, una arquitectura tendrá una u otra.

Ejemplo

Suponga que tenemos el valor hexadecimal de 32 bits: 12345678, y que se almacena en una palabra de 32 bits de una memoria direccionable por bytes, en la posición de byte 184. El valor consta de 4 bytes, con el contenido 78 para el byte menos significativo, y el byte más significativo con el valor 12 como contenido. Hay dos maneras de memorizar este valor:

Dirección	Valor en Big-Endian	Valor en Little-Endian
184	12	78
185	34	56
186	56	34
187	78	12

El otro aspecto referido al almacenamiento de los datos es donde se guardan, particularmente cuando ocupan más de una palabra:

- **Alineados.**
- **No alineados.**

La regla generalmente aplicable para que un dato esté bien alineado es que esté ubicado en una dirección divisible por su tamaño. Un objeto de datos de n bytes ubicado en la dirección de memoria

D está alineado si $D \% n = 0$.

Así, un dato que ocupa un solo byte siempre está bien alineado, un dato de dos bytes está bien alineado si está en una dirección par, un dato de 4 bytes está bien alineado si está en una dirección divisible por 4, etc.

Algunos procesadores solo permiten acceso alineado, otros no alineados y otros ambos.

Direccionamiento

Los modos de direccionamiento son las diferentes formas de especificar el lugar donde reside un operando, donde se guardará el resultado o donde la próxima instrucción a ejecutar.

Los modos de direccionamiento típicos son:

Inmediato: el operando está presente en la propia instrucción. Una vez captada la instrucción, no se requiere resolver una referencia a memoria para obtener el operando, ahorrando un ciclo de memoria o de cache. Si la instrucción es de tamaño fijo, el número estará restringido al tamaño de la instrucción, y si es variable, el tamaño de la instrucción estará dado por el tamaño del operando.

```
mov ax, 10h
```

Directo: el campo de direcciones contiene la dirección efectiva del operando. (una referencia a memoria)

```
; directo absoluto (a memoria):  
mov ax, num  
mov num1, bl  
  
; directo registro:  
add ax, bx  
mov ah, al
```

Indirecto: El problema del direccionamiento directo es que la longitud del campo de direcciones es normalmente menor que la longitud de palabra, limitando el rango de direcciones. Una solución es hacer que el campo de direcciones referencie la dirección de una palabra de memoria, la cual contenga la dirección completa del operando. (tiene una dirección, donde se encuentra la dirección en que está el dato).

El direccionamiento indirecto a registro requiere 1 acceso más a memoria que el directo.

El direccionamiento indirecto a memoria requiere 2 accesos más a memoria que el directo.

```
; indirecto a registro  
mov ax, [bx]  
  
; indirecto a memoria  
mov ax, [1001h]  
  
; indirecto a registro con desplazamiento  
mov ax, [bx + 2]
```

Base (o con desplazamiento): uno de los operandos está en una localidad de memoria cuya dirección es la suma de una dirección contenida en un registro base y una constante que forma parte de misma instrucción. Le decimos **base indexado** si la constante se encuentra también en otro registro.

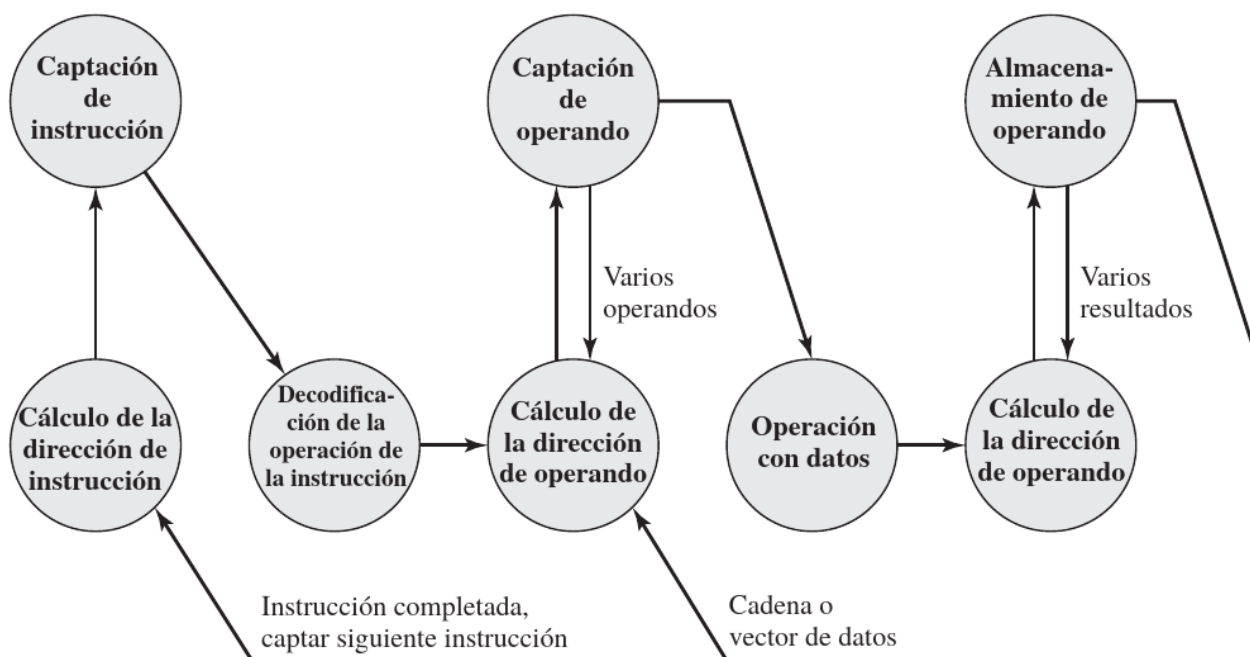
Relativo al PC: la dirección está formada por la suma de una constante con el valor del registro *PC*.

Pila: un operando se encuentra en la cabecera de pila.

Ciclo de instrucción

El **ciclo de instrucción** es la secuencia de acciones que realiza el procesador para resolver las instrucciones.

En su forma más sencilla se descompone en dos acciones básicas **captación** y **ejecución**.



En el ciclo de captación de la instrucción:

- La dirección de la instrucción se carga en el *PC* (contador de programa).
- La *UC* (unidad de control) envía al bus de direcciones el valor almacenado en el *PC*.
- La *UC* lee la instrucción desde la memoria, y la almacena en el registro temporal *IR* (instruction register).
- El *PC* se incrementa en 1, apuntando a la siguiente dirección.
- La *UC* decodifica la instrucción captada.

El ciclo de ejecución puede variar, dependiendo del tipo de instrucción a ejecutar. Las posibles acciones son:

- Acciones procesador-memoria.
- Acciones procesador-E/S.
- Acciones de procesamiento de datos.
- Acciones de control.
- Acciones combinadas de las anteriores.

Pila

Es una estructura de datos con acceso **LIFO** (Last In First Out), esto significa que solo un elemento es accesible en un instante dado. Solo se pueden "apilar" y "desapilar" elementos de la **cabecera de la pila**.

El registro de la CPU **stack pointer** (*SP*) tendrá la dirección de la cabecera.

La implementación de una pila requiere que exista un cierto conjunto de posiciones en memoria utilizadas para almacenar sus elementos de manera contigua.

Sobre la pila se pueden realizar dos operaciones:

- **push** : almacenar o apilar un elemento en la cabecera.
- **pop** : extraer o desapilar un elemento de la cabecera.

Estas operaciones requieren de dos acciones:

- Movimiento de datos:
 - Registro a memoria.
 - Memoria a registro.
 - Memoria a memoria.
- Operaciones del puntero *SP*.

En la familia i80x86:

- Al hacer **push** :
 - se decrementa el *SP*.
 - se almacena el dato en el tope (donde apunta el *sp*).
- Al hacer **pop** :
 - se extrae el dato desde el tope.
 - se incrementa el *SP*.

Al hacer **pop**, el dato extraído permanece en la memoria pero es ignorado.

En otras familia de procesadores como la de motorola M68xx, el *SP* en vez de apuntar al último dato guardado, apunta al primer espacio libre, donde se almacenaría un próximo dato.

```
org 2000h
mov ax, 3000h
push ax ; El SP se incrementa en 2. Se almacena 3000h en la pila (tope).
pop bx ; Se almacena 3000h en el registro bx. El SP decrementa en 2.
```

Subrutinas

Una **subrutina** es una sección de código a la que se transfiere el control con el fin de realizar una tarea concreta. Una vez finalizada la tarea, devuelve el control al programa que la invocó en el punto donde fue invocada.

La subrutina se invoca mediante una instrucción del repertorio de instrucciones del procesador, usualmente, **call**. La subrutina finaliza mediante una instrucción de retorno, por ejemplo **ret**.

El uso de subrutinas presenta, como principales ventajas, la reutilización de código y modularidad.

```
; programa principal
...
call nombreDeSubrutina
proximaInstrucción ; dirección de retorno
...

; subrutina
nombreDeSubrutina Proc
...
...
ret
```

Al invocar una subrutina:

1. Hace un `push` a la pila con la dirección de la instrucción actual (siguiente instrucción en `vonsim` y `msx88`).
2. Hace un salto incondicional al código indicado por la etiqueta.
3. Se ejecuta el código de la subrutina con normalidad.
4. Para retornar (`ret`), se hace un `pop` de la pila.
5. Se hace un salto incondicional al valor recuperado de la pila, de modo que al avanzar a la siguiente instrucción (incrementar `PC`), se ejecute la instrucción siguiente a la que invocó la subrutina.

Hay que asegurarse de que al ejecutar la instrucción de retorno, el dato que se encuentra en el tope de la pila sea la dirección de retorno

Notar que este mecanismo nos permite con igual facilidad ir del programa principal a una subrutina como de una subrutina a otra, esto se le llama **anidar llamadas a subrutinas**. Se puede hacer tantas veces como memoria stack tengamos disponible para la estructura de datos de cada subrutina. Como la información se guarda en una estructura de tipo LIFO, al terminar una subrutina, se devuelve el control a la de nivel anterior.

Si una subrutina hace uso de un registro, debe guardar el valor (usualmente en la pila) y restaurarlo antes de retornar.

Para realizar su trabajo, se requiere pasar **parámetros** entre la subrutina y el programa que la invoca. Para realizar esto, existen 3 métodos:

- **Vía registros:** los parámetros se pasan vía registros de la CPU.
 - La cantidad de parámetros está limitada por los registros disponibles.
 - Dado que se modifica el contenido de los registros, es importante documentar los registros que serán utilizados para este fin.
 - A su vez, la subrutina puede devolver datos al programa principal utilizando el mismo mecanismo.
- **Vía memoria:** los parámetros se transfieren a través de un área de memoria definida.
 - Difícil de estandarizar, debido a las dificultades en asignar un área de memoria.
- **Vía pila (stack):** los datos se pasan a través de la pila.
 - Es el método más ampliamente usado, ya que representa más ventajas que los otros.

- Es independiente de la memoria y los registros (principal ventaja).

Pasar los parámetros vía pila requiere hacer `push` de los parámetros requeridos por la subrutina antes de invocarla. El 80x86 no dispone de un modo de direccionamiento relativo al `SP`, ni al `BP`, por lo que debemos realizar una serie de pasos para circunventar la limitación.

1. Salvar el estado de `BP` en la pila.
2. Apuntar con `BP` al tope de la pila (`BP=SP`)

Entonces, toda subrutina que utilice la pila deberá comenzar con estas dos líneas:

```
push bp ; Salvar BP
mov bp, sp ; BP = SP
call nombreDeSubrutina
```

3. Reservar espacio para datos locales (opcional)

Le decimos variables locales, a variables temporales usadas por la subrutina y que al finalizar deben ser descartadas.

Una manera de lograrlo es moviendo `SP` una determinada cantidad de bytes, por ejemplo:

```
sub sp, 2 ; Reservar 2 bytes
```

4. Salvar valores de otros registros (opcional)

En el caso que la subrutina necesite modificar algún registro de la CPU lo salvamos de la misma manera que al `BP`.

```
push dl; salvar registro dl
```

5. Acceder a parámetros

Para acceder a los datos en el área de la Pila asignada al procedimiento se deberá sumar un desplazamiento fijo al `BP` (direccionamiento base con offset).

El offset que se suma al `BP` se obtiene sumando todos los bytes intercalados entre la posición actual del `BP` y la posición del parámetro a buscar. El `bp` viejo ocupa 2 bytes, la dirección de retorno 4 y cada parámetro 2 bytes.

```
mov cx, [bp + 8] ; Accede al parametro 1 4+2+2 = 8 bytes
```

6. Escribir sentencias a ejecutar

Se escriben las instrucciones propias de la tarea que desarrolla la subrutina, constituyen su cuerpo.

7. Restaurar los registros modificados (opcional)

Antes de retornar, en caso de haber salvado el valor de registros se restauran.

```
pop dl
```

8. Restaurar el valor original de SP y BP

El SP pudo haberse movido cuando se reservó espacio para las variables locales. Notar que el BP no se movió durante el acceso a la estructura de datos en la Pila.

```
mov sp, bp ; restablece el valor de SP
pop bp ; restaura el valor de bp
```

9. Retornar parámetro (opcional)

10. Regresar (correctamente del procedimiento)

```
ret
```

Ejemplo uso de subrutinas en MSX88:

```
                                org 1000h

num1 dw 5h
num2 dw 3h
res   dw ?

org 2000h
mov  ax, num1
push ax ; parámetro 1
mov  ax, num2
push ax ; parámetro 2
mov  ax, offset res
push ax ; dirección donde almacenar el resultado
mov  dx, 0 ;
call mul ; salto no condicional a la subrutina
pop  ax ; remueve la dirección de res de la pila
pop  ax ; remueve num2 de la pila
pop  ax ; remueve num1 de la pila
hlt   ; Al regresar del procedimiento "limpia" los valores que se
almacenaron                                ; en la pila para
usar como parámetros

mul:  org 3000h
      push bx ; salva el valor del registro
      mov  bx, sp ; para acceder con un desplazamiento sobre bx al stack
      push cx ; salva el valor del registro (2 bytes)
      push ax ; salva el valor del registro (2 bytes)
      push dx ; salva el valor del registro (2 bytes)
      add  bx, 6 ; ahora bx contiene la dirección de num2
      mov  cx, [bx] ; cx = num2
      add  bx, 2 ; ahora bx contiene la dirección de num1
      mov  ax, [bx] ; ax = num1
suma:  add  dx, ax ; dx = dx + ax
      dec  cx ; cx = cx - 1
```

```

jnz suma ; jump if not zero (salta a suma si ZF ≠ 0)
sub bx, 4 ; ahora bx contiene la dirección de la línea que invocó al módulo
mov ax, [bx] ; ax tiene la instrucción que invocó al modulo
mov bx, ax ; bx = ax
pop dx ; restaura el valor de dx
pop ax ; restaura el valor de ax
pop cx ; restaura el valor de cx
pop bx ; restaura el valor de bx
ret      ; notar el orden inverso al restaurar los registros

```

end

Las subrutinas pueden producir un resultado y que sea "devuelto" al programa que la invoca. El resultado se retorna por las mismas direcciones de memoria donde la subrutina recibió los parámetros.

Interrupciones

Definición

En condiciones "normales" la CPU ejecuta instrucciones de manera secuencial, una a continuación de la otra.

Definición

Una **interrupción** es una señal que recibe el procesador y le indica que debe "interrumpir" el curso de ejecución actual y pasar a ejecutar código específico para tratar esta situación. Una vez finalizada, reanuda la ejecución de la tarea suspendida en el punto en que se detuvo.

Se dan en tres pasos:

1. Suspensión. Se suspende la ejecución de la tarea en curso.
2. Bifurcar. Salta a otra tarea, asociada a la interrupción (servicio de interrupción).
3. Reestablecer la tarea suspendida, en las condiciones en que se encontraba en el momento en que se la detuvo.

Para permitir el uso de interrupciones, se añade un ciclo de interrupción al ciclo de instrucción, en el cual la CPU comprueba si se ha generado una interrupción. Si no está activa la señal de interrupción, el procesador continúa con el ciclo de captación. Si hay alguna interrupción pendiente:

1. Detectar el pedido de interrupción: El procesador examina en cada ciclo de instrucción la presencia de interrupciones a través de flags.
2. Detener la tarea que se estaba ejecutando.
3. Salvar el **contexto** (estado de la tarea que se estaba ejecutando): se guarda el estado del proceso, usualmente en la pila, de dos posibles formas:
 - Guardar la próxima instrucción a ejecutar y algún registro crítico, como el PC y PSW^[2].
 - O guardando todos los registros del procesador.
4. Buscar en un área de memoria definida la dirección de comienzo del **servicio de interrupción**. Este área de memoria reservada donde están estas direcciones (varias direcciones, para cada

interrupción existe una) se denomina **área de vectores de interrupciones**. Carga la dirección de comienzo de la interrupción al contador de programa, bifurcando el servicio de la interrupción.

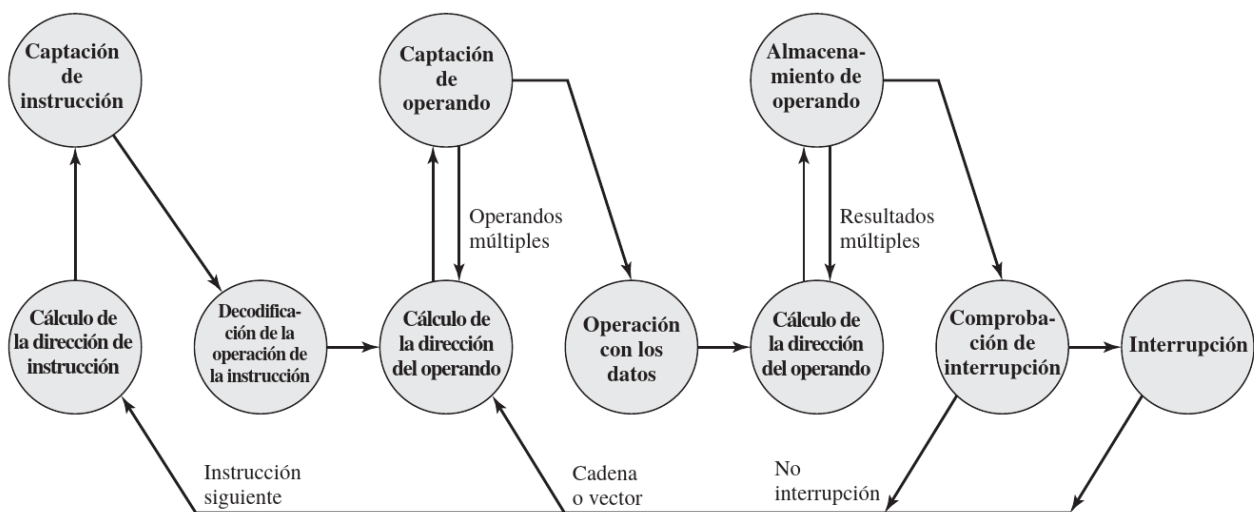
5. Ejecutar el servicio de la interrupción.
6. Cuando termina de ejecutar el servicio de interrupción, regresa mediante una instrucción especial de retorno *RTI* (Return From Interruption).
7. La instrucción *RTI* desapila lo salvado en la pila cuando se atendió la interrupción, continuando la normal ejecución de la tarea interrumpida.

Dada la diversidad de causas de origen de una interrupción, algunas se consideran más importantes o prioritarias que otras. El CPU es, usualmente, capaz de manejar múltiples interrupciones, de múltiples orígenes.

Cuanto mayor prioridad tenga una interrupción, mayor es la urgencia con que será atendida. Al arribar una interrupción y ser atendida, se inhabilitan las interrupciones de igual o menor nivel de prioridad. Una interrupción de mayor prioridad puede incluso interrumpir a una de inferior prioridad en curso.

Si llega una nueva interrupción, de menor o igual prioridad a la en curso, quedará pendiente hasta finalizar el servicio de la actual. Cuando tienen una misma prioridad se atienden en el orden en que llegan.

Al finalizar una interrupción se habilitarán nuevamente las interrupciones, y son atendidas las que estaban pendiente. Una vez que todas fueron atendidas, se retoma el programa del usuario.



Clasificación de interrupciones

Típicamente, según su prioridad, podemos categorizar las interrupciones por como responde el sistema de cómputo ante estas:

- **No enmascarables:** no pueden ser ignoradas, se atienden indefectiblemente. Asociadas a eventos críticos, peligrosos o de alta prioridad.
- **Enmascarables:** pueden ser eventualmente ignoradas. El procesador puede inhibir la atención de la interrupción. Están asociadas a operaciones menos críticas como de E/S.

Según su origen, podemos clasificarlas:

- **Interrupciones por hardware:** son generadas por señales físicas asociadas a, de acuerdo a su origen, eventos externos o internos.

- **Evento externo:** conocidas como **interrupt request**. Asociado a operaciones de dispositivos conectados al subsistema de E/S. Algunos eventos típicos que pueden generar interrupción son: finalización de una transferencia, error en la transferencia, dispositivo indisponible, etc. Se consideran "verdaderas" interrupciones, pueden ocurrir en cualquier instante de tiempo, pudiendo estar o no relacionadas con el proceso en ejecución en ese momento. El computador tiene que ser capaz de manejar estos eventos externos no planeados o asincrónicos.
- **Evento interno:** conocidas como **traps o excepciones**. Son señales creadas dentro del sistema de cómputo en respuesta a situaciones propias del proceso en ejecución y no vinculadas con operaciones de E/S, por lo que no son estrictamente aleatorias. Por ej. un error asociado a la ejecución de una instrucción, desbordamiento aritmético, división por cero, temporizadores propios del sistema, error de paridad, pérdida de energía.
- **Interrupciones por software:** conocidas como **software interrupt**. Son instrucciones explícitas, con efecto similar a una interrupción por hardware. Como normalmente el SO administra los servicios de interrupciones, se suelen utilizar para invocar servicios del sistema operativo, son "llamadas" a funciones del SO (**syscalls**).

Sobre interrupciones por hardware

La CPU es significativamente más rápida que los dispositivos E/S. Además estos son inconstantes, es difícil predecir cuando van a estar disponibles o cuanto tardarán en realizar una tarea. Por ejemplo, no sabemos cuando una persona presionará una tecla. Para evitar que la CPU quede sin hacer nada esperando (ociosa) a un dispositivo, necesitamos un mecanismo para que los dispositivos le avisen a la CPU, de modo que pueda seguir ejecutando programas sin verificar de forma continua el estado de estos.

Sobre interrupciones por software

Los sistemas operativos centralizan el manejo de algunos recursos de la computadora, siendo intermediarios entre programas y recursos. Los programas necesitan comunicarse con el sistema operativo, para ello, realizan llamadas de sistema (o syscalls). Estas interrumpen la ejecución del programa y pasan el control a una subrutina del SO, que luego de finalizada continúa la ejecución del mismo. Las subrutinas del sistema operativo suelen invocarse utilizando interrupciones.

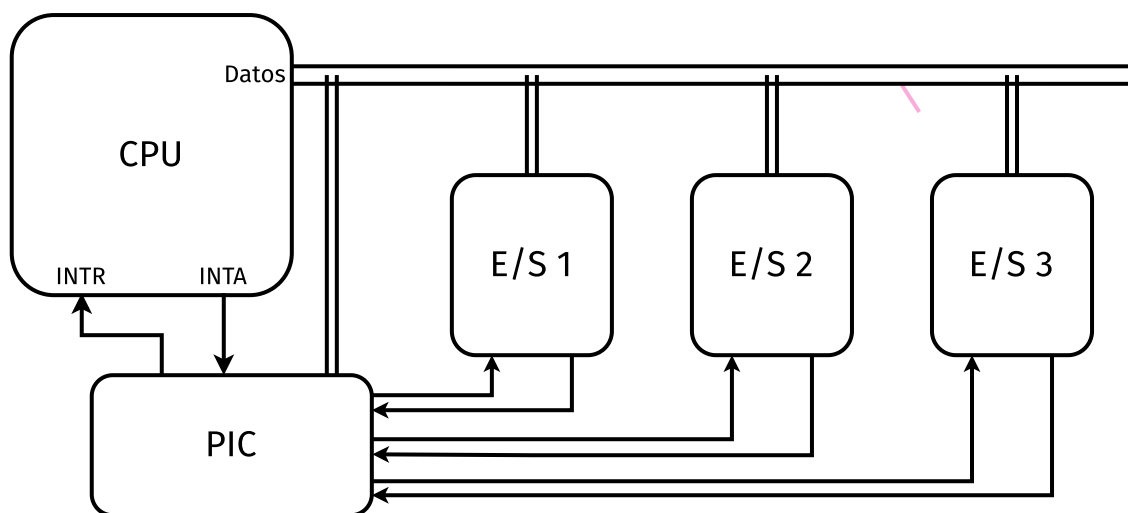
Identificar origen de la interrupción

Cuando hay múltiples fuentes de interrupción, hay varias formas de identificar el origen del pedido:

- **Múltiples líneas de interrupción:** una línea para cada dispositivo, lo cual limita la cantidad de dispositivos a conectar, debido a la cantidad restringida de señales que puede la CPU manejar. Es sencilla de implementar pero no resulta práctico dedicar más de unas pocas líneas del bus o terminales del procesador a ser líneas de interrupción. En consecuencia, incluso si se utilizan varias líneas, es probable que a cada una se conecten varios módulos de E/S. Por eso, se debe utilizar alguna de las otras tres técnicas en cada línea.
- **Software Poll o encuesta:** una única línea de interrupción para todos los dispositivos. Cuando ocurre una interrupción se produce una bifurcación a una rutina de servicio (programa) que se

encarga de consultar a cada módulo de E/S para determinar cuál la provocó. El orden en el que se consultan los módulos determina su prioridad. Puede resultar muy lento e ineficiente.

- **Daisy Chain** (*hardware poll*) (vectorizada): una línea de solicitud de interrupción *INTR* para todos los dispositivos. La línea de reconocimiento *INTA* se conecta encadenando los módulos uno tras otro. Cuando el procesador recibe una interrupción, activa el reconocimiento de interrupciones, esta señal se propaga a través de la secuencia de módulos de E/S, hasta que alcanza al que solicitó la interrupción. Normalmente este módulo responde colocando una palabra (*vector*) en las líneas de datos. El vector es la dirección del módulo de E/S o algún otro identificador específico, que la CPU utilizará como puntero a la rutina de servicio del dispositivo. Evitando así ejecutar una rutina de servicio general en primer lugar. Esto se conoce como **interrupción vectorizada**. El orden de los módulos en la conexión en cadena determina su prioridad.
- **Arbitraje de bus** (vectorizado): una única línea de interrupción *INTR* para todos los dispositivos. Un módulo de E/S debe disponer del control del bus antes de poder activar la línea de petición de interrupción para que así solo un módulo pueda activar la línea en un instante dado. Cuando el procesador detecta la interrupción, responde mediante la línea de reconocimiento de interrupción *INTA* al **controlador PIC**, que provee el vector a través del bus de datos, que identifica la fuente de interrupción. El vector es un número o identificador que usará la CPU para transferir el control a la subrutina que le corresponde al periférico (**interrupciones vectorizadas**). Las líneas de interrupción tienen un orden de prioridad, las líneas con menor prioridad podrán ser interrumpidas por las de mayor. Si existe un maestro bus, solo él puede interrumpir.



Interrupciones vectorizadas y PIC

El **vector de interrupciones** se encuentra almacenado en la memoria principal. En este, cada elemento de índice n , contiene una dirección. Esta dirección corresponde a la subrutina a la que ceder el control cuando se recibe un pedido de interrupción con identificador n . Estas subrutinas se denominan *interrupt handler* o *interrupt service routine*.

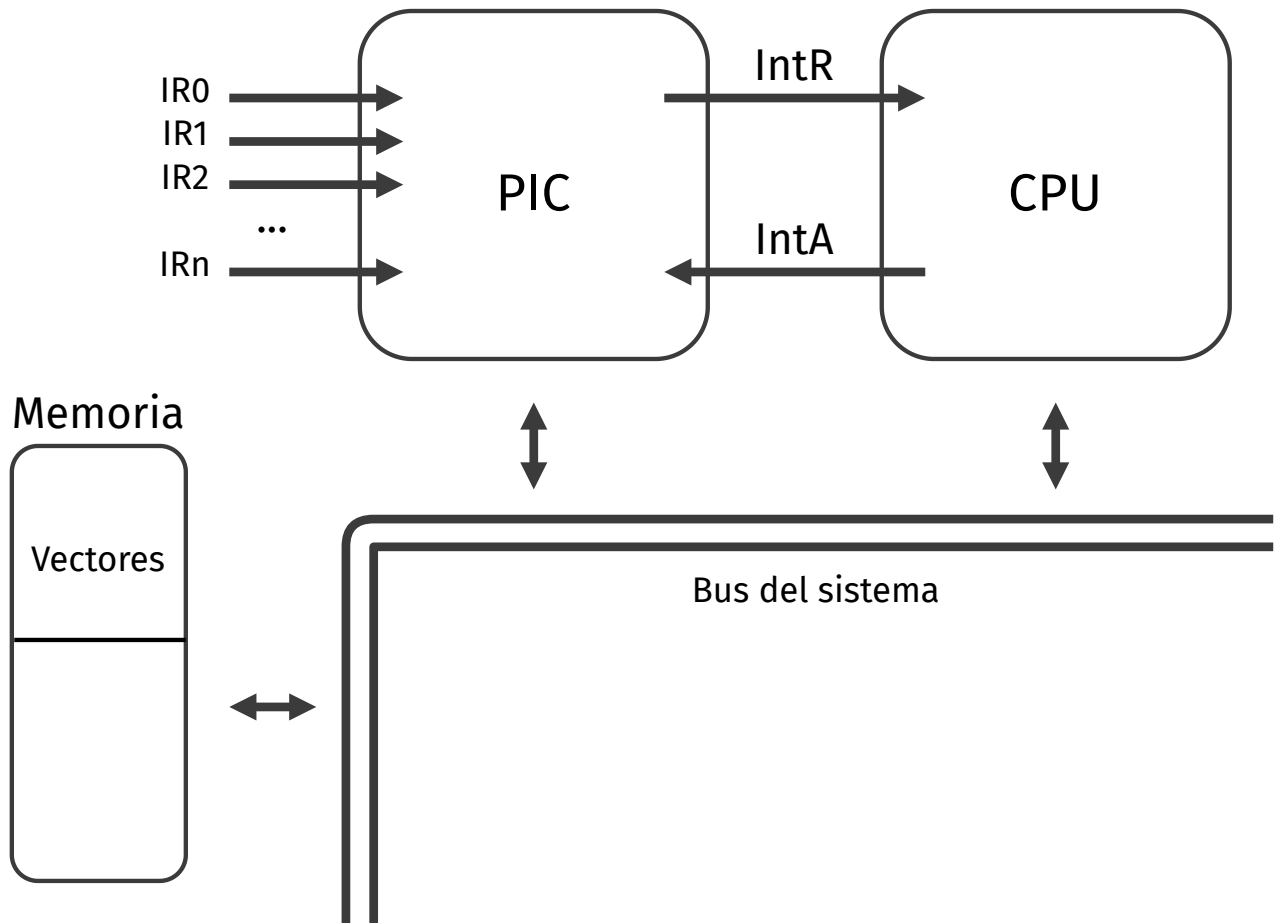
El **PIC** (programmable interrupt controller) es un circuito integrado que ayuda a la CPU a gestionar las **interrupt request** (IRQ) provenientes de diversas fuentes, que pueden ocurrir de manera simultánea.

1. El PIC recibe los pedidos de interrupción, típicamente de periféricos que piden atención, comprueba si la petición de entrada tiene un valor de prioridad más alta que el nivel de que esté dando servicio y, en base a esta determinación, emite una interrupción al CPU.
2. El PIC solicita atención a la CPU con la única señal de pedido de interrupción *IntR* (*Interrupt Request*).

3. Cuando la CPU está lista para atender la interrupción, le avisa al PIC mediante la señal *IntA* (*Interrupt Acknowledge*).
4. El PIC genera en el bus de datos el número de la interrupción (*vector*) o identificador, que la CPU usará para identificar la subrutina a la que ceder el control.
5. La CPU lee el número del bus de datos y busca información en la memoria del vector que corresponde al servicio de esa interrupción.

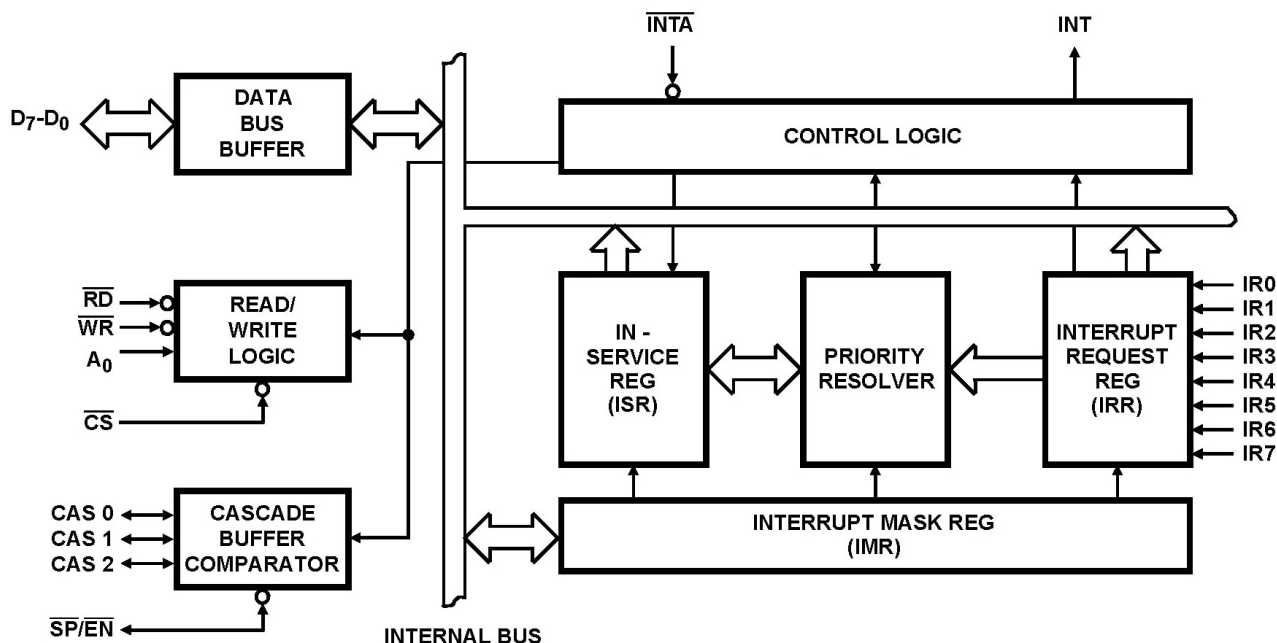
Como puede apreciarse, el PIC *multiplexa* los pedidos de interrupciones de varios dispositivos a una única línea de interrupción del procesador.

Dado que se hace por hardware, es mucho más rápido.



El PIC tiene registros internos:

- **IRR** (*Interrupt Request Register*): contiene los pedidos de interrupción en espera de ser atendidos.
- **ISR** (*In-Service register*): identifica la interrupción en servicio, que está siendo atendida.
- **IMR** (*Interrupt Mask Register*): se usa para habilitar/deshabilitar ciertos pedidos de interrupción (enmascaramiento de interrupciones).



Una señal **EOI** se utiliza para indicar la conclusión de un servicio de interrupción. Cuando el PIC recibe esta señal, "limpia" el bit correspondiente a la interrupción que estaba en servicio. Dando lugar a otras IRQs de menor o igual prioridad generadas por el PIC.

Interrupciones en el i8086

Tiene 2 interrupciones por hardware:

- **INTR**: del tipo enmascarable. Tiene asociado un flag **IF** que determina si va a ser atendida o no.
- **NMI**: del tipo no-enmascarable.

El procesador dispone de una señal de reconocimiento de interrupciones **INTA**.

Tiene sólo una instrucción de interrupción por software: **INT n** , con n entre 0 y 255.

La instrucción de retorno de interrupción es **IRET**.

Dispone de dos flags asociadas con interrupciones:

- **IF**: habilita y deshabilita **INTR**.
- **TF**: habilita y deshabilita el modo **trace** o **single step**. Que es un mecanismo implementado mediante una interrupción que habilita la cpu a ejecutar de a una instrucción a la vez.

El esquema de manejo de interrupciones es vectorizado. Este se ubica en las posiciones más bajas de memoria. Hay 256 vectores (0..255). Cada uno ocupa 4 bytes, 2 para el registro de segmento de código (CS) y 2 para el contador de programa **IP**.

Interrupciones en el MSX88

Vector de interrupciones que contiene 255 direcciones de memoria, una por subrutina. Al recibir una interrupción con identificador n la CPU accede a la posición n de dicho vector, obtiene la dirección de comienzo de la subrutina, y ejecuta un **call**.

Como en el MSX88 las direcciones son de 2 bytes, en teoría todo el vector debería ocupar $256 \times 2 = 512$ bytes. No obstante, por razones históricas, en el MSX88 cada elemento ocupa 4 bytes. Los primeros 2 tienen la dirección, y los otros 2 no se usan. Entonces, el tamaño total del vector es de $256 \times 4 = 1024$ bytes o 1 kB. Comienza en la dirección de memoria 000h y finaliza en 03FF.

En general, el elemento n del vector de interrupciones se encontrará en la dirección $4 \times n$

Las posiciones 0, 3, 6 y 7 se encuentran predefinidas con los handlers de las interrupciones por software.

Ejemplo:

Entonces, por ejemplo, si el procesador es interrumpido por la interrupción con ID 9, irá a buscar el elemento 9 del vector de interrupciones que está en la dirección de memoria $9 \times 4 = 36$, o 24h ..

Info

El vector de interrupciones es necesario tanto para el funcionamiento de ambos tipos de interrupciones. No obstante, no será necesario considerarlo a la hora de programar con interrupciones por software en el simulador.

Interrupciones por hardware

Las interrupciones las inicia el PIC, pero el programador es quien debe escribir los manejadores de interrupciones.

El PIC posee 8 líneas de interrupción por hardware, donde se conectan dispositivos que pueden interrumpir a la CPU, pero solo 4 en uso.

- **INT0**: tecla F10, cada vez que se presiona la tecla.
- **INT1**: timer. Conectada a la salida del timer.
- **INT2**: handshake. Conectada a una salida para el handshake.
- **INT3**: DMA. Conectada a la salida del puerto a impresora.

El PIC permite:

1. Configurar el ID de interrupción (un número entre 0 y 255) asociado a cada línea de interrupción.
2. Deshabilitar las interrupciones de ciertos dispositivos.
3. Establecer prioridades entre dispositivos (por si varios quieren interrumpir al mismo tiempo).
4. Consultar qué dispositivos se encuentran solicitando interrumpir.
5. Consultar qué dispositivos están siendo atendidos.

Cuando un dispositivo quiere interrumpir a través de la línea n , el PIC:

1. Verifica que la línea n esté habilitada (que no este enmascarada).
2. Verifica que no haya otros dispositivos con mayor prioridad que también quieran interrumpir.

3. Si las dos verificaciones previas fueron exitosas, manda la señal de interrupción a la CPU, al mismo tiempo que pone en el bus de direcciones el identificador de interrupción correspondiente a la línea n .
4. La CPU responde buscando la dirección de la subrutina asociada al identificador de interrupciones en el vector de interrupciones, y ejecutándola.

Registros internos del PIC:

Dirección	Registro	Nombre	Propósito	E/S
20h	EOI	Fin de interrupción	Avisa al PIC que se terminó una interrupción, desmarca la línea del registro ISR.	S
21h	IMR	Máscara de interrupción	Enmascara la línea n cuando su bit n es 1. Impidiendo que emita una interrupción al CPU.	S
22h	IRR	Interrupciones pedidas	Si el bit n es 1, el dispositivo de la línea n está solicitando de interrupción. Es de solo lectura para la CPU.	E
23h	ISR	Interrupción en servicio	Si el bit n es 1 una interrupción del dispositivo de la línea n está siendo atendida (nunca habrá más de uno a la vez). Para la CPU es de solo lectura.	E
24h..2Bh	INT n	ID de línea n	Almacena el ID de la interrupción asociada a la línea n (0..7).	S

Para escribir el manejador de interrupción se hace en 3000h como una subrutina usual, pero difieren un par de aspectos.

Se utiliza la instrucción `iret` en vez de `ret`. Dado que cuando se produce una interrupción no solo se apila la dirección de retorno, sino que también los flags. La instrucción `iret` desapila tanto la dirección de retorno como los flags.

Los registros del PIC son accedidos mediante instrucciones `IN` / `OUT`. Ya que son parte de la memoria de E/S, y no comparte el mismo espacio de direcciones con la memoria principal. Son parecidas a la instrucción `mov`, pero en lugar de intercambiar datos entre la CPU y la memoria principal, mueven datos desde la CPU hacia los dispositivos (`out`) o traen datos desde los mismos a la CPU (`in`). El registro `al` es el único que puede utilizarse con estas instrucciones, no se permite utilizar otros registros ni otros tipos de direccionamiento.

Para configurar el PIC de manera que se ejecuta nuestra subrutina ante una interrupción de un dispositivo debemos:

1. Escribimos el interrupt handler (subrutina): debe escribir el valor 20h en el registro EOI del PIC antes de finalizar y terminar con `iret`.
2. Ponemos la dirección de la subrutina en el vector de interrupciones: Elegimos un ID y colocamos la dirección inicial de la subrutina en la dirección de memoria principal: $ID \times 4$. (No elegir un ID correspondiente a interrupciones por software)
3. Ponemos el ID en el registro INT n del PIC que corresponda al dispositivo.

4. Habilitamos las interrupciones en la línea correspondiente al dispositivo que queramos usar, modificando el registro `IMR` del PIC. El bit 7 de la máscara corresponde a la línea 7 mientras que el bit 0 a la línea 0.

Las instrucciones `CLI` y `STI`, respectivamente, deshabilitan y habilitan las interrupciones. Debemos usarlas al programar el PIC, desactivarlas antes y activarlas luego.

Para mejorar la legibilidad, a la hora de referirnos a las direcciones de registros en la memoria de E/S, podemos hacerlo utilizando un nombre, en vez de una dirección. Para esto la directiva `EQU`, permite asignar un valor a una etiqueta, que al compilar el programa, será reemplazada por el valor asignado.

Ejemplo de interrupciones por hardware

Programa que cuenta la cantidad de pulsaciones de la tecla F10.

```
EOI EQU 20h
IMR EQU 21h
INT0 EQU 24h

org 1000H
pulsaciones db 0

org 3000h
; 1. Escribir la subrutina
contar_F10: inc pulsaciones
            mov al, 20h ; enviar el valor 20h al registro EOI
            out EOI, al ; para indicarle al PIC que finalizo
            iret        ; el manejo de la interrupcion

; 2. poner la direccion de la subrutina en la
; posicion 11 del vector (11*4=44, en decimal)
; 11 fue elegido de manera arbitraria
org 44
dw contar_F10 ; no es necesario ponerle una etiqueta

org 2000h
cli; deshabilitar TODAS las interrupciones
; 3. Poner el ID 11 en el registro INT0 (tecla F10)
mov al, 11
out INT0, al
; 4. Habilitar las interrupciones de INT0
mov al, 11111110B
out IMR, al
sti; habilitar las interrupciones que no esten enmascaradas (IMR)

loop: jmp loop ; loop infinito, espera interrupciones
end
```

El timer es otro de los dispositivos de E/S del MSX88. Se utiliza como un reloj despertador para la CPU. Se configura para contar una cantidad determinada de segundos y cuando finaliza la cuenta emite una interrupción. Tiene dos registros `CONT` y `COMP`:

Dirección	Registro	Nombre	Propósito
10h	CONT	Contador	Se incrementa automáticamente, todo el tiempo, una vez por segundo
11h	COMP	Comparador	Contiene el tiempo límite. Cuando <code>CONT = COMP</code> , se dispara la interrupción

Ejemplo de interrupciones por hardware

Programa que imprime el mensaje "Hola" cada 10 segundos, utilizando el Timer.

```

IMR EQU 21h
EOI EQU 20h
INT1 EQU 25h
CONT EQU 10h
COMP EQU 11h
MASCARA_TIMER EQU 1111101B
VALOR_FIN_INTERRUPCION EQU 20h
ID_TIMER EQU 13 ; id de interrupcion

org 1000H
msg db "Hola"

org 3000h
print_msg: mov bx, offset msg
           mov al, 4
           int 7
           mov al, 0 ; Poner el CONT en 0 para que vuelva a contar desde 0
           out CONT, al
           mov al, VALOR_FIN_INTERRUPCION
           out EOI, al
           iret

org 52 ; 13 * 4 = 52d
dw print_msg

org 2000h
cli
mov al, ID_TIMER ; Poner el ID 13 en el registro INT1
out INT1, al
mov al, MASCARA_TIMER ; Habilitar la interrupcion de INT1
out IMR, al
mov al, 10 ; Configurar el COMP del Timer en 10 segundos
out COMP, al
mov al, 0 ; CONT = 0, para que comience a contar desde 0
out CONT, al
sti
loop: jmp loop
end

```

Interrupciones por software

Son invocadas por el programador mediante la instrucción `INT`, seguida por el número de interrupción. Se atiende una subrutina del sistema operativo. Se retorna con la instrucción `IRET`.

Hay 4 tipos de interrupción por software en el MSX88:

- `INT 0`: Terminar el programa.
- `INT 3`: Poner un punto de parada (breakpoint).
- `INT 6`: Lee un carácter de teclado.
 - Se pasa como parámetro en el registro `bx` la dirección de memoria donde será almacenado el carácter leído.
- `INT 7`: Escribe un string en pantalla.
 - Se pasa como parámetro: en el registro `bx` la dirección de memoria donde comienza el string a imprimir. En el registro `al` la cantidad de caracteres.

Ejemplo de interrupciones por software

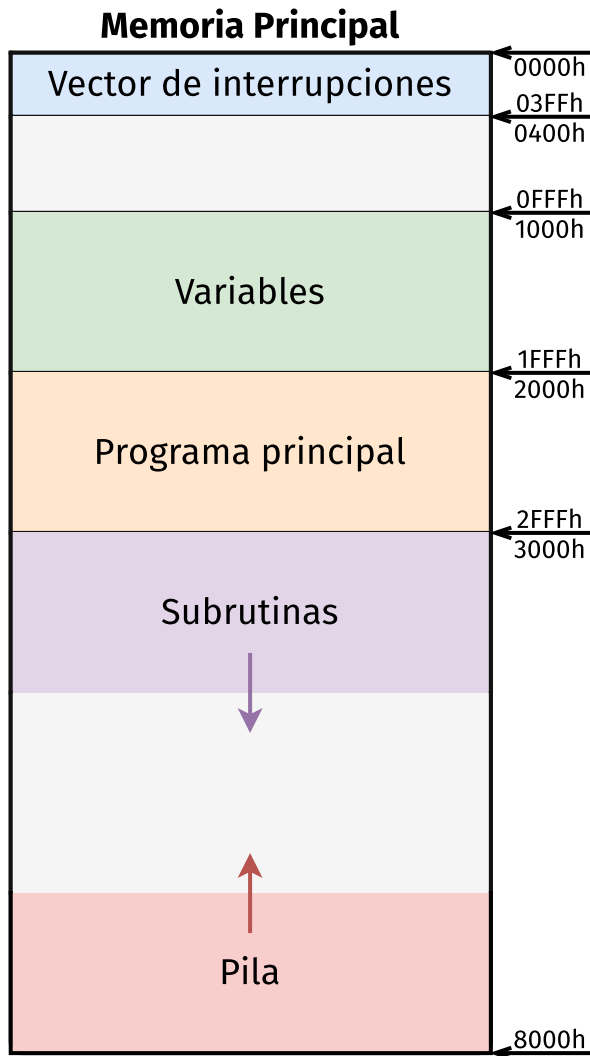
Programa que lee un string de `longitud` caracteres y lo muestra en pantalla.

```
org 1000h
longitud db 10
mensaje db ?

org 3000h
leer_string: cmp al, 0
             jz fin ; no hacer nada si piden leer un string de longitud 0
             push bx
             push ax; preserva bx y ax (y al)
loop:        int 6 ; leer los caracteres
             inc bx; apuntar al siguiente byte con bx
             dec al
             jnz loop; hasta que al quede en 0
             pop ax
             pop bx; restaura bx y ax (y al)
fin:         ret

org 2000h
mov bx, offset mensaje
mov al, longitud
call leer_string; lee el string de longitud al y lo guarda a
partir de bx

int 7
int 0
end
```



TIMER	10h	CONT	Se incrementa una vez por segundo automáticamente.
	11h	COMP	Cuando CONT = COMP dispara una interrupción por la línea INT1 del PIC.
PIC	20h	EOI	Señal. Cuando el PIC recibe 20h en 20h desmarca la línea en interrupción del ISR.
	21h	IMR	Enmascara la línea n, cuando su bit n = 1. Esta línea no podrá interrumpir.
	22h	IRR	Si el bit n = 1, hay una interrupción pendiente de la línea n. Es de solo lectura para la CPU.
	23h	ISR	Si el bit n = 1, se está atendiendo una interrupción de línea n (solo un bit = 1 a la vez). Es de solo lectura.
	24h	INT0	Contiene el ID de interrupción correspondiente a la tecla F10.
	25h	INT1	Contiene el ID de interrupción correspondiente al timer.
	26h	INT2	Contiene el ID de interrupción correspondiente al handshake.
PIO	27h	INT3	Contiene el ID de interrupción correspondiente al DMA.
	30h	PA	Puerto A. Estarán conectadas las llaves (entrada), o la impresora.
	31h	PB	Puerto B. Estarán conectadas las luces (salida), o la impresora.
	32h	CA	El valor del bit n determinará si el bit n del puerto A es de entrada o salida. (0 será salida, 1 será entrada)
HANDSHAKE	33h	CB	El valor del bit n determinará si el bit n del puerto B es de entrada o salida. (0 será salida, 1 será entrada)
	40h		Registro de datos, almacena el carácter a imprimir.
	41h		Registro de estado, análogo a PA con la impresora conectada.

Subsistema de E/S

Definición

Vimos que la arquitectura Von Neumann está basada en tres subsistemas: CPU, Memoria y E/S.

¿Qué es?

El subsistema de E/S provee una manera eficiente de comunicación entre la CPU y dispositivos del exterior. Es la interfaz entre el exterior y el sistema de computo.

Dada la gran variedad de periféricos, con distintas características, no se conectan de manera directa al bus del sistema. Junto con el procesador y el conjunto de módulos de memoria, el tercer elemento clave de un computador es un **conjunto de módulos de E/S**

El subsistema debe ser flexible para permitir:

- Transmisión de diferentes cantidades de datos.
- Rango de velocidades de transmisión muy amplio.
- Diferentes formatos de dato y tamaños de palabra.

Módulo de E/S

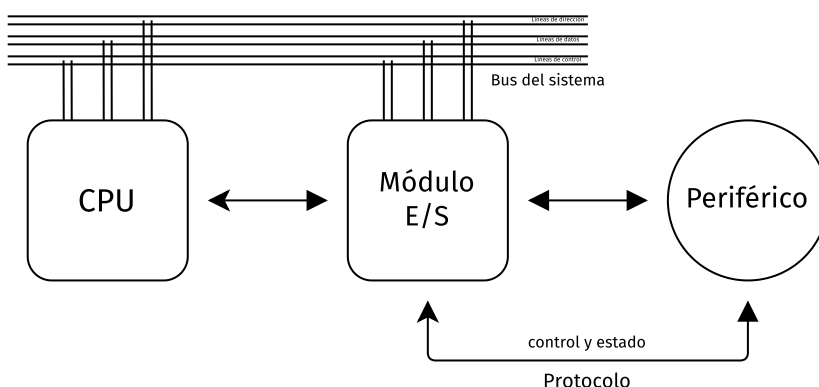
Un dispositivo externo se conecta al computador mediante un enlace a un módulo de E/S. A un dispositivo externo conectado a un módulo de E/S se lo denomina **dispositivo periférico**.

Cada **módulo de E/S** se conecta al bus del sistema y controla uno o más periféricos. Cada módulo contiene la lógica necesaria para permitir la comunicación entre el periférico y el bus.

El funcionamiento de un módulo de E/S permite que el procesador vea a una amplia gama de dispositivos de una forma simplificada.

El módulo de E/S debe ejecutar dos tipos de comunicaciones:

- **Hacia el periférico (externas):**
 - Transferir datos con el periférico.
 - Controlar y temporizar uno o más dispositivos externos.
 - Almacenar temporalmente datos ("buffer").
 - Detectar errores.
- **Hacia el bus (con CPU y memoria):**
 - Interpretar las órdenes que recibe del CPU, generalmente a través del bus de control, y transmitirlas al periférico.
 - Transferir datos con la CPU (registros) y Memoria.
 - Informar a la CPU del estado del periférico.



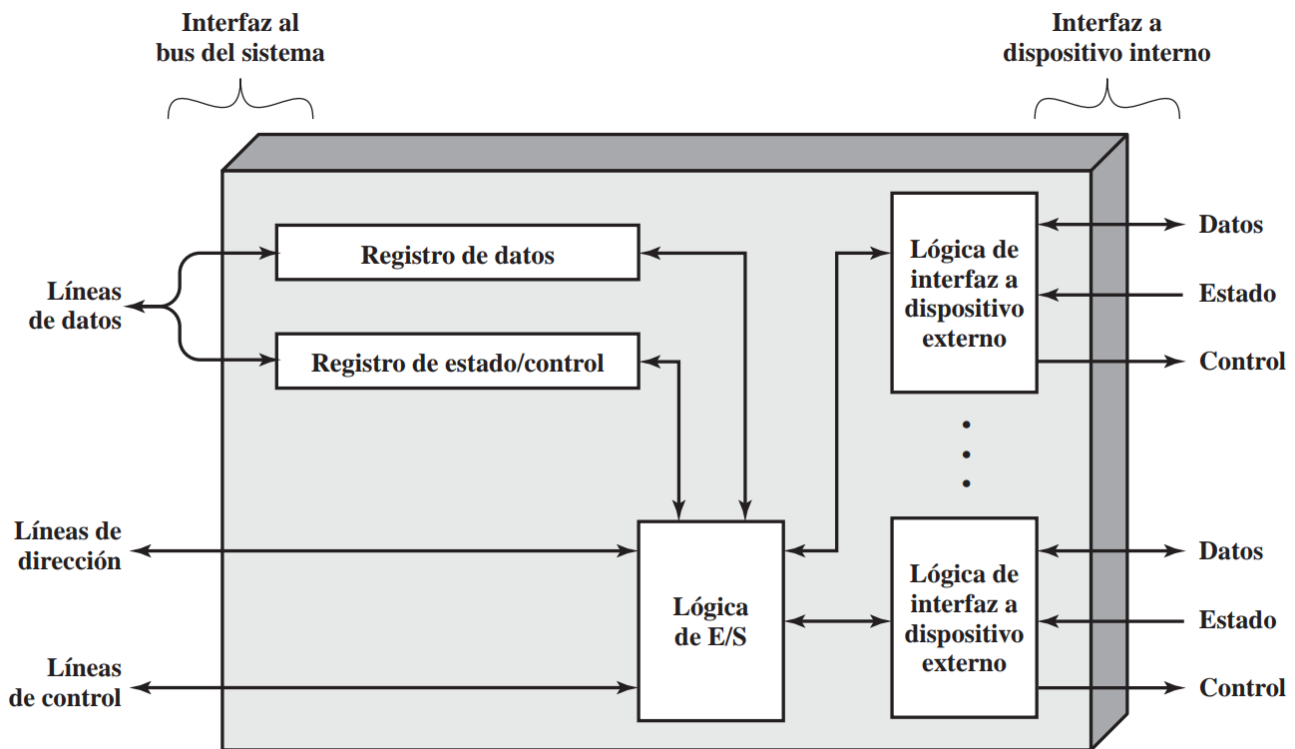


Figura 7.3. Diagrama de bloques de un módulo de E/S.

La lógica que hay en el módulo interactúa con el procesador a través de una serie de líneas de control. Estas son las que utiliza el procesador para proporcionar las órdenes al módulo de E/S. Algunas de las líneas de control pueden ser utilizadas por el módulo de E/S, por ej, para las señales de arbitraje.

El módulo también debe ser capaz de reconocer y generar las direcciones asociadas a los dispositivos que controla. Cada módulo de E/S tiene una dirección única o, si controla más de un dispositivo externo, un conjunto único de direcciones.

Los recursos "visibles" a la CPU (y al programador) del módulo son básicamente registros. Existen dos tipos:

- **Registros de datos:** Intervienen en la transferencia, en ambas vías, entre el sistema de cómputo y el periférico, almacenando información temporalmente.
 - Operación de entrada: lectura de un registro de dato de entrada (registro escrito por el periférico y leído por la CPU).
 - Operación de salida: escritura de un registro de dato de salida (registro escrito por la CPU y leído por el periférico).
- **Registros de control y estado:** controlan las características de la transferencia, y almacenan señales de estado de la comunicación (falla, no listo, etc). Controlan y registran el funcionamiento del módulo, la transferencia y el periférico.
 - Control: adecuar la configuración del módulo para ajustar formatos, sincronizaciones, etc.
 - Estado: registrar el estado operativo del módulo y del periférico.

Tipos de puerto de E/S básicos según la transferencia de datos:

- **Paralelo:** Hay n líneas de datos que transfieren n bits simultáneamente entre el puerto de e/s y el periférico.
 - Ej: impresora paralelo, scanner.

- Se requiere disponer de una conexión mediante un cable que incluya al menos los n bits de datos, lo que lo hace costoso.
- **Serie:** hay sólo una línea de datos para la transferencia entre el puerto de e/s y el periférico.
 - Ej: impresora serie, red ethernet, mouse, teclado.
 - Se requiere una conexión mediante un cable sencillo. Menor costo.
 - Los datos deben serializarse (transmitirse de uno a la vez), lo que en teoría es más lento que transmitir n bits de manera simultanea.

Periférico

En general, todos los periféricos son más lentos que la CPU y memoria.

La conexión con el módulo de E/S se realiza a través del intercambio de **señales de control, de estado y datos**.

- Las señales de **control** determinan la función que debe realizar el dispositivo. Por ej. leer o escribir, indicar el estado o realizar alguna función particular del dispositivo.
- Los **datos** se intercambian como conjuntos de bits enviados a, o recibidos desde, el módulo.
- Las señales de **estado** indican el estado del dispositivo. Por ej. si está listo o si no está listo, indicando si está preparado para la transferencia de datos.
- La **lógica de control** asociada al dispositivo controla su operación en respuesta a las indicaciones del módulo de E/S. El **transductor** convierte las señales eléctricas asociadas al dato a otra forma de energía en el caso de una salida y viceversa en el caso de una entrada. El **buffer** asociado al transductor almacena temporalmente el dato que se está transfiriendo entre el módulo de E/S y el exterior.

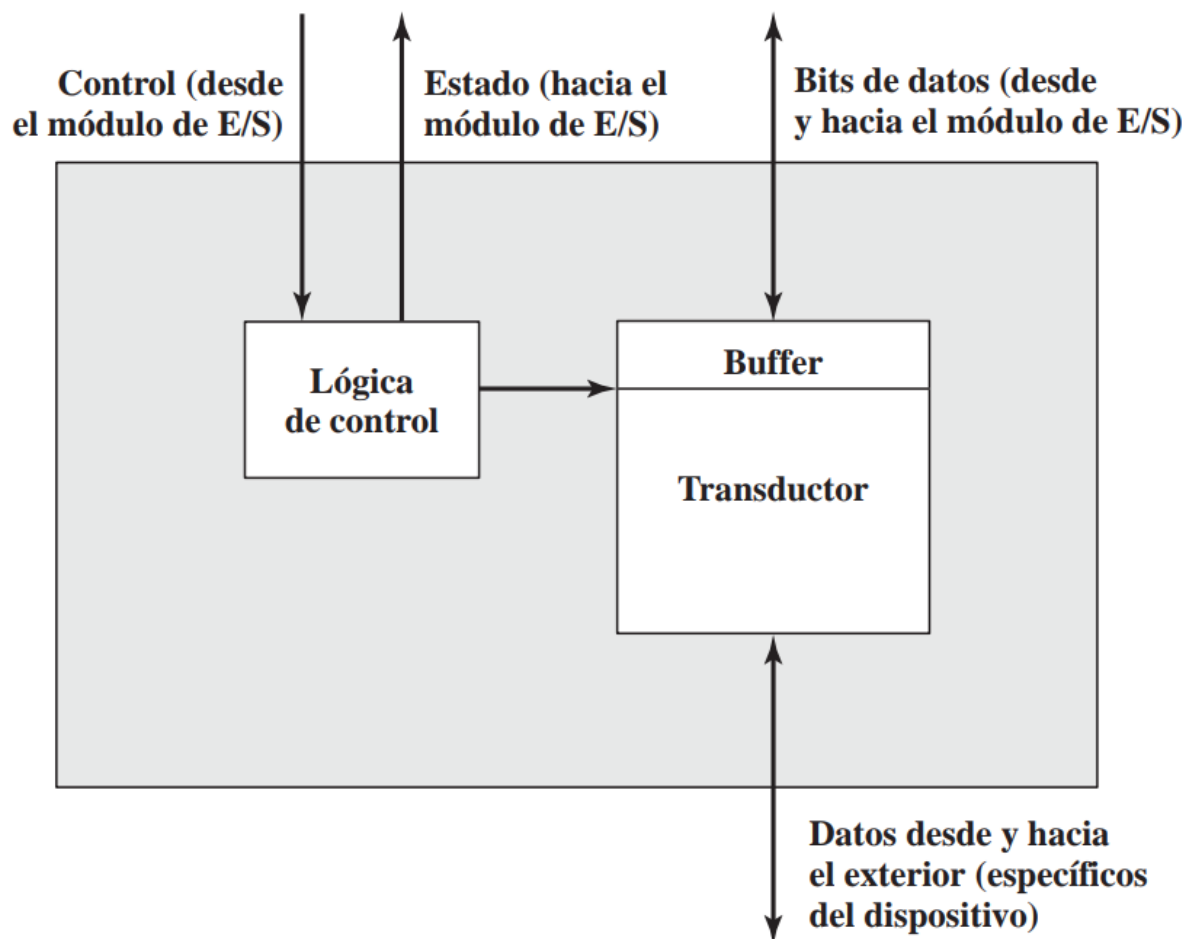


Figura 7.2. Diagrama de bloques de un dispositivo externo.

Implementación

Normalmente, habrá muchos dispositivos de E/S conectados al sistema a través de los módulos de E/S. Cada dispositivo tiene asociado un identificador único o dirección.

Cuando el procesador envía una orden de E/S, la orden contiene la dirección del dispositivo deseado. Así, cada módulo de E/S debe interpretar las líneas de dirección para determinar si la orden es para él.

Cuando el procesador, la memoria principal, y las E/S comparten un bus común, son posibles dos modos de direccionamiento: i/o asignado en memoria y aislado:

- **Espacio de E/S compartido con memoria:** (*memory mapped i/o*)

Una porción de la memoria principal se reserva para ser usada para e/s.

Los registros de los dispositivos E/S y memoria comparten un único espacio de direcciones. Y se comportan idéntico a una memoria de lectura/escritura.

No hay instrucciones específicas para E/S, se utilizan las mismas instrucciones máquina para acceder tanto a memoria como a los dispositivos de E/S.

El amplio repertorio de instrucciones disponibles permite una programación más eficiente. Aunque como desventaja, se utiliza espacio de direcciones de memoria.

- **Espacio de E/S separado de la memoria o E/S aislado:** (*port mapped i/o / isolated i/o*)

Como el MSX88.

Los registros de los dispositivos de E/S y la memoria están en diferentes espacios de direcciones.

Dado que el bus de direcciones es compartido por la memoria y el subsistema de E/S, requiere de señales de control adicionales para identificar a donde está accediendo la CPU, si a la memoria o la E/S.

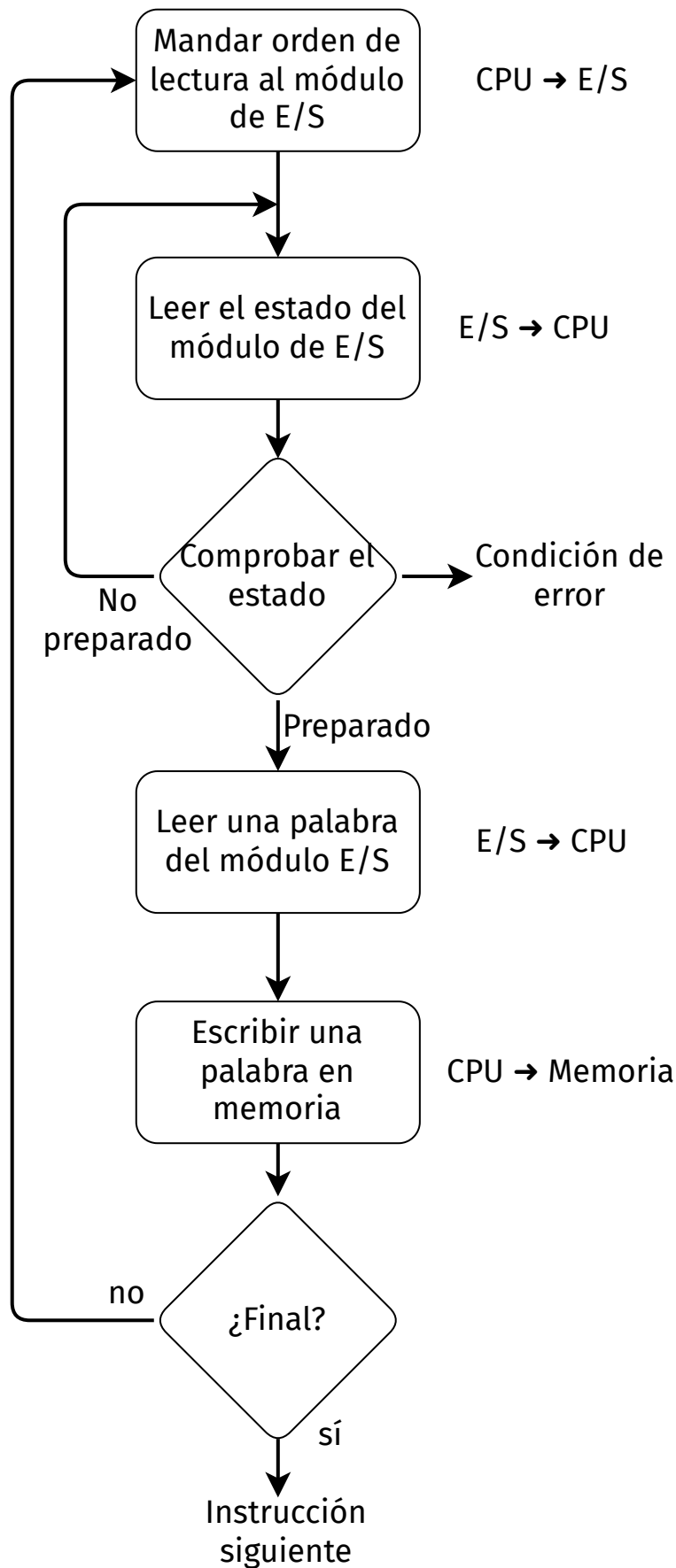
Se utilizan instrucciones específicas de E/S, distintas de las instrucciones de acceso a la memoria.

Técnicas de E/S

Respecto a la gestión de la transferencia de datos entre el computador y el periférico, existen tres estrategias:

	Sin interrupciones	Usando Interrupciones
Transferencia de E/S a memoria a través de la CPU	E/S programada	E/S mediante interrupciones
Transferencia directa de E/S a memoria	-	Acceso directo a memoria (DMA)

E/S programada y espera de respuesta



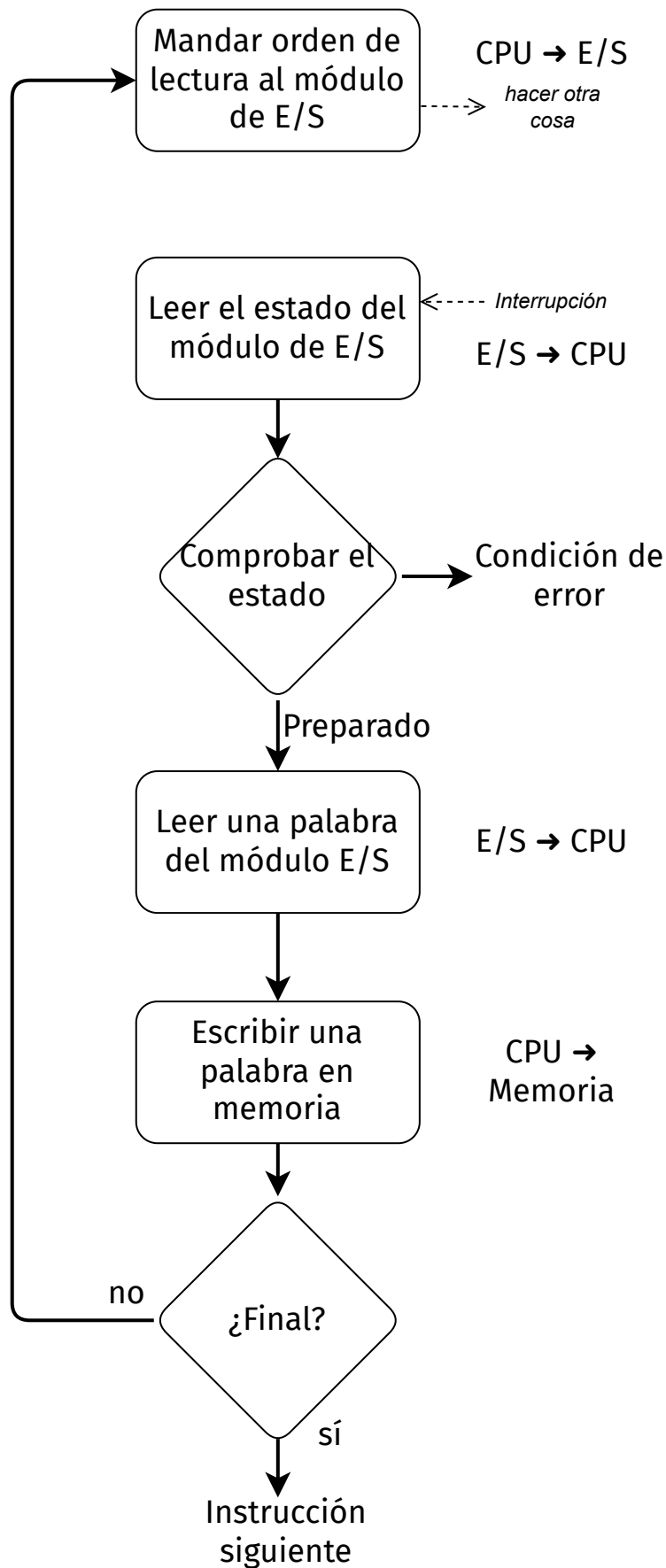
La CPU interviene en la transferencia de cada unidad de información (byte, word) con el módulo. La CPU debe comprobar el estado del módulo periódicamente hasta que finalice la operación de E/S.

Para cada dato que es transferido, la CPU espera que el modulo de E/S termine la operación, típicamente que el dispositivo "acepte" el dato. Durante la espera la CPU permanece ociosa (no

deseable).

1. La CPU lee un registro del módulo para verificar su estado (preparado/no-preparado).
2. Examina el estado del periférico chequeando el bit/bits que identifica dicho estado.
3. Si el dispositivo no está listo, la CPU vuelve al paso 1. (La CPU de esta forma "espera" hasta que el dispositivo esté "preparado", listo para transferir).
4. Cuando el dispositivo está listo, la CPU transfiere un dato hacia o desde el módulo de interfaz.
5. Si hay más datos que transferir vuelve al paso 1.
6. Si se completó la transferencia, termina el servicio de E/S.

E/S programada y administrada por interrupción



La CPU interviene en la transferencia de cada unidad de información (byte, word) con el módulo. Cada vez que el módulo está listo (o completó una transferencia) avisa a la CPU con un pedido de interrupción. Evita que la CPU compruebe el estado del módulo esperando el tiempo que el periférico tarde en estar listo y continuando con la ejecución de otra tarea.

Secuencia de acciones para una transferencia de entrada:

1. La CPU inicia la operación de lectura (entrada) enviando una orden de lectura (read) al módulo de E/S.
2. El módulo de e/s solicita el dato al periférico.
3. El periférico busca el dato, mientras la CPU continúa con sus tareas.
4. Cuando el módulo e/s tiene el dato enviado por el periférico emite un pedido de interrupción a la CPU.
5. La CPU detecta el pedido, interrumpe el proceso, y bifurca el servicio de la interrupción.
6. Durante la interrupción, la CPU lee el dato desde el módulo de E/S.
7. La CPU retorna la tarea interrumpida.

Para **identificar el dispositivo del cuál se origina la interrupción** hay cuatro técnicas que se detallan en Identificar origen de la interrupción.

Las operaciones de E/S administradas por interrupción son más eficientes que las programadas con espera, aunque ambas técnicas requieren la intervención de la CPU, lo cual presenta dos problemas:

- La velocidad de transferencia depende de la capacidad de la CPU de atender estas tareas.
- La CPU puede permanecer ocupada mucho tiempo durante la operación, sin poder hacer otras tareas. Cuanto mayor volumen de datos, mayor tiempo de ocupación de la CPU.

Por ejemplo, en una transferencia de un bloque de datos utilizando E/S programada, el procesador se dedica a la tarea de la E/S y puede transferir datos a alta velocidad al precio de no hacer nada más. La E/S con interrupciones libera en parte al procesador, a expensas de reducir la velocidad de E/S. Ambos métodos tienen un impacto negativo tanto en la actividad del procesador como en la velocidad de transferencia de E/S.

Ejemplos: sobre la cpu

Se tiene un procesador con un reloj de 200 Mhz (período de 5ns). En promedio necesita 2 ciclos de reloj por instrucción. Este parámetro se conoce como CPI, en este caso CPI = 2. Es decir, cada instrucción tarda en promedio: $2 \times 5\text{ns} = 10\text{ns} = 10^{-8}$. Por lo tanto el procesador tiene una capacidad de procesamiento de:

Nº de instrucciones por segundo: $\frac{1}{\text{tiempo de ciclo de instrucción}} = \frac{1}{10^{-8}} = 100 \text{ millones de instrucciones por segundo (100 Mips)}$

Ejemplo 1: transferencia a una impresora

Se tiene que imprimir (operación de salida) un archivo de 10 Kbytes en una impresora laser de 20 páginas por minuto. Ya que una página tiene (estimado) 3000 caracteres y cada caracter es un byte, la impresora imprime:

$20\text{ppm} = 20 \text{ pag/min} \times 3000 \text{ car/pag} = 60000 \text{ car/min}$.

Es decir, la velocidad de transferencia de la impresora es de:

$vt = 60000\text{car/min} = 60000\text{car}/60\text{seg} = 1000\text{car/s} = 1\text{Kbyte/s}$

- E/S con espera de respuesta:
 - La CPU entra en un bucle y envía un nuevo byte cada vez que la impresora está preparada para recibirlo.

- Ya que la impresora tarda 1 seg. en imprimir 1 Kbyte, necesitará 10 segundos para 10 Kbyte,
 - En los 10 seg. que la CPU quedará ocupada con la operación de E/S, dada la velocidad de la CPU de 100 MIPS, podría haber ejecutado $100\text{MIPS} \times 10\text{seg} = 1000$ millones de instrucciones.
 - Administrada con interrupciones:
 - La impresora genera una interrupción cada vez que está preparada para recibir un nuevo byte.
 - Si la gestión de la interrupción (ATI) requiere 10 instrucciones (entre las que se incluyen salvar contexto, comprobar estado, transferir byte, restaurar contexto, y retornar), entonces requiere ejecutar 10000 veces la ATI para transferir 10Kbytes
 - El tiempo que dura el servicio de la transferencia es aproximadamente el requerido por $= 10000 \times 10_{\text{inst}}$
- las 10000 ATI: $= 10000 \times 10 \times 10^{-8}\text{seg}$
 $= 10000 \times 10^{-8} = 10^{-3}\text{seg} = 0,001\text{seg}$
- Entonces la CPU está ocupada 0,001seg, de los 10 segundos que dura la transferencia.

Ejemplo 2: transferencia a un disco

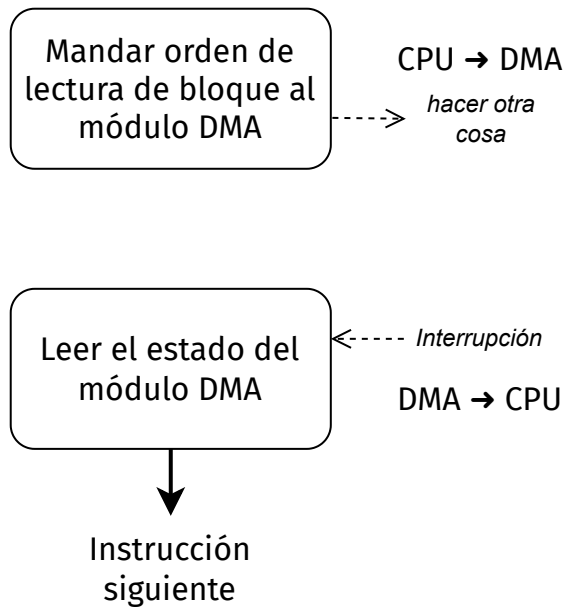
Se tiene que transferir un archivo de memoria a disco de 10 Mbytes

Consideramos que el disco posee una velocidad de transferencia de 10MB/s (1 byte cada 10^{-7} o 100 nanosegundos). Osea el tiempo total de transferencia será de 1seg.

- E/S con espera de respuesta:
 - La CPU entra en un bucle y envía un nuevo byte cada vez que el disco está preparado para recibirlo
 - El disco transfiere necesita 1s para recibir un archivo de 10 Mbytes.
 - Entonces la CPU estará ocupada con la operación de E/S 1s. Dada la velocidad del CPU de 100MIPS, podría haber ejecutado 100 millones de operaciones en ese tiempo.
- Administrada con interrupciones:
 - El disco genera una interrupción cada vez que está listo para recibir un nuevo byte.
 - Si la gestión de la interrupción (ATI) requiere 10 instrucciones, entonces para transferir 10Mbytes se ejecutará 10^7 veces la ATI.
 - Entonces la CPU está ocupada con la operación 1s, ya que es el tiempo que tarda el disco en transferir 10Mbytes.
- Conclusión: en ambos casos la CPU se encuentra ocupada administrando la transferencia la totalidad del tiempo.

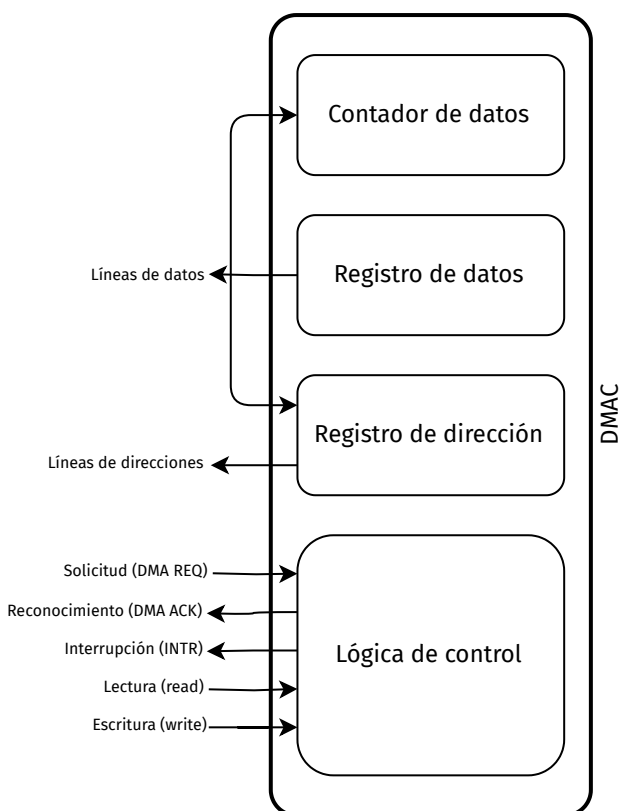
Cuando hay que transferir grandes volúmenes de datos, se requiere una técnica más eficiente: el acceso directo a memoria (DMA).

E/S con acceso directo a memoria (DMA)



El **acceso directo a memoria (DMA)** es una técnica de transferencia de datos entre periférico y memoria sin intervención directa de la CPU.

Requiere un módulo adicional en el bus del sistema. El **DMAC o controlador de DMA**.

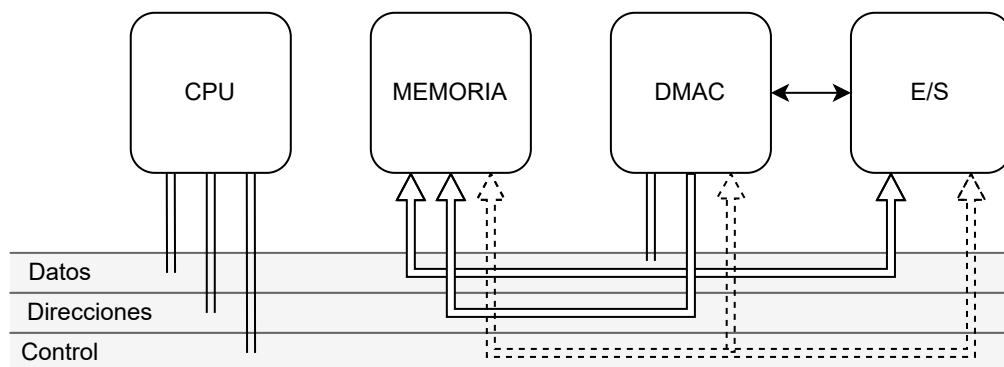


El procesador solo interviene al comienzo y al final de la transferencia.

Cuando el procesador desea leer o escribir un bloque de datos, envía una orden al módulo de DMA:

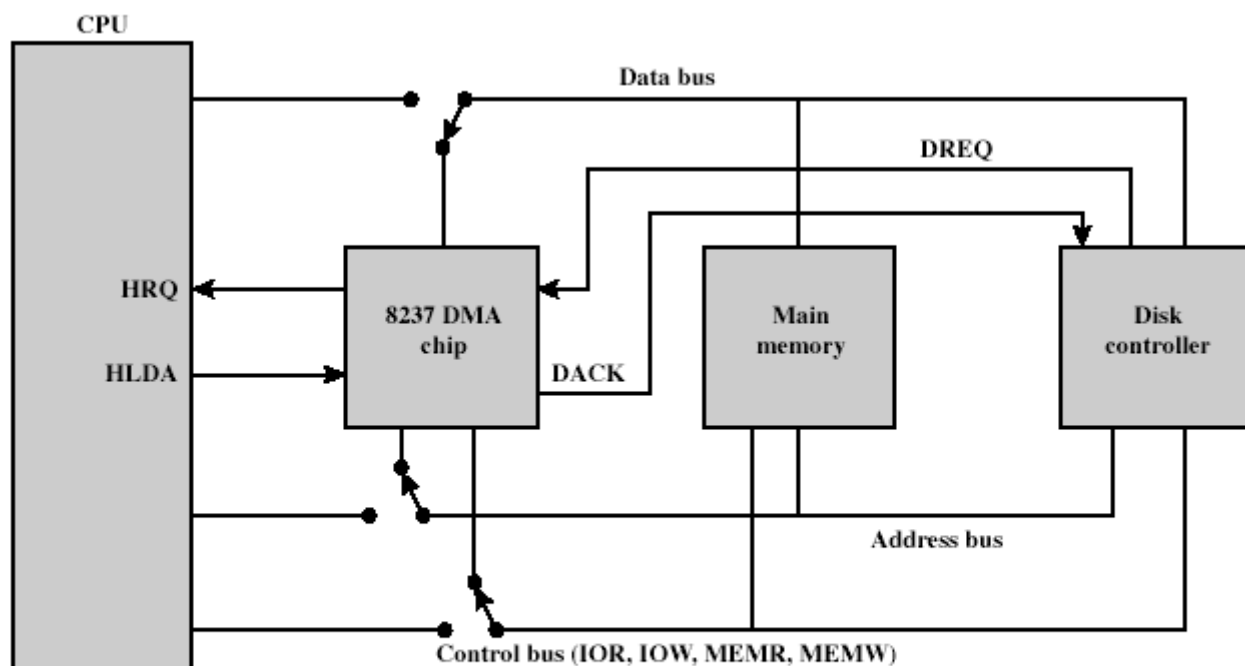
- Si se solicita una lectura o una escritura, utilizando la línea de control de lectura o escritura entre el procesador y el módulo de DMA.
- La dirección del dispositivo de E/S en cuestión, indicada a través de las líneas de datos.

- La posición inicial de memoria a partir de donde se lee o se escribe, indicada a través de las líneas de datos y almacenada por el módulo de DMA en su registro de direcciones.
- El número de palabras a leer o escribir, también indicado a través de las líneas de datos y almacenado en el registro de cuenta de datos.



La CPU entrega el bus al DMAC, se desconecta lógicamente del mismo, actuando este último de "bus master" durante la transferencia. El procesador continúa con otro trabajo, delegando la operación al DMAC. El módulo de DMA transfiere el bloque completo de datos, palabra a palabra, directamente desde o hacia la memoria.

Cuando la transferencia se ha terminado, el módulo de DMA envía una señal de interrupción al procesador.



DACK = DMA acknowledge
DREQ = DMA request
HLDA = HOLD acknowledge
HRQ = HOLD request

El proceso de transferencia consta de tres pasos:

1. Fase de Inicialización.

- La CPU debe configurar el módulo de E/S y DMAC con los parámetros de la transferencia:
 - Inicialización interfaz E/S:
 - Tipo de transferencia.

- Configuración de periférico.
 - Otra información de control, por ejemplo en un disco se especifica el número de pista.
 - **Inicialización DMAC:**
 - N° de bytes o palabras a transferir.
 - Tipo de transferencia (escritura/lectura)
 - Direcciones de memoria inicial para la transferencia.
 - Otra información sobre la transferencia.
2. Fase de ejecución de la transferencia.
- Cuando el periférico está listo, pide al DMAC iniciar la transferencia mediante una señal física.
 - Cuando el DMAC recibe el pedido, pide el control del bus mediante alguna señal específica a la CPU.
 - Cuando reconoce el pedido de DMA, la CPU encarga (libera) el bus y se "desconecta" lógicamente del mismo.
 - La CPU avisa al DMAC que liberó el bus mediante otra señal especial.
 - Al liberar la CPU, el DMAC toma el control del bus y ejecuta la transferencia hasta terminarla:
 - El DMAC avisa al periférico que puede iniciar la transferencia.
 - El periférico comienza a transferir datos, a través del bus, con la memoria, uno a la vez.
 - La transferencia implica que:
 - Bus master: DMAC + periférico
 - Bus slave: memoria
 - Una vez transferidas cada palabra, se actualizan los registros del DMAC:
 - Número de bytes faltantes (o cuenta los que se transfirieron).
 - Próxima dirección de memoria donde guardar el dato.
 - Cuando el número de bytes faltantes es igual a 0, transfirió todos los datos y terminó la transferencia.
3. Fase de finalización y análisis de la transferencia.
- Una vez finalizada la fase 2, el DMAC libera el bus y le avisa a la CPU por medio de una señal física.
 - La CPU retoma control de bus.
 - El DMAC suele activar, además, una señal de interrupción para indicar a la CPU la finalización de la operación de E/S solicitada.
 - Mediante esta interrupción, la CPU verifica el resultado de la transferencia vía registros internos del DMAC, como:
 - Transferencia OK/fallida
 - Errores? tipos de errores?
 - Estado periférico.

La principal ventaja del DMA es la eficiencia, dado que la CPU se libera durante la transferencia. La principal desventaja se origina en el uso del bus. Como las transferencias DMA pueden tener mayor prioridad que la CPU, puede degradar el rendimiento de la CPU por hacer uso intensivo del bus.

Sin embargo, la CPU no necesita necesariamente del bus todo el tiempo, ya que:

- En computadoras con cache: la mayor parte del tiempo, la CPU lee instrucciones de la cache, por lo que no necesita hacer uso del bus de memoria. El DMAC puede aprovechar estos intervalos en

los que la CPU está leyendo instrucciones de la cache, y no necesita del bus, para realizar transferencias.

- En computadoras sin cache: el procesador no utiliza el bus en todas las fases de la ejecución de una instrucción. El DMAC puede aprovechar las fases de ejecución de una instrucción en las que la CPU no hace uso del bus.

Implementación

Existen dos formas:

Por ráfagas (*burst*):

- Es la explicada previamente. El DMAC solicita el control del bus a la CPU.
- Cuando la CPU concede el bus, el DMAC no lo libera hasta haber finalizado la transferencia de todo el bloque de datos completo.
- Como ventaja, la transferencia se realiza de forma muy rápida, limitada por la velocidad del periférico.
Como desventaja, durante el tiempo que dura la transferencia la CPU no puede utilizar el bus con memoria, lo que puede degradar el rendimiento del sistema.

Por robo de ciclo (*cycle-stealing*):

- El DMAC solicita el control del bus a la CPU.
- Cuando la CPU concede el bus al DMAC, se realiza la transferencia de una única palabra y luego el DMAC libera el bus.
- El DMAC solicita el control del bus tantas veces necesite, hasta finalizar la transferencia del bloque completo. Así el uso del bus se comparte entre la CPU y el DMAC.
- Como ventaja, no degrada tanto el rendimiento del sistema y la CPU.
- Como desventaja puede tardar un poco más de tiempo.
- Dado que la toma y liberación del bus por parte de la CPU no es una interrupción, es decir que no debe guardar el contexto, no está interrumpiendo su tarea. Si bien el trabajo de la CPU es más lento (que si no estuviera presente la transferencia por DMA), no será tanto como si ella tuviera que estar desconectada del bus todo el tiempo.
- En general, para transferencia de E/S de múltiples palabras, la técnica por robo de ciclo es la más eficiente.

Se denomina robo de ciclo ya que el DMAC "roba" un ciclo de bus.

Evolución de E/S

Es uno de los aspectos que más ha evolucionado a lo largo de la historia del sistema de computo. Podemos resumir sus etapas como:

1. La CPU controla directamente al periférico.
2. Se añade un módulo de E/S. La CPU utiliza E/S programada y espera de respuesta.
3. Como el paso anterior pero se emplea E/S programada con interrupciones. Mejora la eficiencia.
4. El módulo de E/S tiene acceso a memoria a través del DMA. Ahora la transferencia solo implica la CPU al comienzo y final.
5. Se mejora el módulo de E/S para que se comporte como un procesador en sí, con un repertorio de instrucciones especializado orientado a la E/S. Este procesador de E/S puede captar e interpretar

instrucciones, entonces la CPU puede especificar una secuencia de actividades de E/S y recibir una interrupción cuando sean finalizadas.

6. El módulo de E/S tiene una memoria local propia y es, de hecho, un computador en sí. Con esta arquitectura, se puede controlar un conjunto grande de dispositivos de E/S con la mínima intervención de la CPU.

Los módulos capaces de ejecutar instrucciones, en la etapa 5 y 6, son denominados **canal de E/S** o **procesador de E/S**.

cada vez más funciones de E/S se realizan sin la intervención de la CPU

Canales de E/S

El **canal de E/S** representa una ampliación del concepto de DMA.

En un computador con canal de E/S, este ejecuta las instrucciones de E/S, en lugar de la CPU, lo que le confiere un control completo sobre las operaciones de E/S.

La CPU inicia una transferencia de E/S indicando al canal de E/S que debe ejecutar un programa de la memoria. El programa especifica el dispositivo o dispositivos, el área o áreas de memoria para almacenamiento la prioridad, y las acciones a realizar en ciertas situaciones de error. El canal de E/S sigue estas instrucciones y controla la transferencia de datos.

Son comunes dos tipos básicos de canales de E/S:

Canal selector:

- El canal controla varios dispositivos de alta velocidad, de a uno por vez, selecciona un dispositivo y efectúa la transferencia sobre el dispositivo seleccionado.
- Cada dispositivo tiene asociado un controlador o módulo de E/S que lo maneja.
- El canal de E/S ocupa el lugar de la CPU en el control del módulo de E/S.

Canal multiplexor:

- El canal controla varios dispositivos de alta velocidad, incluso simultáneamente.
- El canal multiplexa la atención entre los dispositivos seleccionados. El multiplexado puede ser:
- Multiplexor de bytes: acepta y transmite de a caracteres (para periféricos más lentos).
- Multiplexor de bloques: intercala bloques de datos desde distintos dispositivos (para periféricos más rápidos).

E/S en el MSX88

Retomando de lo explicado en [Interrupciones](#), la memoria de E/S en el MSX88 es una memoria de 256 bytes, separada de la principal. Por esto es que se usan las instrucciones `in` y `out`. El PIC se encarga de multiplexar los pedidos de interrupción provenientes de múltiples dispositivos.

El **programmed input-output (PIO)** es un módulo que hace de interfaz para conectar dispositivos genéricos al CPU.

Cuenta con dos puertos de 8 bits (A y B) programables. Los registros disponibles son:

- **PA**: dirección `30h` de la memoria E/S. Puerto A.

- **PB**: dirección 31h de la memoria E/S. Puerto B.
- **CA**: dirección 32h de la memoria E/S. Configuración del puerto A.
- **CB**: dirección 33h de la memoria E/S. Configuración del puerto B.

Todos son registros de un byte. El valor del bit n del registro de configuración, corresponde al modo (entrada/salida) del bit n del puerto que configura. Si el bit es 0 será de salida, y si es 1 de entrada.

El PIO puede estar conectado a llaves y luces o a una impresora.

Las llaves estarán conectadas al puerto **PA** del PIO, y son dispositivos de entrada (**CA** deberá ser 1 para cada bit). Al cambiar el estado de las llaves, se verá reflejado en **PA**.

Las luces están conectadas al puerto **PB** del PIO, y son dispositivos de salida (**CB** deberá ser 0 para cada bit). Los cambios en **PB** se verán reflejados en las luces.

Ejemplo:

Encender las luces con las llaves

```
PA equ 30h
CA equ 32h
PB equ 31h
CB equ 33h

org 2000h
mov al, 0ffh ; configurar switches (entrada)
out CA, al

mov al, 00h ; configurar leds (salida)
out CB, al

loop: in al, PA
      out PB, al
      jmp loop

end
```

La impresora recibe un carácter por vez, y lo imprime en el papel. Es muy lenta, por lo que almacena hasta 5 caracteres en un buffer (5 bytes) mientras imprime. Si se le envía otro caracter cuando el buffer está lleno, se pierde.

La impresora estará conectadas a ambos puertos **PA** y **PB**. En **PB** se almacenará el caracter a imprimir. En **PA** solo hace uso de los dos bits menos significativos. El bit 1 es el flag *busy*, que será 1 si el buffer está lleno (bit de entrada), el bit 0 es el *strobe*. La impresora leerá el valor de **PB** en el flanco ascendente del bit *strobe* (0 → 1).

strobe
 PA = 0 0 0 0 0 0 0 0
busy

Para imprimir:

1. Configurar los puertos PA y PB. Con CA = 11111101 y CB = 00000000.
2. Iterar tantas veces como caracteres se quieran imprimir:
 1. Verificar el flag *busy* en PA, que el buffer no esté lleno.
 2. Escribir el caracter en PB.
 3. Poner *strobe* en 0 y luego en 1.

Ejemplo

Imprimir una cadena

```
PA equ 30h
CA equ 32h
PB equ 31h
CB equ 33h

org 1000h
cadena db "ORGANIZACION Y ARQUITECTURA DE COMPUTADORAS"
cadena_fin db ?

org 2000h
mov al, 11111101b
out ca, al
mov al, 00h
out cb, al

mov cx, offset cadena_fin
mov bx, offset cadena

print:  cmp cx, bx
        jz fin

        in al, PA
        and al, 00000001b
        jnz print

        mov al, [bx]
        out PB, al

        inc bx

        in al, PA
        and al, 0FDh
        out PA, al
        or al, 2h
        out PA, al

        jmp print

fin:    int 0

end
```

El **handshake** es un dispositivo diseñado con el fin de facilitar la comunicación con la impresora. Hace uso de dos registros de 8 bits en la memoria de E/S, el registro de datos en la dirección **40h** y el registro de estado en **41h**. En el registro de datos se almacenará cada carácter a imprimir. Cada vez que la CPU escriba sobre ese registro, el handshake se encargará de realizar un flanco ascendente en el **strobe** para que el carácter se imprima automáticamente.

La posición de los bits *strobe* y *busy* es análoga a la impresora por PIO. La diferencia es que el bit de *busy* no puede ser modificado por la CPU y el bit de *strobe* siempre es 0. Si la CPU trata de escribir un 1 en el bit de *strobe*, este causará un flanco ascendente en el *strobe*, enviando lo almacenado en el registro de datos, y el handshake lo volverá a 0 automáticamente.

Además, el bit más significativo del registro de estado habilita/inhabilita las interrupciones. Si este bit es 1, mientras la impresora no esté ocupada (*busy*= 0), el Handshake disparará una interrupción por hardware. Está conectado al puerto *INT2* del PIC.

Handshake, 41h : 0 0 0 0 0 **0** **0**

habilitar busy
interrupciones

Para imprimir por *polling* (consulta de estado):

1. Iterar tantas veces como caracteres se quieran imprimir:
 1. Verificar el flag *busy* en 41h, que el buffer no esté lleno.
 2. Escribir el caracter en 40h.

Para imprimir por interrupciones:

Imprimir una cadena por handshake, polling

1. Configurar el PIC, línea INT2.
2. Activar las interrupciones en el registro de estado del handshake.
3. Iterar tantas veces como caracteres se quieran imprimir:
 1. Verificar el flag *busy* en 41h, que el buffer no esté lleno.
 2. Escribir el caracter en 40h.

Ejemplo

Imprimir una cadena por handshake, *polling*

```

HS_ESTADO equ 41h
HS_DATO equ 40h

ARQUITECTURA DE COMPUTADORAS"

org 1000h
cadena db "ORGANIZACION Y

cadena_fin db ?

org 2000h
mov cx, offset cadena_fin
mov bx, offset cadena

print:      cmp cx, bx

            jz fin

```

```

                                in al, HS_ESTADO
                                and al, 00000001b
                                jnz print

                                mov al, [bx]
                                out HS_DAT0, al

                                inc bx

                                jmp print

fin:                                int 0

                                end

```

Ejemplo

Imprimir una cadena por handshake, interrupciones

```

HS_ESTADO equ 41h
HS_DAT0 equ 40h
EOI equ 20h
IMR equ 21h
INT3 equ 26h
INT3_ID equ 11

org 44
dw print

org 1000h
cadena db "ORGANIZACION Y ARQUITECTURA DE COMPUTADORAS"
cadena_fin db ?

org 2000h
mov cx, offset cadena_fin
mov bx, offset cadena

cli
mov al, 11111011b
out IMR, al
mov al, INT3_ID
out INT3, al
in al, HS_ESTADO
or al, 10000000b
out HS_ESTADO, al
sti

loop: cmp bx, cx
      jz fin
      jmp loop
fin:  in al, IMR
      or al, 00000100b
      out IMR, al

```

```

int 0

    org 3000h
print: in al, HS_ESTADO
    and al, 00000001b
    jnz print
    cmp bx, cx
    jz fin

    mov al, [bx]
    out HS_DAT0, al
    inc bx
    mov al, EOI
    out EOI, al
    iret

end

```

RISC - Reduced Instruction Set Computing

Contexto histórico

Desde el desarrollo del computador de programa almacenado, alrededor de 1950, hubieron pocas innovaciones significativas, algunas fueron:

- **El concepto de familia:** introducido por IBM en su System/360 en 1964, y seguido poco después por DEC, en su PDP-8. El concepto de familia separa la arquitectura de una máquina de su implementación. Se ofrece un conjunto de computadores, con distintas características en cuanto a precio/prestaciones, que presentan al usuario la misma arquitectura. Las diferencias en precio y prestaciones se deben a las distintas implementaciones de la misma arquitectura.
- **Unidad de control microprogramada:** propuesta por Wilkes en 1951, e introducida por IBM en la línea S/360 en 1964. La microprogramación facilita la tarea de diseñar e implementar la unidad de control y da soporte al concepto de familia.
- **Memoria caché:** introducida comercialmente por primera vez en el IBM S/360 Model 85 en 1968. La introducción de este elemento en la jerarquía de memoria mejoró las prestaciones de manera espectacular.
- **Segmentación de cauce:** una manera de introducir paralelismo en la naturaleza secuencial de un programa constituido por instrucciones máquina.
- **Múltiples procesadores:** esta categoría cubre diferentes organizaciones y objetivos.
- **Arquitectura RISC.**

Uno de los aspectos que evolucionó de manera más visible es el de los **lenguajes de programación**:

El desarrollo de la fabricación de circuitos integrados de muy alta escala de integración (VHSI), hizo posible incorporar más funcionalidades en el mismo chip, mejorando la formance. A medida que disminuía el costo del hardware, aumentaba el costo del software. Por la escasez de programadores, junto con otros inconvenientes como la falta de fiabilidad (ya que pueden aparecer errores incluso años después de que un programa sea escrito), la industria desarrolla los lenguajes de alto nivel o **HLL** (High Level Languages).

Esta solución ocasionó otro problema relacionado con el salto o **gap semántico**, la diferencia entre las operaciones que proporciona un HLL y las que proporciona la arquitectura del sistema de cómputo. Los diseñadores, con el fin de acotar la brecha, respondieron con grandes repertorios de instrucciones, docenas de modo de direccionamiento, y secuencias de HLL implementadas en hardware. Esto facilitó el trabajo de escribir compiladores y mejoró la eficiencia a la hora de ejecutar código, ya que las secuencias complejas se pueden implementar en microcódigo, dando lugar a lenguajes HLL más sofisticados.

¿Por qué RISC?

Durante años, se hicieron estudios sobre la arquitectura CISC, con el fin de determinar las características y patrones de ejecución de las instrucciones máquina generadas por programas escritos en HLL.

Se realizaron estudios estáticos y dinámicos:

- Los **estudios estáticos** computaban cantidades basados en el código objeto ejecutable. Es una forma sencilla de obtener métricas, pero el problema es que no tienen en cuenta el flujo del programa ejecutado, y pueden resultar bastante equivocados. Por ejemplo, la cantidad de instrucciones que componen un lazo no tiene nada que ver con las veces que se ejecuta.
- Los **estudios dinámicos** computan cantidades en base a la ejecución de diferentes programas (típicamente programas de prueba o "benchmarks"). Son más realistas porque los resultados tienen en cuenta la situación real por la que pasan los procesadores.

Los aspectos cuyo cálculo tiene interés son:

- **Repertorio de instrucciones:** determinar el uso de las instrucciones y, por lo tanto, su interacción con la memoria. Se concluyó:
 - Las sentencias de asignación son las más numerosas, seguidas por las de llamadas a procedimientos y las sentencias condicionales.
 - Cuando se hace la ponderación basado en la cantidad de instrucciones de máquina que resuelven las sentencias de alto nivel, las sentencias condicionales y llamadas a procedimientos son las que requieren más instrucciones de máquina.
 - Cuando se hace la ponderación basado en la cantidad de accesos a memoria, siguen predominando las sentencias condicionales y llamadas a procedimiento.
 - Los accesos a memoria tienen mucho impacto en el tiempo de ejecución del programa. Por lo tanto, los llamados a procedimiento consumen mucho tiempo y tiene que hacerse lo más eficiente posible para optimizar los accesos a memoria.
- **Uso de los operandos:** determinar cuales eran los tipos de datos usados y su frecuencia de uso, la organización de memoria para almacenarlos y los modos de direccionamiento para acceder a ellos. Se concluyó:
 - En algunos estudios se analizó el comportamiento en llamadas y retornos a procedimientos:
 - En un alto porcentaje (>98%) se pasan menos de 6 datos.
 - En un alto porcentaje (>92%) la cantidad de variables locales es menor a 6.
 - En general el nivel de anidamiento es menor a 7.
 - La mayoría de los programas tienen una secuencia corta de llamadas seguida por la secuencia de retornos.
 - La mayoría de las variables son locales.
 - Las referencias a operandos están muy localizadas.

- **Secuencia de ejecución de las instrucciones:** determinar el impacto de disponer de un repertorio de instrucciones largo y complejo, en la Unidad de Control, y en el cauce de datos que tiene relación con la eficiencia en la ejecución de una secuencia de instrucciones:
 - La mayoría de los programas tienen una secuencia corta de llamadas seguida por la secuencia de retornos.
 - La mayoría de las variables son locales.
 - Las referencias a operandos están muy localizadas.
 - Además, en otros estudios se observó:
 - Uso de variables locales (80%) del procedimiento.
 - Uso de punteros del tipo variable local y escalar para manejo de estructuras de datos.
 - Uso intensivo de registros.

Finalmente, se concluyó luego de muchos estudios:

1. Se necesita optimizar el tiempo de ejecución de las instrucciones más usadas y las que consumen más tiempo. Los llamados y retornos de subrutina son las acciones que más tiempo consumen.
2. Se debe ajustar el cauce de datos para resolver las instrucciones más usadas lo más rápido posible. Debido a la alta proporción de instrucciones de salto condicional y de llamada a procedimientos, un cauce de instrucciones sencillo será ineficiente. Esto se pone de manifiesto como una gran proporción de instrucciones que se precaptan, pero que nunca llegan a ejecutarse.
3. Simplificación de las instrucciones, para resolverlas más rápidamente.
4. Usar un gran número de registros, o un compilador que optimice su tratamiento. Su finalidad es optimizar las referencias a operandos reduciendo los accesos a memoria dada la alta proporción de sentencias de transferencia (asignación) y la localidad y predominio de datos escalares^[3].

Los estudios del comportamiento de la ejecución de los programas escritos en lenguajes de alto nivel, condujeron a un nuevo enfoque a la hora de diseñar un procesador: **RISC (Reduced Instruction Set Computer)**.

Características

Conceptualmente, los primeros procesadores RISC surgieron a mediados de los 80 en las Universidades de Berkeley y Stanford.

Los primeros RISC tenían, entre otras, las siguientes características:

- Típicamente máquinas tipo Harvard.
- Repertorio de instrucciones reducido y básico. Los CISC tienen muchas más instrucciones.
- Formato de instrucción fijo y sencillo. En los CISC es muy variable.
- Los CISC tiene una buena variedad de modos de direccionamiento, mientras que los RISC son mínimos, sencillos.
- Unidad de control cableada (sin microcódigo). En los CISC es típicamente microprogramada.
- Diseño del cauce optimizado para ejecutar una instrucción por ciclo (uso preciso, o eficiente, de la segmentación)
- Máquina tipo load/store: acceso a memoria solo con instrucciones de movimiento de datos.
- Instrucciones aritméticas y lógicas solo entre registros.
- Uso intensivo de registros (de uso general) con optimización por software.
- Amplia cantidad de registros, a diferencia de CISC que son limitados.

- Operaciones registro a registro.
- Mayor tiempo/esfuerzo de compilación.

Ventajas y desventajas de los procesadores CISC:

- Ventaja: El compilador es mas sencillo por disponer de un repertorio amplio de instrucciones y modos de direccionamiento
- Desventaja: las instrucciones de máquina complejas son difíciles de aprovechar por el compilador, es decir las usa poco nada.
- Desventaja: la optimización es más difícil de realizar.
- Ventaja: los programas son más pequeños, tiene menos instrucciones, lo que probablemente implique que ocupa menos memoria. Sin embargo, la memoria hoy día es muy barata, por lo que esta ventaja es muy relativa.
- El número de bits de memoria que ocupa no tiene porqué ser más pequeño al tener menos instrucciones
- Desventaja: al tener repertorio de instrucciones más amplio, los campos de código de operación son más largos y aumentan el tamaño de la instrucción.
- Las referencias a registros necesitan menos bits

El objetivo principal en el desarrollo de los procesadores es mejorar la velocidad de ejecución de los programas. Se supone que aumentar la complejidad del procesador debería mejorar su velocidad en ejecutar los programas. Pero los procesadores CISC casi no usan las instrucciones más complejas. El repertorio de instrucciones complejo exige una unidad de control más compleja y lenta, esto aumenta el tiempo de ejecución de las instrucciones simples, es decir, penaliza las instrucciones que podrían hacerse más rápido. En definitiva, no está comprobado que la tendencia hacia CISC fuera la apropiada.

Se han realizado numerosas mediciones comparativas entre procesadores RISC y CISC. Pero las comparaciones tienen varios problemas:

- No existe un par de máquinas RISC y CISC directamente comparables.
- No hay un conjunto de programas de prueba definitivo.
- En los análisis, es muy difícil separar los efectos del hardware de los del compilador.
- En muchos casos, las comparaciones se realizan usando prototipos o simulaciones en un “ambiente” controlado, y pocas veces con productos comerciales.
- La mayoría de las máquinas son una mezcla de ambas.

Por otra parte es necesario definir qué parámetros se van a medir. En general, las evaluaciones pueden ser:

- Cuantitativas: básicamente comparando los tamaño de los programas y su velocidad de ejecución
- Cualitativas: revisión de soporte de lenguajes de alto nivel y uso óptimo de los recursos VLSI^[4]

Conclusiones:

- No existe una marcada diferencia de performance entre uno y otro.
- No está clara la barrera que separa uno u otro estilo.
- Muchos diseños incluyen características de ambos criterios, por ejemplo PowerPC y Pentium II.

Optimizaciones del banco de registros

Los registros son el dispositivo de almacenamiento más rápido disponible. El banco de registro es físicamente pequeño y está en el mismo chip que la ALU y la UC, además emplea direcciones mucho más cortas que las de la caché y la memoria.

Se necesita una estrategia que permita que los operandos a los que se acceda con mayor frecuencia se encuentren en registros, **minimizando los accesos a memoria**.

La **optimización del banco de registros** se puede hacer de 2 maneras:

- Aproximación por Hardware: consiste en usar más registros, de manera que puedan mantenerse en ellos más variables durante períodos de tiempo más largos. Si bien logra reducir los accesos a memoria y la disponibilidad de variables escalares locales almacenadas en registros, aumenta la cantidad de bits requeridos para direccionar el banco de registros.
- Aproximación por Software: consiste en confiar al compilador la maximización del uso de los registros. El compilador intentará asignar registros a las variables que se usen más en un periodo de tiempo dado. Esta solución requiere el uso de sofisticados algoritmos de análisis de programas.

Ya que la mayoría de las referencias a operandos se hacen a datos escalares locales, la solución evidente es almacenarlos en registros. El problema es que la definición de local varía con cada llamada y retorno de procedimiento, operaciones que suceden con frecuencia. En cada llamada, las variables locales deben guardarse desde los registros en la memoria, para que los registros puedan ser reutilizados por el programa llamado. Además, deben pasarse parámetros. En el retorno, las variables del programa padre tienen que ser cargadas de nuevo en los registros y hay que devolver los resultados al programa padre.

Variables locales - ventana de registros

La **ventana de registros** es solución por software, por parte del compilador, para optimizar el uso de registros, basada en otros dos resultados de los estudios:

- Un procedimiento típico emplea:
 - pocas variables locales
 - recibe y pasa pocos argumentos
 - hace uso de pocas variables globales
- La profundidad de activación de procedimientos (anidamientos) fluctúa dentro de un rango relativamente pequeño.

Para explotar estas propiedades, se usan múltiples conjuntos pequeños de registros, cada conjunto, que llamamos ventana, asignado a un procedimiento distinto. Una llamada a un procedimiento hace que el procesador conmute a una ventana de registros distinta de tamaño fijo, en lugar de salvaguardar los registros en memoria (pila). Las ventanas de procedimientos adyacentes están parcialmente solapadas para permitir el paso de parámetros.

En un momento dado, solo es visible una ventana de registros, y se direcciona como si fuera el único conjunto de registros. La ventana se divide en tres áreas de tamaño fijo:

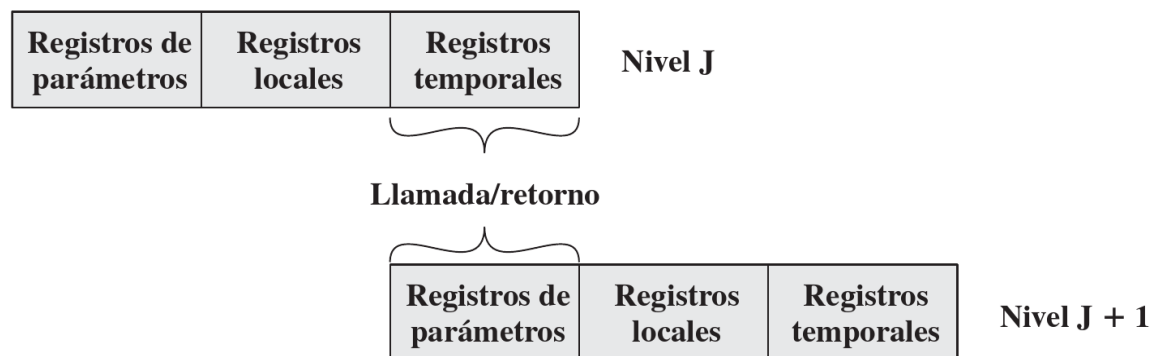
- Los **registros de parámetros** contienen parámetros pasados al procedimiento actual desde el procedimiento que lo llamó, y los resultados a devolver a este.
- Los **registros locales** se usan para variables locales, según la asignación que realice el compilador.

- Los **registros temporales** se usan para intercambiar parámetros y resultados con el siguiente nivel más bajo (el procedimiento llamado por el procedimiento en curso). Los registros temporales de un nivel son físicamente los mismos que los registros de parámetros del nivel más bajo adyacente. Este solapamiento posibilita que los parámetros se pasen sin que exista una transferencia de datos real.

Cada subrutina tendrá su banco de registros propios para variables locales. Los registros donde recibe los parámetros una subrutina serán los registros donde pasa los argumentos la subrutina que la llama.

Ejemplo:

Una subrutina de nivel $j + 1$ recibe los parámetros de la subrutina de nivel j en los registros R8 a R15. La subrutina de nivel j le pasa los parámetros a la subrutina de nivel $j + 1$ en los registros R24 a R31. Entonces si se solapan los registros R24 a R31 de la subrutina j con los R8 a R15 de la $j + 1$, se pueden pasar los parámetros directamente entre la subrutina j y la $j + 1$ a través del mismo grupo físico de registros.



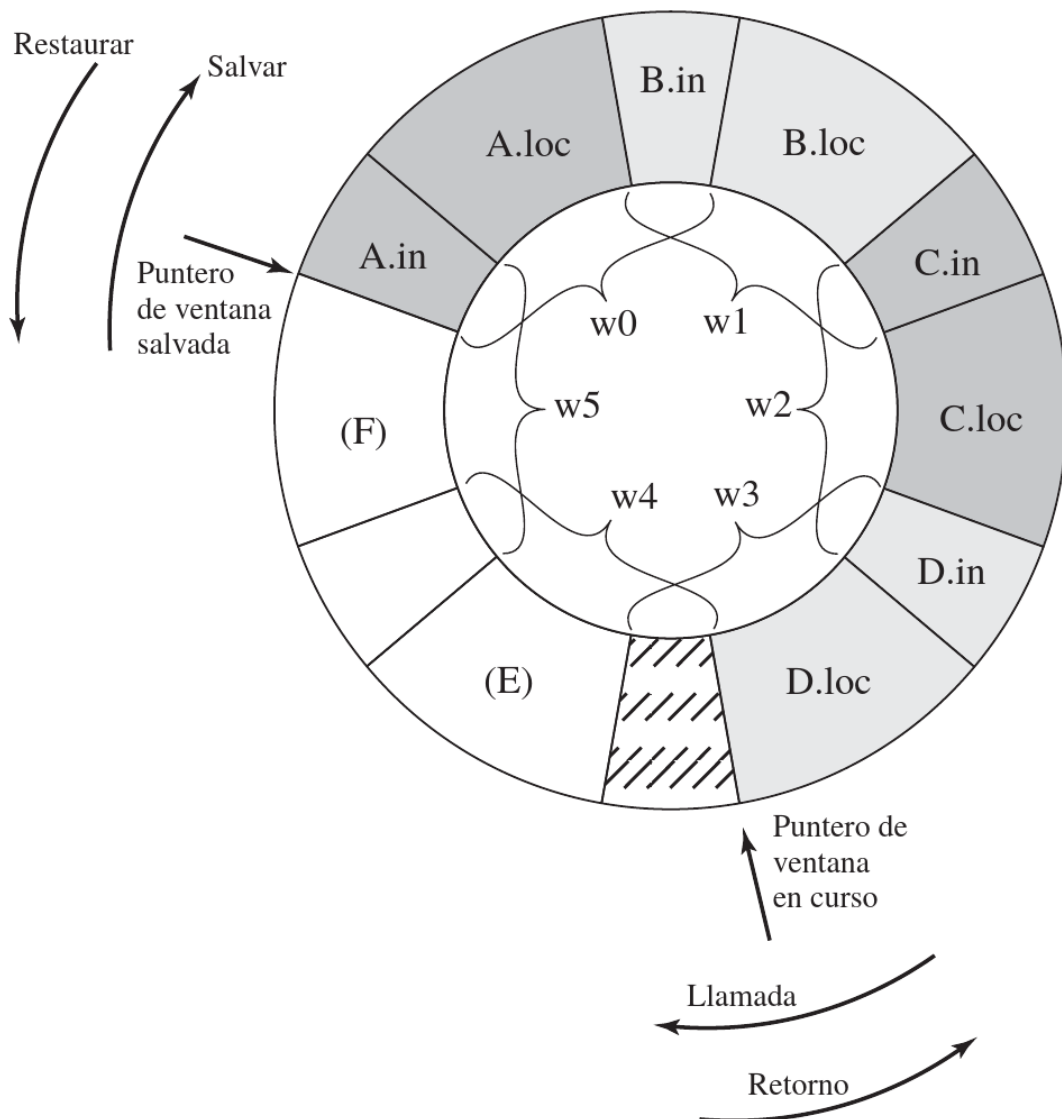
La cantidad de registros accesibles por cada subrutina es la misma, pero cambian los registros físicos a los que accede.

Las ventanas de registros se pueden usar para contener las más recientes activaciones de procedimientos. Las activaciones más antiguas han de salvaguardarse en memoria y restaurarse más tarde cuando la profundidad de anidamiento disminuya.

Un banco de registros de n ventanas puede contener solo $n - 1$ activaciones de procedimientos.

Ejemplo: Ventana de registros

La organización real del banco de registros es un buffer circular de ventanas solapadas:



- Se muestra representa un buffer circular de seis ventanas.
- El buffer está lleno hasta una profundidad de 4 (A llamó a B; B llamó a C; y C llamó a D) siendo D el procedimiento activo.
- El puntero de ventana en curso (Current-Window Pointer, CWP) apunta a la ventana del procedimiento activo actualmente. Las referencias de una instrucción máquina a registros se transforman con este puntero para determinar el registro físico real.
- El puntero de ventana salvada (Saved-Window Pointer, SWP) identifica la ventana guardada en memoria más recientemente.

Si el procedimiento D llama ahora al procedimiento E, los argumentos para E se colocan en los registros temporales de D (el solapamiento entre w3 y w4) y el CWP avanza en una ventana. Si el procedimiento E hace después una llamada al procedimiento F, esta no puede llevarse a cabo con el estado actual del buffer. Ello se debe a que la ventana de F se solapa con la de A. Si F comienza a cargar sus registros temporales, como preparación para una llamada, sobrescribirá los registros de parámetros de A (A.in). Por tanto, cuando al incrementar CWP (módulo 6), este llega a ser igual a SWP, tiene lugar una interrupción, y la ventana de A se guarda. Solo hay que guardar las dos primeras partes (A.in y A.loc). Después se incrementa SWP y la llamada a F sigue adelante. En los retornos ocurre una interrupción similar. Por ejemplo, con posterioridad a la activación de F, cuando B retorna a A, CWP se decrementa y llega a ser igual a SWP. Esto provoca una interrupción cuyo resultado es la restauración de la ventana de A.

Por qué usar una ventana de registros?

Supongamos una organización de registros de 8 variables por tipo de dato:

- R0-R7: registros para datos globales (comunes a todas las subrutinas)
- R8-R15: registros para parámetros
- R16-R23: registros para variables locales
- R24-R31: registros para argumentos a pasar

Se requerirían 5 bits para identificar todos los posibles registros. Cada subrutina dispondrá de 24 registros para hacer uso.

Esto genera ciertos problemas:

- Cada vez que una subrutina invoca otra subrutina (anidamiento de subrutinas) debe pasarle los argumentos, lo que produce un gasto de tiempo considerable.
- Cuanto más registros tenga la subrutina, más tiempo se consume en pasar la información.
- Cuantos mas registros necesite localmente, mas tiempo se requiere en reservarlos.

El uso de una ventana de registros, da solución a estos inconvenientes en el uso de procedimientos y el pasaje de parámetros.

Variables globales

El esquema de ventanas proporciona una organización eficiente para el almacenamiento de variables escalares locales en registros, pero no considera las variables globales, que son las accedidas por más de un procedimiento.

Para el manejo de las variables globales se sugieren 2 opciones:

1. Que el compilador asigne posiciones de memoria a todas las variables declaradas como globales en un HLL: es algo ineficiente para variables globales a las que se accede frecuentemente porque requiere accesos a memoria, que son lentos.
2. Incorporar en el procesador un conjunto de registros globales, accesibles en todos los niveles (no está mostrado en la ventana de registros).

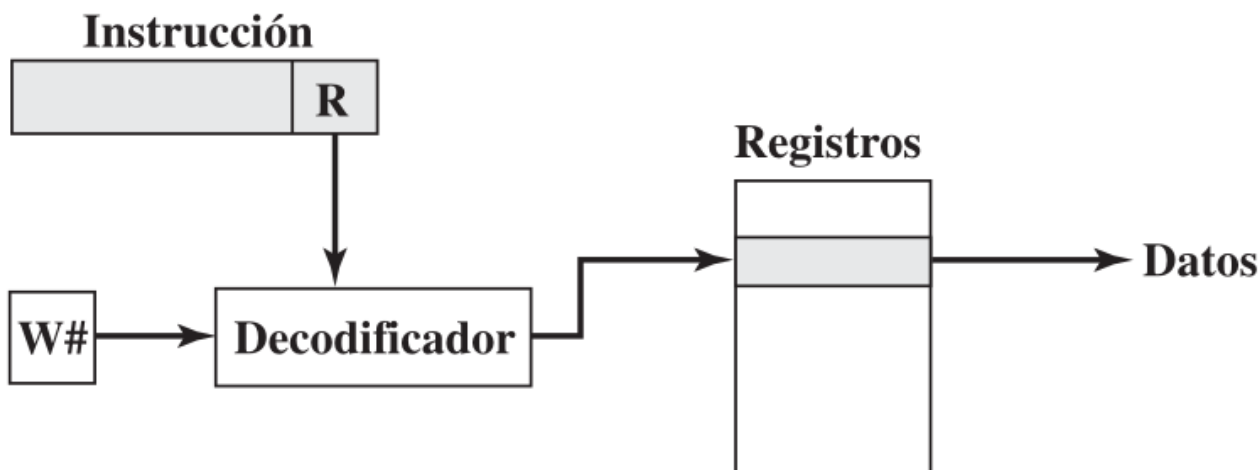
¿Por qué no una cache?

El banco de registros organizado en ventanas funciona como un buffer pequeño y rápido que contiene un subconjunto de todas las variables que probablemente se usen más. Desde este punto de vista, el banco de registros se comporta similar a una caché, aunque se trate de una memoria mucho más rápida. Surge por tanto la cuestión de si sería más sencillo y mejor usar una caché y un pequeño banco de registros tradicional.

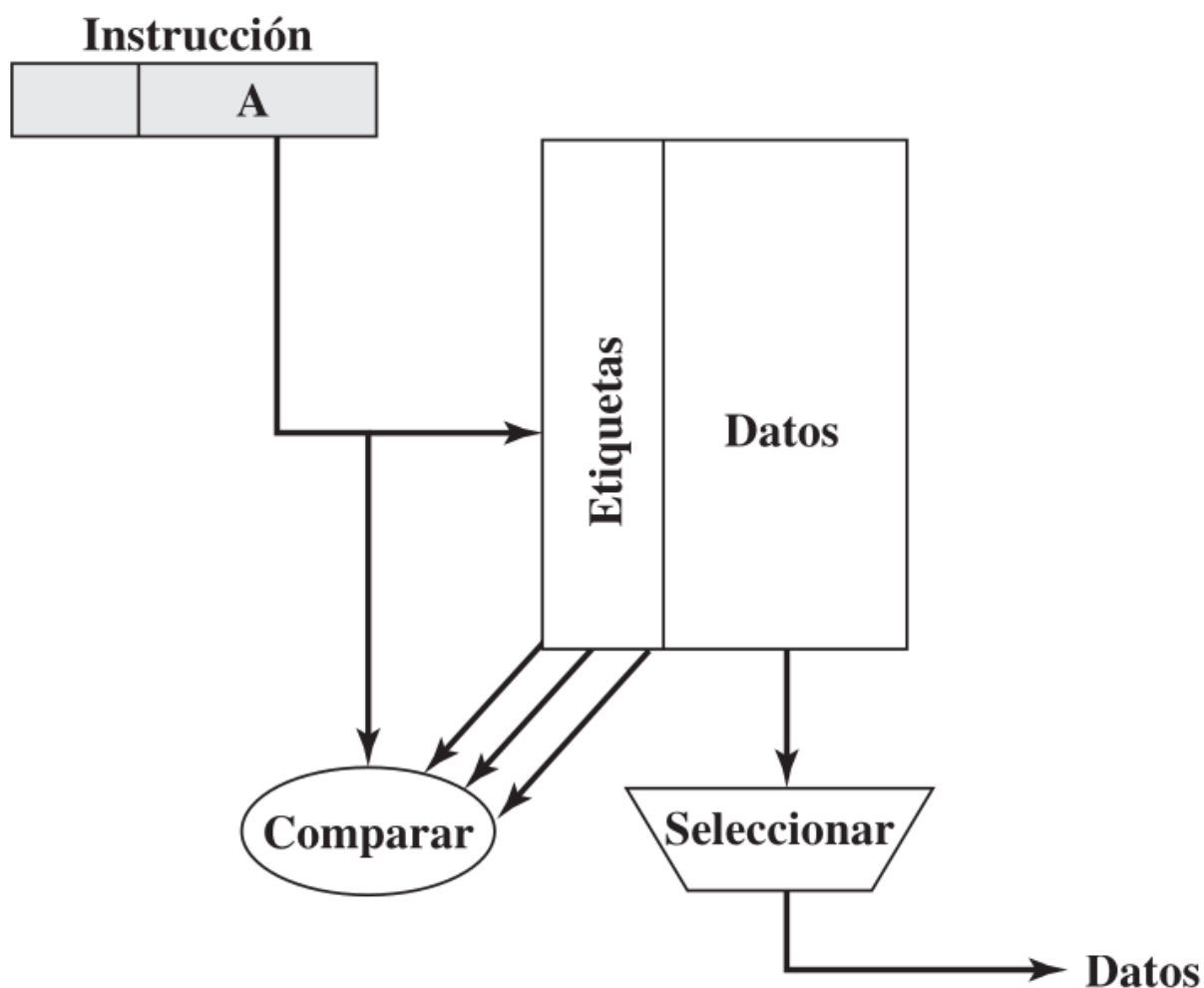
Ventana de registros	Memoria cache
Todos los datos son escalares y locales	Datos escalares locales recientemente usados
Acceso individual a variables	Acceso a bloques de memoria

Ventana de registros	Memoria cache
Variables globales asignadas por el compilador	Variables locales y globales usadas recientemente
Salvaguarda/restauración basadas en la profundidad de anidamiento	Salvaguarda/restauración basadas en algún algoritmo de reemplazo
Direccionamiento de registro	Direccionamiento de memoria

La elección entre un amplio banco de registros basado en ventanas y una caché no está clara. No obstante, existe una característica en la cual la aproximación de los registros es superior y nos indica que un sistema basado en caché será notablemente más lento: el coste de direccionamiento. Para referenciar un dato escalar local en un banco de registros basado en ventanas, se usan un número de registro virtual y un número de ventana. Los dos pueden pasar a través de un decodificador relativamente sencillo para seleccionar uno de los registros físicos. Para referenciar una posición de memoria en la caché, hay que generar una dirección de memoria completa. La complejidad de esta operación depende del modo de direccionamiento. En una caché asociativa por conjuntos, una parte de la dirección se usa para leer un número de palabras y etiquetas igual al tamaño del conjunto. Otra parte de la dirección se compara con las etiquetas, y se selecciona una de las palabras leídas. Incluso en el caso de que la caché sea tan rápida como el banco de registros, el tiempo de acceso será considerablemente mayor. Por consiguiente, desde el punto de vista de las prestaciones, el banco de registros basado en ventanas es superior para datos escalares locales. Se puede conseguir una mejora adicional en las prestaciones añadiendo una caché solo para instrucciones.



(a) Banco de registros basado en ventanas



(b) Caché

Optimización por software

Los lenguajes de alto nivel no tienen referencias explícitas a los registros, las cantidades usadas en el programa son referidas simbólicamente. El compilador se encarga de la asignación de registros, y el uso optimizado de estos es su responsabilidad (y no del programador).

El objetivo del compilador de un HLL es mantener en registros en lugar de en memoria los operandos necesarios para tantos cálculos como sea posible, y minimizar las operaciones de carga y

almacenamiento. Tratando de asignar los pocos registros a las variables más usadas o las que más tiempo permanecerán en el registro.

Una estrategia consiste en suponer que se disponen de infinitos registros: cada variable candidata para residir en un registro se asigna a un registro simbólico o virtual. El compilador asigna el número ilimitado de registros simbólicos a un número fijo de registros reales. Los registros simbólicos cuya utilización no se solape pueden compartir el mismo registro real. Si en una parte concreta del programa hay más cantidades a tratar que registros reales, algunas de las cantidades se asignan a posiciones de memoria.

Para colocar temporalmente las cantidades en los registros durante las operaciones de cálculo se usan instrucciones de carga y almacenamiento. Lo esencial de la labor de optimización es decidir qué cantidades tienen que ser asignadas a registros en un determinado punto del programa.

La técnica usada más comúnmente en los compiladores para RISC se conoce como **coloreado de grafos**, una técnica que procede de la disciplina de topología.

Segmentación del cauce (pipelining)

Cálculo de eficiencia

Utilizamos como medida de eficiencia de un programa **CPI**, que significa "Cycles Per Instruction" o "Ciclos por Instrucción". Refleja el promedio de ciclos de reloj requeridos para ejecutar una instrucción, en un programa o fragmento de un programa.

$$\text{CPI} = \frac{\text{cantidad de ciclos}}{\text{cantidad de instrucciones}}$$

Diseño de procesadores secuenciales

Actualmente el diseño de procesadores sigue una metodología prácticamente universal gracias al estándar de facto en el que se han convertido las arquitecturas RISC.

En el diseño de una CPU se pueden distinguir dos grandes módulos dentro del procesador: la ruta de datos y la unidad de control.

La **ruta de datos** es la parte combinacional del procesador, encargada de operar con datos y producir resultados. Por otro lado, la **unidad de control** es la responsable del estado del procesador y de gestionar las operaciones que se realizan en la ruta de datos.

Un **procesador secuencial** es aquel que hasta que no termina de ejecutar una instrucción no comienza a ejecutar la siguiente.

Existen dos tipos de procesadores secuenciales dependiendo del método de temporización escogido durante su diseño:

- **Procesador monociclo:** Cada instrucción debe completarse en un único ciclo de reloj.
- **Procesador multiciclo:** Cada instrucción puede tardar más de un ciclo en ejecutarse, tantos como sea necesario.

El problema en la ejecución monociclo se da en que la duración del ciclo de reloj del procesador vendrá fijada por la instrucción que más tarde en ejecutarse, ya que le tiene que dar tiempo a finalizar en un único ciclo de reloj. El CPI será siempre igual a 1. Por el contrario, un procesador multiciclo tendrá siempre CPI mayor 1, pero el periodo de reloj será menor que en el caso del procesador monociclo.

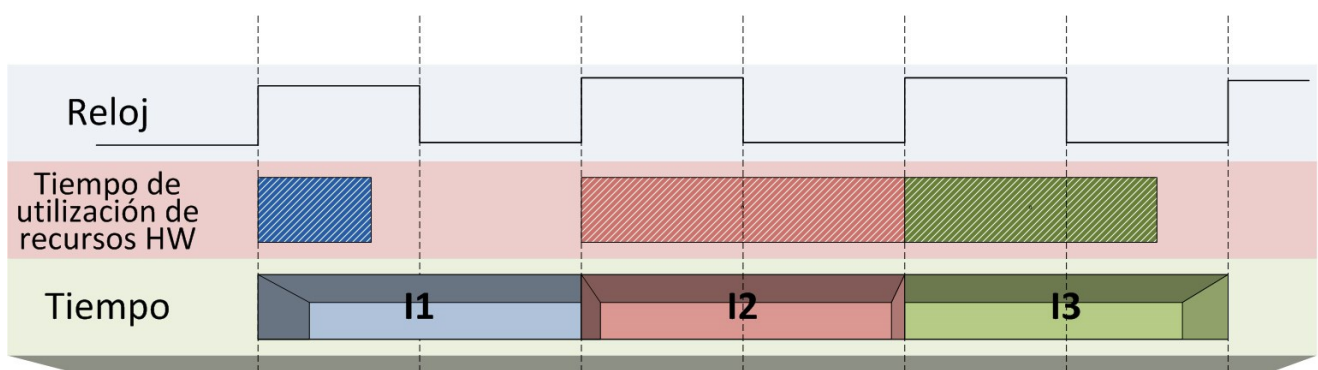
Ejecución monociclo

El dato fluye por las unidades funcionales hasta que se completa la ejecución de la instrucción en curso. En cada ciclo de reloj se inicia una nueva instrucción, cuando el PC se carga con un nuevo valor.

El período del reloj se hace con el tiempo necesario para completar la instrucción más lenta. En las instrucciones más rápidas el procesador "espera" el fin del ciclo para continuar.

La eficiencia dependerá entonces del tiempo de resolución de la instrucción más lenta, en otras palabras, el ciclo de reloj se hará en función de la instrucción más larga.

Dado que las instrucciones que son "mas cortas" tienen que esperar un tiempo hasta que se complete el período de reloj, si la tasa de instrucciones cortas es mucho mayor que la de las largas, se puede perder mucho tiempo.



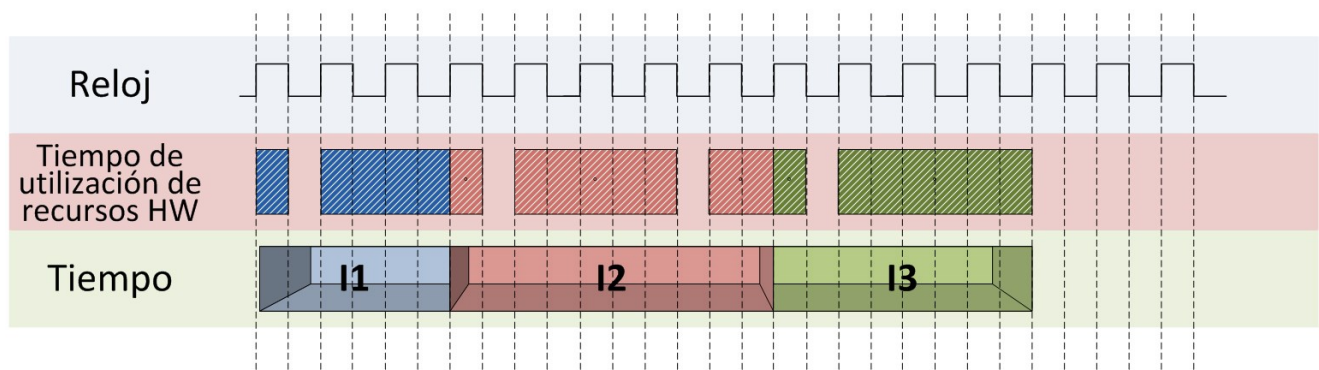
Los diseños monociclo no se emplean en la actualidad ya que no son lo suficientemente eficientes.

Un ciclo por instrucción $CPI = 1$.

Ejecución multiciclo

Una solución al desperdicio de tiempo introducido por la ejecución monociclo, es fijar un ciclo de reloj más pequeño, y disponer que cada instrucción ocupe los ciclos de reloj que requiera.

El ciclo de instrucción es entonces variable, y se ajusta al tipo de instrucción. Las instrucciones más rápidas usarán menos ciclos de reloj que las más lentas.



La ruta de datos es muy similar a la del procesador monociclo, pero es necesario incluir **registros intermedios** que almacenen los resultados temporales que se van generando según la instrucción avanza en su ejecución.

Siempre habrá múltiples ciclos por instrucción, variando dependiendo la instrucción, el CPI nunca será 1.

¿Qué es un cauce segmentado?

La **segmentación de cause** o **pipelining** es una técnica de diseño de hardware para implementar **paralelismo a nivel instrucciones** (ILP, *Instruction Level Parallelism*) con un solo procesador, acelerando la ejecución de instrucciones.

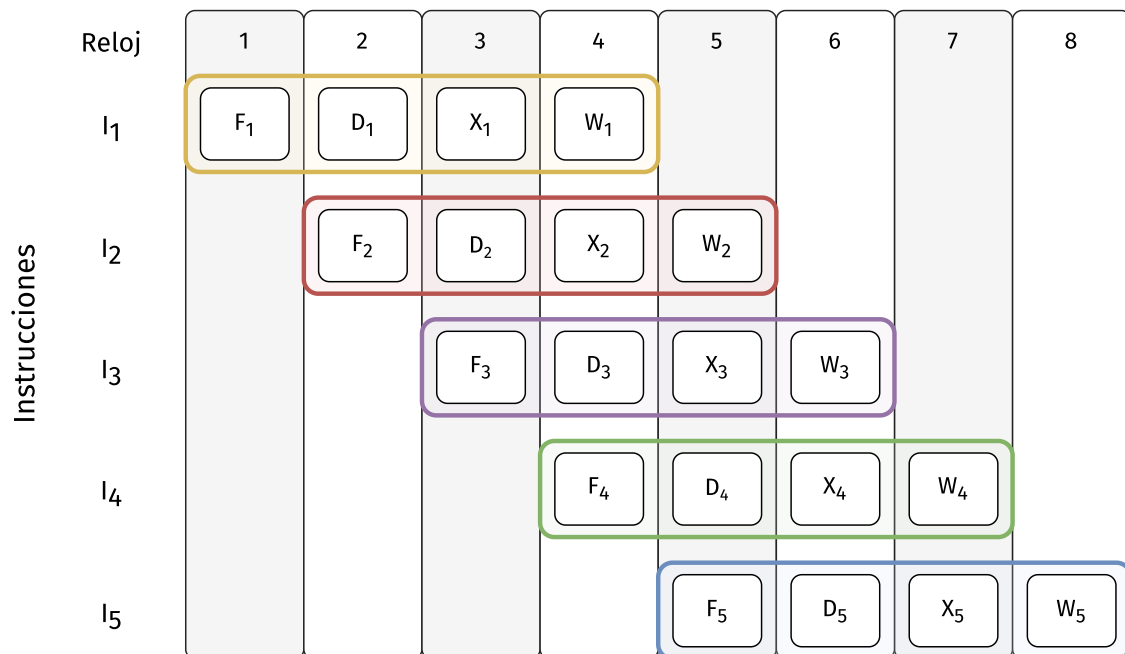
Permite combinar las mejores características de los diseños de ejecución monociclo y multiciclo. Se incorpora en prácticamente todos los procesadores actuales y existe tanto en arquitecturas CISC como RISC.

Las instrucciones son divididas en etapas a lo largo de su ciclo. Las etapas que forman el ciclo de instrucción, son ejecutadas por diferentes unidades (recursos de hardware) capaces de operar de manera simultánea.

Cada instrucción no requiere esperar a la finalización de la previa para comenzar a resolverse, esto sucede a medida que se liberan las unidades. En contrapartida, ejecutar una única instrucción cada vez, de manera secuencial, desaprovecha tiempo.

Ejemplo: ejecución segmentada

Ejemplificando en una máquina de 4 fases:



Cada instrucción tarda 4 ciclos de reloj:

- En la ejecución segmentada, se tardan 5 ciclos de reloj en completar dos instrucciones.
- En la ejecución secuencial, se tardarían 8 ciclos de reloj en completar las mismas dos instrucciones.

No se mejora la velocidad de ejecución de la instrucción, la segmentación incrementa la **productividad (throughput)**, es decir, la cantidad de instrucciones resueltas en un período de tiempo determinado.

Idealmente se termina una nueva instrucción por ciclo.

El diseño de procesadores segmentados tiene gran dependencia del repertorio de instrucciones.

El CPI ideal de un procesador segmentado, o el **máximo rendimiento teórico** es $CPI = 1$, de manera que se finaliza una nueva instrucción cada ciclo y, en promedio, es como si cada instrucción sólo necesitara 1 ciclo para ejecutarse. Es decir, se consigue el CPI de un procesador secuencial monociclo.

El periodo de reloj viene limitado por la etapa más lenta, a la que tiene que dar tiempo a terminar en un ciclo. Es decir, se consigue el periodo de un procesador secuencial multiciclo. (técnicamente ligeramente peor, ya que el nuevo hardware introducido para segmentar el cauce aumenta el tiempo de instrucción)

El incremento potencial de la segmentación de cauce es proporcional al número de etapas del cauce. Se puede calcular como:

$$\text{vel. procesador segmentado} = K \cdot \text{vel. secuencial}$$

Donde K es el número de etapas del cauce.

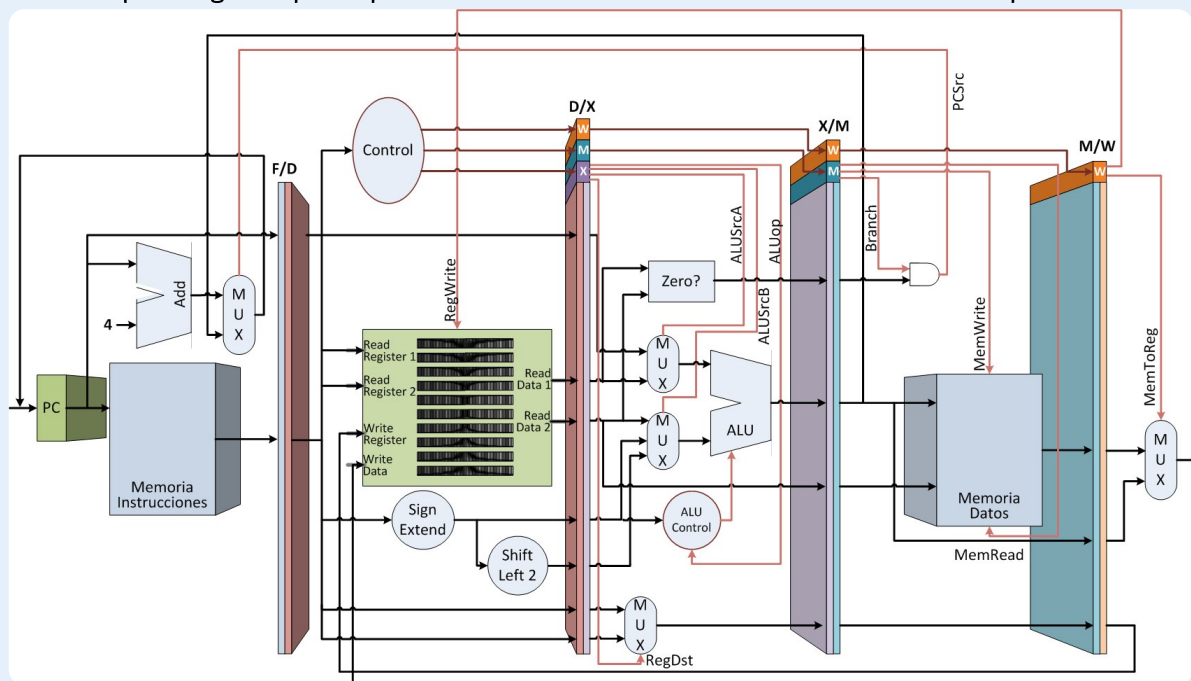
Procesador multiciclo a procesador segmentado

Partiendo de un procesador multiciclo, para segmentarlo basta con comenzar la ejecución de una nueva instrucción cada ciclo. Asegurándose de que en cada etapa las diferentes

instrucciones están utilizando diferentes recursos para que no haya ningún conflicto.

En cuanto a hardware:

- Requiere una memoria datos e instrucciones separadas para evitar conflictos al captar instrucción y querer escribir en ella. (etapa F, etapa W)
- Dado que se accede al banco de registros para decodificar la instrucción (D) y para escribir el resultado (W), se requiere subdividir el ciclo y así quAe ocurran en diferentes periodos. *Esto es posible por la velocidad de los registros*
- Se utiliza un multiplexor en la etapa F que se encarga de controlar todas las modificaciones del PC y se añade un restador (bloque Zero?) en la etapa de X para poder realizar al mismo tiempo la suma del desplazamiento al PC en la ALU y la comparación de los registros en el caso de las instrucciones BEQ (evaluación de la condición del salto).
- Se reemplazan los registros intermedios con registros de segmentación, en los que se almacenan los resultados de cada etapa al final del ciclo de reloj, y se utilizan como entrada de la etapa siguiente, al comenzar el siguiente ciclo de reloj. Esto permite evitar conflictos en la utilización de recursos. Cada uno tendrá una longitud diferente, dependiendo de la información que tenga que almacenar para que las instrucciones puedan avanzar por la ruta de datos (la propia instrucción, operandos y resultados, señales de control, etc.).
- La unidad de control es un circuito combinacional que genera las señales de control necesarias para cada tipo de instrucción a partir del opcode. La diferencia con el procesador monociclo es que hay que dimensionar adecuadamente los registros de segmentación, de manera que tengan espacio para almacenar las señales de control en cada etapa.



Si bien el CPI ideal es 1, en la práctica nunca se alcanza, debido a los conflictos que introduce el pipelining.

Conflictos

No todas las etapas pueden ser manejadas en paralelo, pueden existir conflictos al necesitar el uso simultáneo de un mismo recurso.

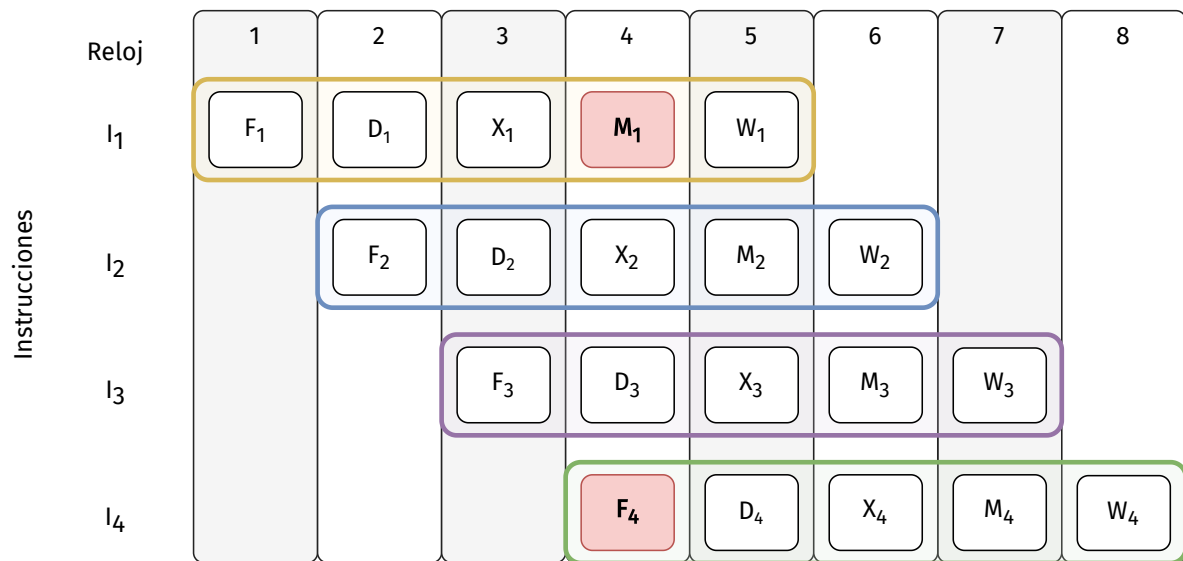
Existen 3 tipos de **riesgos** que producen **stalls** o atascos:

1. **Estructurales** (*structural hazard or resource hazard*).
2. **Por dependencia de datos** (*data hazard*).
3. **Por dependencia de control** (*control hazard or branch hazard*).

Riesgo estructural

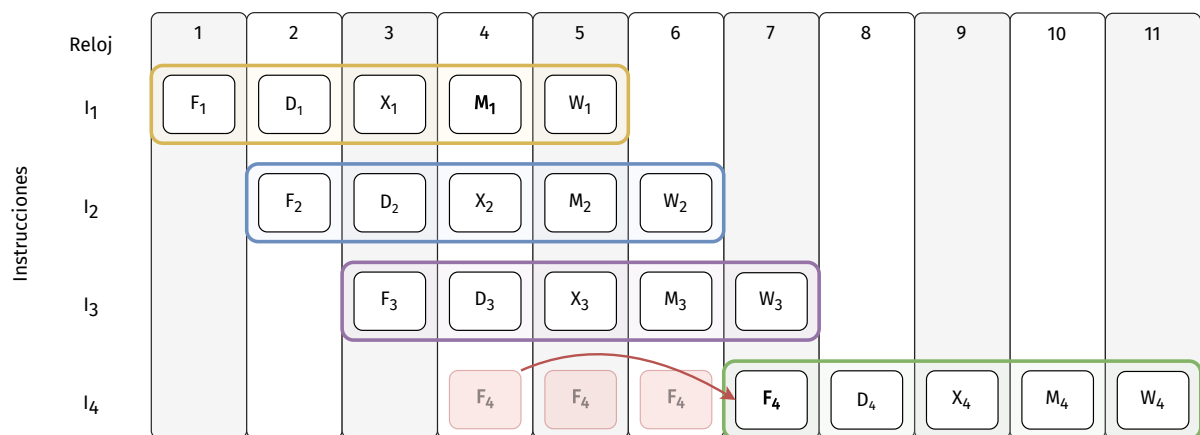
Sucede cuando dos (o más) instrucciones, que están en el pipeline, necesitan hacer uso del mismo "recurso", en un ciclo de reloj dado.

Ejemplo: conflicto estructural



Conflicto por uso de la memoria. Una forma de evitarlo es tener separada la memoria de instrucciones de la memoria de datos, como en nanoMIPS.

Solución con ciclos de parada:



Riesgo de dependencia de datos

Sucede cuando en una determinada etapa del cauce, los operandos fuente o destino de una instrucción (registros) no están disponibles, ya que dependen del resultado de una instrucción previa,

que está aún en el cauce.

Pueden llevar a la obtención de resultados erróneos en la ejecución del código, por realizar operaciones de escritura o lectura en un orden diferente al indicado por la secuencia de instrucciones.

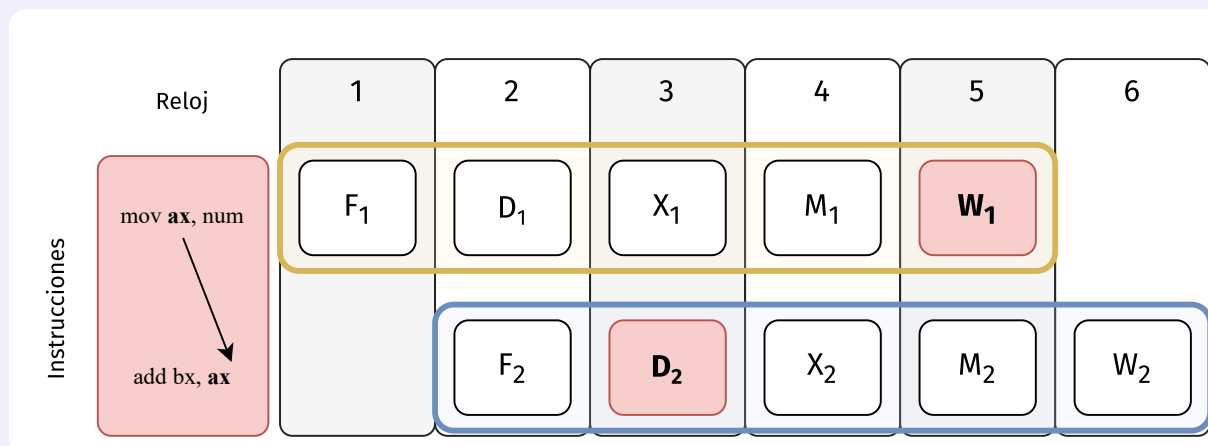
Se pueden dar 3 tipos de **dependencias de datos**. Dadas dos instrucciones A y B, B depende de A cuando:

- **RAW** (Read After Write): Dependencia verdadera. B necesita leer un registro que A aún no escribió.
- **WAW** (Write After Write): Dependencia de salida. B necesita escribir un registro que A aún no escribió. Solo ocurre si se permite que las instrucciones se adelanten unas a otras (se altera la secuencia de ejecución de instrucciones).
- **WAR** (Write After Read): Antidependencia. B necesita escribir un registro que la instrucción A aún no leyó.

Las RAW se dicen dependencias verdaderas porque las WAW y WAR, aunque pueden provocar riesgos, se deben a la reutilización de los registros visibles para el programador, pero no existe un flujo real de información entre las instrucciones que provocan la dependencia.

Las dependencias WAR se denominan antidependencia, porque son la inversa de la dependencia verdadera (RAW), en lugar de que la primera instrucción produzca un valor que usa la segunda, la segunda instrucción destruye un valor que usa la primera.

Ejemplo dependencia de datos, RAW:

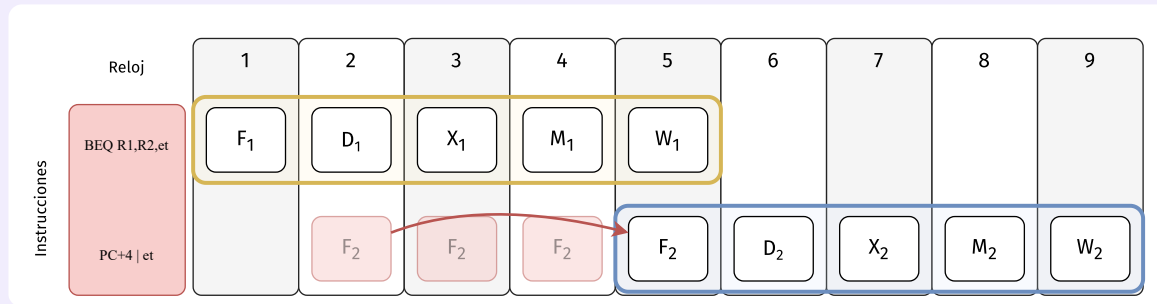


Riesgo de dependencia de control

Se producen cuando una instrucción que modifica el valor del PC todavía no lo ha hecho cuando se tiene que comenzar la ejecución de la siguiente instrucción.

Implican la presencia de instrucciones de salto.

Ejemplo: dependencia de control, salto condicional en nanoMIPS



Con la instrucción **BEQ** recién se calcula la dirección de salto durante la fase W.

Una instrucción tiene que calcular el nuevo valor que modifica el valor del PC. La próxima instrucción no puede comenzar hasta que no se resuelva el salto.

Soluciones

La solución elemental, pero menos deseada, de lidiar con atascos, es **retrasar la ejecución de la tarea conflictiva** los ciclos de reloj necesarios, hasta que desaparezca el conflicto. Al perder ciclos de reloj se pierde tiempo y disminuye la performance del procesador.

Existen varias técnicas empleables al diseñar una arquitectura, que permiten reducir y, en algunos casos, eliminar los efectos de los atascos.

Riesgos estructurales

La solución a estos conflictos consiste en "agregar más hardware", algunas de las estrategias usadas son:

1. **Duplicación de recursos de hardware.**
 - Por ej. agregar sumadores o restadores además de la ALU.
2. **Separación del recurso en conflicto.**
 - Por ej. separar memorias de instrucción y datos
3. **Subdividir el acceso al recurso.**
 - Por ej. dividir el ciclo de reloj a la hora de leer/escribir en registros. En un primer sub-ciclo se escribe y en el segundo semiciclo se lee.

Riesgos de datos

Existen soluciones por hardware y software:

1. Por hardware: **Adelantamiento de operandos** (*forwarding*) entre etapas del cauce.
2. Por software: El compilador **reordena las instrucciones**, separando los más posible las instrucciones con dependencia de datos, sin alterar la lógica del programa. Si no es posible, inserta **NOP** (No Operation), osea, los ciclos de parada necesarios, reduciendo la performance del procesador.

Forwarding

El **forwarding** consiste en "adelantar" o pasar directamente (desde una unidad funcional a otra), el resultado obtenido en una instrucción, a las instrucciones siguientes que lo necesiten como operando.

Es decir, estas instrucciones no tienen que esperar a que la que produce el resultado lo escriba en el registro destino, sino que reciben su valor en cuanto está disponible.

Es transparente al programador. En algunos casos el adelantamiento no es suficiente para evitar las paradas en la ruta de datos, en otros sí logra evitar las paradas.

Puede mejorar mucho el rendimiento de un procesador segmentado y es fácil de implementar. Requiere modificar la ruta de datos, identificando todos los posibles adelantamientos para el repertorio y comunicando los registros de segmentación involucrados.

La lógica de adelantamiento suele estar compuesta por nuevos multiplexores para controlar las entradas de las etapas que pueden necesitar un adelantamiento, y unidades hardware sencillas (**unidad de adelantamiento**) que permitan comunicar las salidas de las etapas que producen los resultados con las entradas de estos multiplexores.

También es necesario un nuevo hardware de control que al decodificar la instrucción en la etapa de decodificación, detecte los adelantamientos y las paradas que van a ser necesarias para la ejecución de la instrucción.

Riesgos de control

1. Técnicas por hardware:

- **Adelantamientos.** El forwarding permite solventar estos conflictos sin retardos:
 - En **saltos incondicionales** la dirección de destino se debe determinar lo más pronto posible dentro del cause para reducir la penalización.
 - En **saltos condicionales** se introduce el riesgo adicional por la dependencia entre la condición de salto y el resultado dependiente de una instrucción previa. Con forwarding, se obtendrá el resultado del salto en la etapa (ID).
- **Predicción de saltos.**

2. Técnicas por software: intentan resolver los conflictos por software a nivel del compilador.

- **Salto retardado, relleno de ranura o Delay Slot**

Predicción de saltos

La **predicción de saltos** se basa en utilizar estrategias para predecir el resultado de la instrucción de salto.

Esta circunstancia es conocida como **ejecución especulativa**, ya que se ejecutan instrucciones sin saber si son las correctas en el orden del programa.

Las predicciones siempre consiguen una reducción de la penalización por salto cuando aciertan.

Pueden ser de 2 tipos:

- **Dinámicas:** intentan predecir la próxima instrucción teniendo en cuenta la historia previa del programa en ejecución. Se realizan en tiempo de ejecución. Son más precisas, a cambio de un coste en hardware mayor.
- **Estáticas:** intentan predecir la próxima instrucción sin tener en cuenta información previa de la ejecución del programa.

Estática

En la **predicción de salto tomado/no tomado**, la CPU predice que todos los saltos se van a tomar o que todos los saltos no se van a tomar. Osea, la predicción es siempre la misma para todos los saltos (salto tomado o salto no tomado).

De esta manera solo existe una penalización por salto cuando la predicción falla.

Como ventaja es sencillo de implementar ya que solo requiere pequeños cambios en la unidad de control y no en la ruta de datos.

La desventaja es que, en caso de fallar, la instrucción a descartar puede afectar el estado del procesador (registros y/o memoria). Por lo que, si no se incorporan en la ruta de datos mecanismos para solucionarlo, no se debe dejar que las instrucciones posterior al salto modifiquen el estado del procesador y/o la memoria, es decir, se debe parar su ejecución antes de que lleguen a esas etapas.

Cuando se produce un fallo de predicción y deberá ser truncado el pipeline seguido al salto y proceder a ejecutar la instrucción correcta. Este tipo de atasco que genera una penalización de un ciclo se denomina **Branch Taken Stalls (BTS)**.

Existen diversas técnicas de predicción estática de saltos. Existen otras que no son exclusivamente de hardware, sino que implican al compilador.

Dinámica

Existen varias técnicas de predicción de salto dinámica:

1. **Conmutador saltar/no saltar:** Estando en una de las 2 condiciones (predecir que salta o predecir que no salta), se requieren 2 predicciones fallidas consecutivas para conmutar al otro estado.

Tiene un buen comportamiento en lazos donde el resultado de una consulta (por ejemplo un contador que tiene que llegar a 0) se puede repetir muchas veces antes de que cambie el resultado de la consulta.

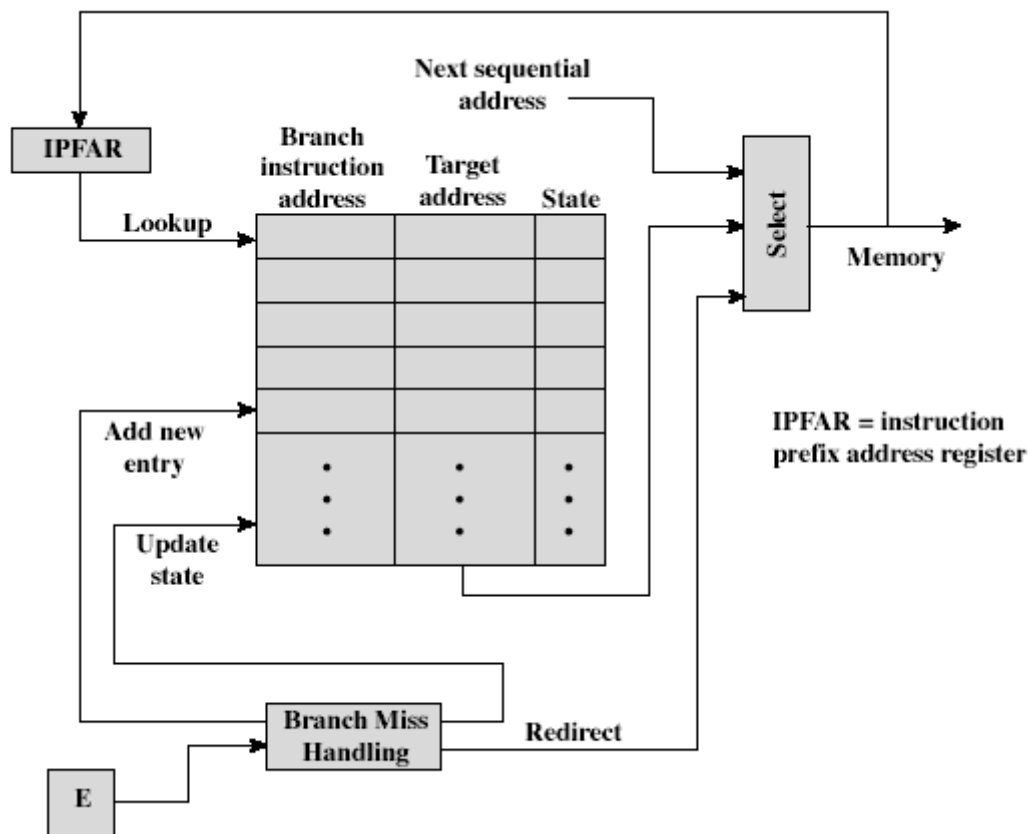
2. **Tabla de historia de saltos (BHT, Branch History Table) (BTB):** Se implementa usando una pequeña cache asociada a la etapa de búsqueda (IF).

La cache tiene 3 campos:

- Dirección de una instrucción que es del tipo bifurcación
- Dirección del destino o Instrucción destino
- n bits de estado (historia de uso)

Cada vez que se precapta una instrucción, se busca en la cache BHT si la instrucción es de ramificación. Se toma una decisión predictiva en función de los bits de la historia de uso:

- Si la predicción es saltar, se capta la dirección de salto.
- Si la predicción falla, se actualiza la tabla
- Si la instrucción de ramificación no estaba en la BHT, se la carga como nueva entrada.



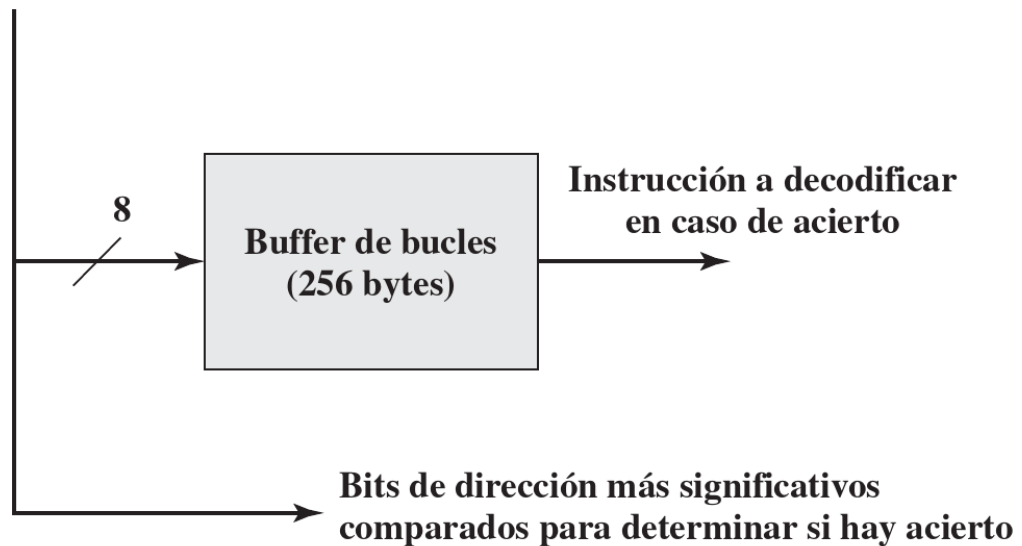
3. **Predicción según el código de operación:** Se basa en la suposición de que algunas instrucciones de salto tienen mayor probabilidad de saltar o de no saltar. La tasa de acierto puede llegar a alcanzar un 75%.

Por ejemplo, si la instrucción de salto involucra un lazo que se repite muchas veces, la condición que se usa para repetir el lazo ocurrirá muchas más veces que la que se usa para salir del lazo (que ocurrirá 1 sola vez).

4. **Varios cauces** (uno por cada opción de salto, multiple stream): se usan cauces distintos (duplicando hardware) para ejecutar simultáneamente los cauces, el que corresponde a la opción de no saltar y el que corresponde a la instrucción de salto.
 - Luego, cuando se determina el resultado de la ramificación, se debe utilizar el cauce correcto y descartar el incorrecto.
 - Como se están captando simultáneamente 2 cauces distintos, aumentan las búsquedas, lo que puede provocar retardos en el acceso al bus y a los registros.
 - Si hay nuevos saltos en los cauces en ejecución, se necesita disponer de más cauces para contemplar esta situación, lo que aumenta la complejidad del hardware.
5. **Precaptación del destino del salto:** se precapta la instrucción destino del salto, además de las instrucciones siguientes a la bifurcación. La instrucción se guarda hasta que se ejecute la instrucción de bifurcación.
6. **Buffer de bucles:** se usa una memoria muy rápida, gestionada por la etapa de captación de instrucciones, que contiene las últimas instrucciones recientemente buscadas. Si hay una instrucción de salto, el hardware comprueba si está en el buffer, si es así, la próxima instrucción se busca desde el buffer.

Muy eficaz para pequeños bucles y saltos, si el buffer es capaz de contener todo el bucle. Funciona parecido a la cache, solo que contiene instrucciones. Comparar los bits mas significativos consecutivos, únicamente.

Dirección de salto



Branch Delay Slot

Cuando hay una instrucción de salto necesita un ciclo o más de tiempo para determinar si se va a ejecutar el salto o no. Ese tiempo se llama **hueco o ranura de retardo de salto**.

Esta técnica **evita la penalización por salto**. Con **Branch Delay Slot** el compilador logra llenar los huecos de las instrucciones de salto, mediante el **reordenamiento de instrucciones**. Con instrucciones que no dependan del salto, eliminando el conflicto y haciendo trabajo útil mientras el salto se resuelve.

La instrucción a insertar no puede ser otro salto. Puede ser una instrucción:

- anterior a la de salto,
- la instrucción destino del salto
- o una instrucción del camino que se sigue cuando el salto no se toma.

Dado que las instrucciones que se usen de relleno serán ejecutadas, se tome o no el salto, no siempre es posible reorganizarlas utilizando instrucciones destino del salto o posteriores al salto, ya que podrían llegar a modificar los resultados de la ejecución. La **mejor solución sería reordenar instrucciones anteriores a la de salto**, sin dependencias con él, ya que siempre deben ejecutarse y van a ser útiles.

Si no le es posible insertar instrucciones, se agregan las instrucciones `nop` (retardo) necesarias para evitar el conflicto.

Ejemplo: relleno de ranura

Caso 1	Caso 2	Caso 3
Antes: ADD R1,R2,R3 si R2=0 entonces retardo ... MULT r6,r7,r8	Antes: SUB R4,R5,R6 MULT r6,r7,r8 ... ADD R1,R2,R3 si R1=0 entonces retardo	Antes: ADD R1,R2,R3 si R1=0 entonces retardo SUB R4,R5,R6 ... MULT R14,R15,R16 Ins+1
Despues: si R2=0 entonces ADD R1,R2,R3 ... MULT r6,r7,r8	Despues: SUB R4,R5,R6 MULT r6,r7,r8 ... ADD R1,R2,R3 si R1=0 entonces SUB R4,R5,R6	Despues: ADD R1,R2,R3 si R1=0 entonces SUB R4,R5,R6 ... Ins-1 MULT R14,R15,R16 Ins+1

1. El hueco se rellena con una instrucción anterior al salto. Sirve siempre, es la mejor opción aunque no siempre es posible.
2. Como la instrucción `ADD R1, R2, R3` se requiere ejecutar antes de la instrucción de salto, no se puede reubicar después. El compilador busca una instrucción a reubicar, en el lugar a donde puede llegar a saltar, por ejemplo `SUB R4, R5, R6`. La copia en el hueco, y corrige el lugar a donde salta. Si la instrucción de salto efectivamente salta, entonces la instrucción insertada en el hueco sirve y se adelantó trabajo. Si la instrucción no salta, la instrucción insertada se ejecuta pero no sirve y el resultado no altera la lógica.
3. Como la instrucción `ADD R1,R2,R3` se requiere ejecutar antes de la instrucción de salto, no se puede reubicar después del salto. El compilador busca una instrucción a reubicar, a continuación de la de salto, por ejemplo `SUB R4, R5, R6`. Si la instrucción no salta, entonces la instrucción insertada en el hueco sirve y se adelantó trabajo. Si la instrucción salta, la instrucción insertada se ejecuta pero no sirve y el resultado no altera la lógica.

nanoMIPS/winMIPS

Características generales

El MIPS es un modelo de procesador comercial de tipo RISC, **nanoMIPS** es una simplificación de este.

- Tamaño de palabra de 32 bits.
- Espacio de direcciones (virtual) de 64 bits.
- Diseño de arquitectura tipo Harvard (memoria de instrucciones separada de la memoria de datos).
- 32 registros de propósito general (`R0 .. R31`) de 64 bits
 - 31 son registros efectivos, `r0` siempre devuelve 0.

- 5 bits para identificar un registro $2^5 = 32$
- Doble entrada, se pueden leer dos registros al mismo tiempo. Se puede escribir uno solo.
- 32 registros de punto flotante (F0 .. F31).
- No hay registros de flags de estado. Solo hay un flag, de punto flotante (FP).
- 2^{30} palabras de memoria.
- Solo se puede operar entre registros, o registro y operando inmediato. No puede realizarse una operación entre memoria y memoria, ni memoria y registro.

Modos de direccionamiento

Dispone, formalmente, de dos **modos de direccionamiento**, pero se pueden considerar unos casos especiales más:

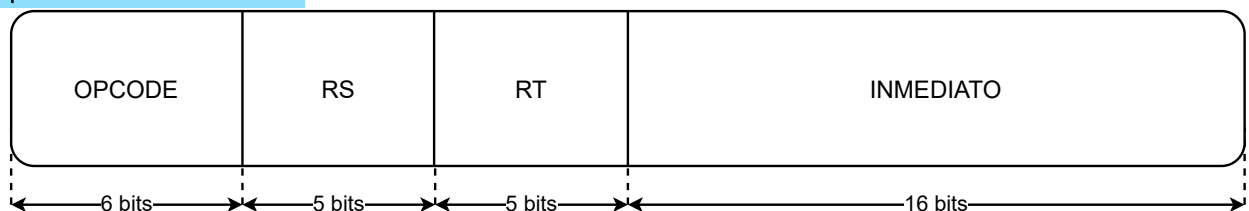
Modo	Descripción
Inmediato	el operando se codifica dentro de la instrucción
Registro	se incluye en la instrucción el identificador del registro en que se almacena el operando
Directo o absoluto	se incluye en la instrucción la dirección de memoria en que está almacenado el dato
Indirecto vía registro	se incluye en la instrucción el identificador del registro en que está almacenada la dirección de memoria en que se encuentra el operando
Inmediato con desplazamiento	como el anterior, pero se incluye también en la instrucción un operando inmediato (desplazamiento), que se suma al contenido del registro para obtener la dirección de memoria en la que se encuentra el operando

ISA

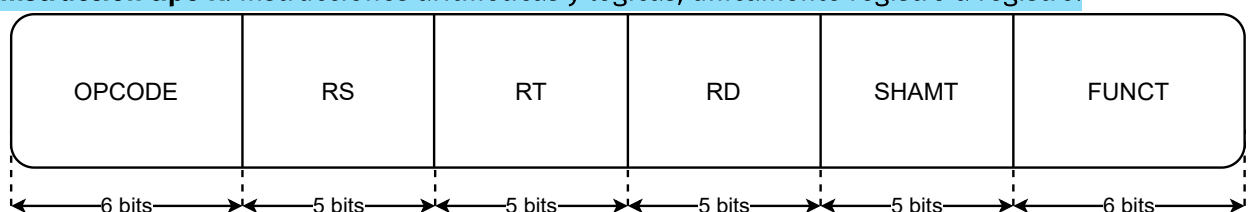
Formato de instrucción

Conjunto de instrucciones de formato fijo, cada instrucción formada por 32 bits (1 palabra):

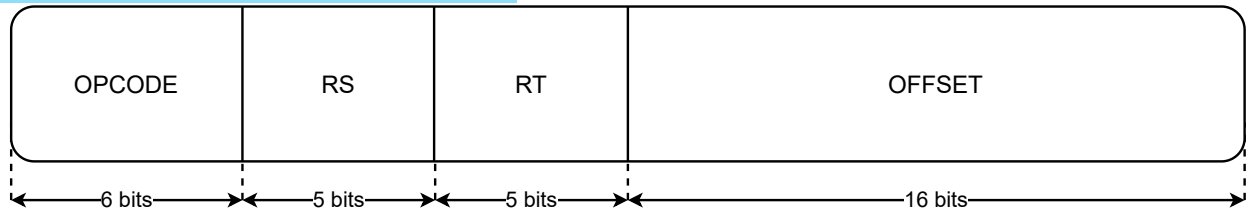
- **Instrucción tipo I:** instrucciones de acceso a memoria. **LOAD** y **STORE** son las únicas instrucciones que acceden a memoria.



- **Instrucción tipo R:** instrucciones aritméticas y lógicas, únicamente registro a registro.



- **Instrucción tipo J:** instrucciones de salto.



- RS : *register source*.
- RT : no es un acrónimo, T siguiente a S.
- RD : *register destination*.
- SHAMT : *shift amount* (0 cuando no es aplicable).
- FUNCT : *function code*, identifica el formato de la instrucción R.
- OFFSET : dirección, valor inmediato.

Instrucciones

Transferencia de datos:

- `lb` : load byte (8 bits)
- `lbu` : load byte unsigned (8 bits)
- `lh` : load halfword (16 bits)
- `lhu` : load halfword unsigned (16 bits)
- `lw` : load word (32 bits)
- `lwu` : load word unsigned (32 bits)
- `ld` : load double word (64 bits)
- `ld` : load floating point (64 bits)
- `sb` : store byte (8 bits)
- `sh` : store halfword (16 bits)
- `sw` : store word (32 bits)
- `sd` : store double word (64 bits)
- `sd` : store floating point (64 bits)
- `mov.d` : move data between floating point registers
- `mtc1` : move to coprocessor 1 (integer to floating point)
- `mfc1` : move from coprocessor 1 (floating point to integer)
- `cvt.d.1` : convert integer to double precision floating point
- `cvt.1.d` : convert double precision floating point to integer (truncates)

Aritméticas:

- `dadd` : add registers (signed)
- `daddi` : add a register with an immediate (signed)
- `daddu` : add registers (unsigned)
- `daddui` : add a register with an immediate (unsigned)
- `add.d` : add double precision floating point registers
- `dsub` : subtract registers (signed)
- `dsubu` : subtract registers (unsigned)
- `sub.d` : subtract double-floating point registers
- `dmul` : multiply registers (signed)

- `dmulu` : multiply registers (unsigned)
- `mul.d` : multiply floating point registers
- `ddiv` : divide registers (signed)
- `ddivu` : divide registers (unsigned)
- `div.d` : divide floating point registers
- `dsll` : double word shift left logical immediate
- `dsllv` : double word shift left logical register
- `dsrl` : double word shift right logical immediate
- `dsrlv` : double word shift right logical register
- `dsra` : double word shift right arithmetic immediate (keeps sign)
- `dsrav` : double word shift right arithmetic register (keeps sign)
- `slt` : set if less than
- `slti` : set if less than immediate

estas instrucciones alteran el flag FP:

- `c.lt.d` : compare less than double precision floating point
- `c.le.d` : compare less or equal than double precision floating point
- `c.eq.d` : compare equal double precision floating point

Lógicas:

- `and` : bitwise AND
- `andi` : bitwise AND with an immediate value
- `or` : bitwise OR
- `ori` : bitwise OR with an immediate value
- `xor` : bitwise XOR
- `xori` : bitwise XOR with an immediate value

Instrucciones de Transferencia de Control:

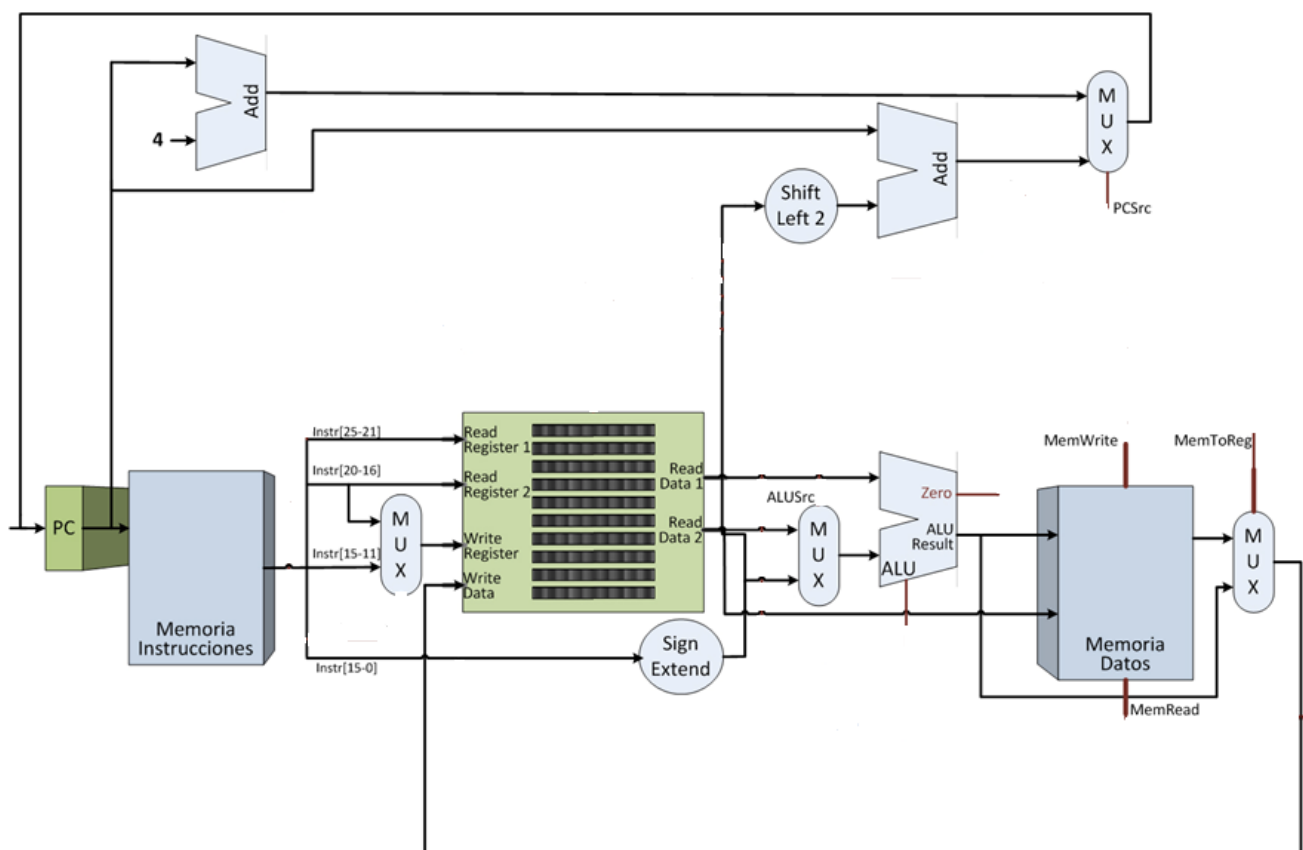
- `j` : jump
- `jal` : jump and link (ret addr in `r31`)
- `jr` : jump register
- `beq` : branch if equal
- `bne` : branch if not equal
- `beqz` : branch if equal zero
- `bnez` : branch if not equal zero
- `bc1f` : branch if floating point flag (FP) is false
- `bc1t` : branch if floating point flag (FP) is true

Directivas del compilador

- `.code` o `.text` : indican al compilador que lo que sigue es código fuente. El compilador asigna automáticamente la posición en memoria del programa.
- `.data` : indica al compilador que el texto que sigue son variables. El compilador asigna automáticamente la dirección en memoria de los datos.

- `.org {direccion}`: ubica el texto que sigue a partir de una dirección específica. Se puede usar tanto con código como datos.
- `space {cant}`: deja un espacio de bytes entre lo anterior y lo siguiente.
- `.ascii {cadena}`: almacena una cadena ASCII.
- `asciiz {cadena}`: almacena una cadena ASCII terminada en 0.
- `.byte {n}`: almacena bytes.
- `.word16 {n}`: almacena números de 16 bits.
- `.word32 {n}`: almacena números de 32 bits.
- `.word {n}`: almacena números de 64 bits.
- `.double {n}`: almacena un número/s en punto flotante.
- `halt`: detiene al procesador.
- `nop`: no hace nada.

Ruta de datos



La **ALU** es una unidad de cálculo típica de dos entradas y una salida. En la entrada se pueden seleccionar diferentes salidas.

La **unidad de cálculo de dirección de próxima instrucción** es quien determina el próximo valor del **PC**. Puede ser el de la instrucción consecutiva ($PC + 4$), o sumando un desplazamiento.

Existen **dos bancos de memoria**:

- **Memoria de instrucciones:** solo contiene el programa a ejecutar. Se accede únicamente para lectura, en la fase de búsqueda de la instrucción, y accedida solo a través del **PC**.

- **Memoria de datos:** contiene los datos a leer o escribir. Solo se accede a través de las instrucciones `LOAD` y `STORE`.

La **CU** es relativamente sencilla, captura la instrucción y a partir de ella genera las señales de control requeridas por los demás bloques operativos. Utiliza el **OPCODE** para determinar que tareas deberá realizar para completar la instrucción corriente.

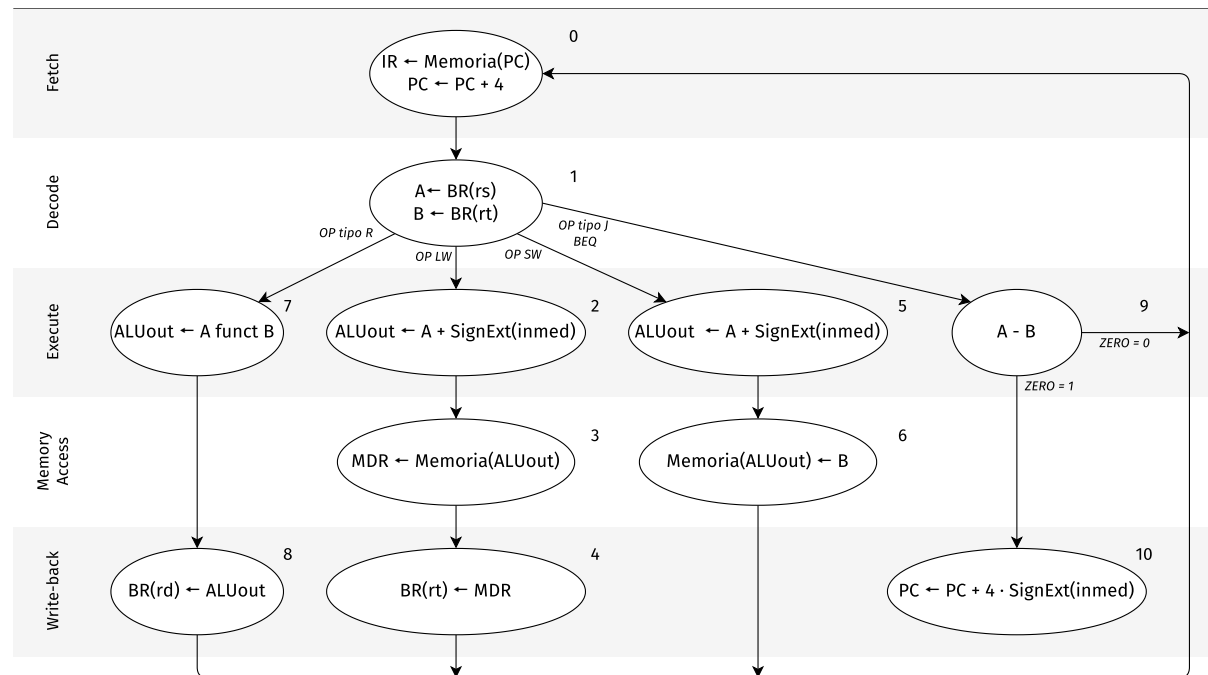
Cauce y el ciclo de instrucción

Esta arquitectura posee un cauce segmentado. Con una secuencia de instrucciones compuesta por cinco estados/ciclos. No todas las tareas o segmentos duran la misma cantidad de ciclos de reloj (tiempo).

Entre las unidades funcionales que llevan a cabo cada etapa, existen **registros sincrónicos**. Los datos avanzan de etapa a etapa en cada ciclo de reloj, cuando esto sucede, el registro de separación de la etapa copia a la salida lo que tiene en su entrada.

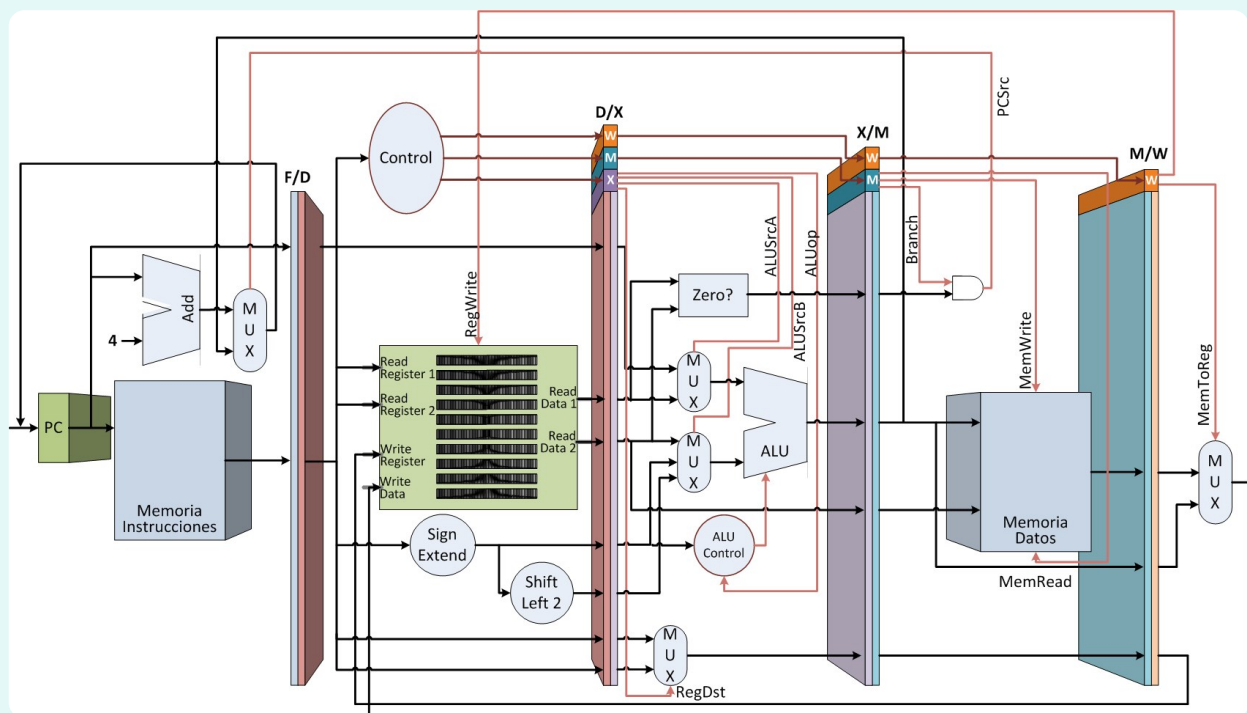
1. **Búsqueda de instrucción** (*fetch*, IF):
 - Buscar y leer la instrucción que indica el PC en la memoria de instrucciones.
 - Actualizar el *Program Counter* ($PC = PC + 4$, instrucciones de 32 bits)
 - `nop` y `fetch` solo actúan en esta fase.
2. **Decodificación** (*decode*, ID):
 - Decodificar la instrucción, separando sus campos, se determina el tipo de operación (*codop*).
 - Si es necesario, leer los operandos del banco de registros (todas menos `j`, `jr`, `bc1f`, `bc1t`).
 - Cuentas: si es operando inmediato (16 bits), se calcula el valor (extiende a 64 bits).
 - En los saltos: se calcula la dirección de destino y si se determina si debe saltar o no.
3. **Ejecución** (*execute*, E):
 - Cuentas: Ejecutar la operación aritmética o lógica en la ALU (menos multiplicación y división) que indica el *codop*.
 - Memoria: se calcula la dirección efectiva.
 - Saltos: se almacena la dirección en PC.
4. **Acceso a memoria** (*memory access*, M) (eventual):
 - Ocurre solo en las instrucciones `LOAD` y `STORE`.
 - Acceder a memoria.
5. **Almacenamiento en registro** (*writeback*, W) (eventual):
 - Se almacena el resultado en el registro de destino.
 - Ocurre solo en las instrucciones que almacenan un dato en un registro.
 - También es en esta fase cuando se puede calcular el desplazamiento al sumar al PC en instrucciones de salto.

Segmentación en el nanoMIPS



- MI: memoria de instrucciones
- BR: banco de registros
- ALU: unidad aritmético lógica
- MD: memoria de datos

Ruta de datos:



Se observa cada etapa de izquierda a derecha (F, D, X, M, W), separada por los registros sincrónicos.

Ejemplo: Etapas de una instrucción tipo I

LD R2, VARIABLE(R1)

- IF: recuperar la instrucción de MI.
- ID: la decodifica, accede a R1 y lo almacena en un registro temporal.
- EX: calcula la dirección efectiva [VARIABLE + R1].
- MEM: recupera el valor de la dirección de MD y lo almacena en un registro temporal.
- WB: guarda en R2 el valor leído de memoria (lo transfiere desde el registro temporal)

Ejemplo: Etapas de una instrucción tipo I

SD R2, VARIABLE(R1)

- IF: Recupera la instrucción de MI.
- ID: la decodifica, recupera el valor de R2 y R1 y los guarda en registros temporales.
- EX: calcula la dirección efectiva [VARIABLE + R1]
- MEM: guarda el valor de R2 en la dirección [VARIABLE + R1] de MD.
- WB: No hace nada.

Ejemplo: Etapas de una instrucción tipo R

DADD R3, R2, R1

- IF: recupera la instrucción de MI.
- ID: la decodifica, recupera el valor de R1 y R2 y los guarda en registros temporales.
- EX: calcula la suma de R1 y R2 y guarda el resultado en un registro temporal.
- MEM: no hace nada.
- WB: guarda el resultado desde el registro temporal en R3

Ejemplo: Etapas de una instrucción tipo J

BEQ R1, R2, etiqueta

- IF: recupera la instrucción de MI.
- ID: la decodifica, recupera el valor de R2 y R1. Determina si salta o no. Calcula la dirección de salto.
- EX: si determinó que debe saltar, ejecuta el salto.
- MEM: no hace nada.
- WB: no hace nada

Calculo del CPI en el nanoMIPS

Debido las 5 etapas en que se divide la ruta de datos, podemos calcular la cantidad de ciclos como $4 + \text{cantidad de instrucciones}$. Quedando finalmente:

$$CPI = \frac{4 + \text{cantidad de instrucciones}}{\text{cantidad de instrucciones}}$$

Ejemplo:

- 10 instrucciones:

$$CPI = \frac{(4 + 10)}{10} = 1,4$$

- 100 instrucciones:

$$CPI = \frac{(4 + 100)}{100} = 1,04$$

- 1000 instrucciones:

$$CPI = \frac{(4 + 1000)}{1000} = 1,004$$

Notar que para un ciclo continuo de ejecución, a medida que aumenta el número de instrucciones, el CPI se aproximará a 1.

Conflictos por el cause segmentado

La ejecución paralela de las instrucciones introduce **atascos**.

- Los **atascos estructurales** se dan solo en la etapa MEM, cuando dos instrucción intentan hacer uso de la memoria de datos en el mismo ciclo. Pasará primero a la etapa la instrucción que comenzó a ejecutarse con anterioridad, mientras la segunda espera.
- La manera primitiva de lidiar con atascos por **dependencia de datos** es el **reordenamiento de instrucciones**, de manera que, sin alterar la lógica del programa, se minimicen o prevengan los stalls.
- Para resolver un **riesgo de control** en el nanoMIPS mediante paradas, son necesarios tres ciclos de espera hasta que la instrucción de salto, carga el valor adecuado en el PC y se puede continuar con la ejecución.

Cálculo del CPI con atascos

Si agregamos la posibilidad de que sucedan atascos al cálculo del CPI:

$$CPI = \frac{4 + \text{cantidad de instrucciones} + \text{retrasos por atasco}}{\text{cantidad de instrucciones}}$$

Forwarding

El **forwarding** permitirá el acceso al resultado de una instrucción en un registro, antes de que esté en la etapa de writeback. Las etapas EX y MEM tendrán un buffer de resultados para guardar valores intermedios de manera temporal, que estarán disponibles para las etapas ID, EX y MEM. Las instrucciones solo se atascan cuando van a utilizar los operandos y no están.

Es una técnica por hardware, para solventar *data hazards* y *control hazards*.

- La etapa IF: búsqueda. No se verá modificada.
- La etapa ID: decodificación. Ya no verifica los operandos (salvo para saltos condicionales). Osea, solo produce un atasco si el operando no está en un salto.
- La etapa EX: ejecución. Deja el resultado en un registro temporario. Produce un atasco si no está el operando.
- La etapa MEM: escribir o leer de memoria. Si es `ld` deja el resultado en un registro temporario. Produce un atasco si no está el operando.
- La etapa WB: almacenar el resultado en registro destino. No se verá modificada.
- Las Instrucciones aritmético-lógicas se atascan en EX.
- Las Instrucciones `ld` / `sd` se atascan en EX (si no está el desplazamiento).
- Las Instrucción `sd` se atasca en MEM (si no está el valor a guardar).

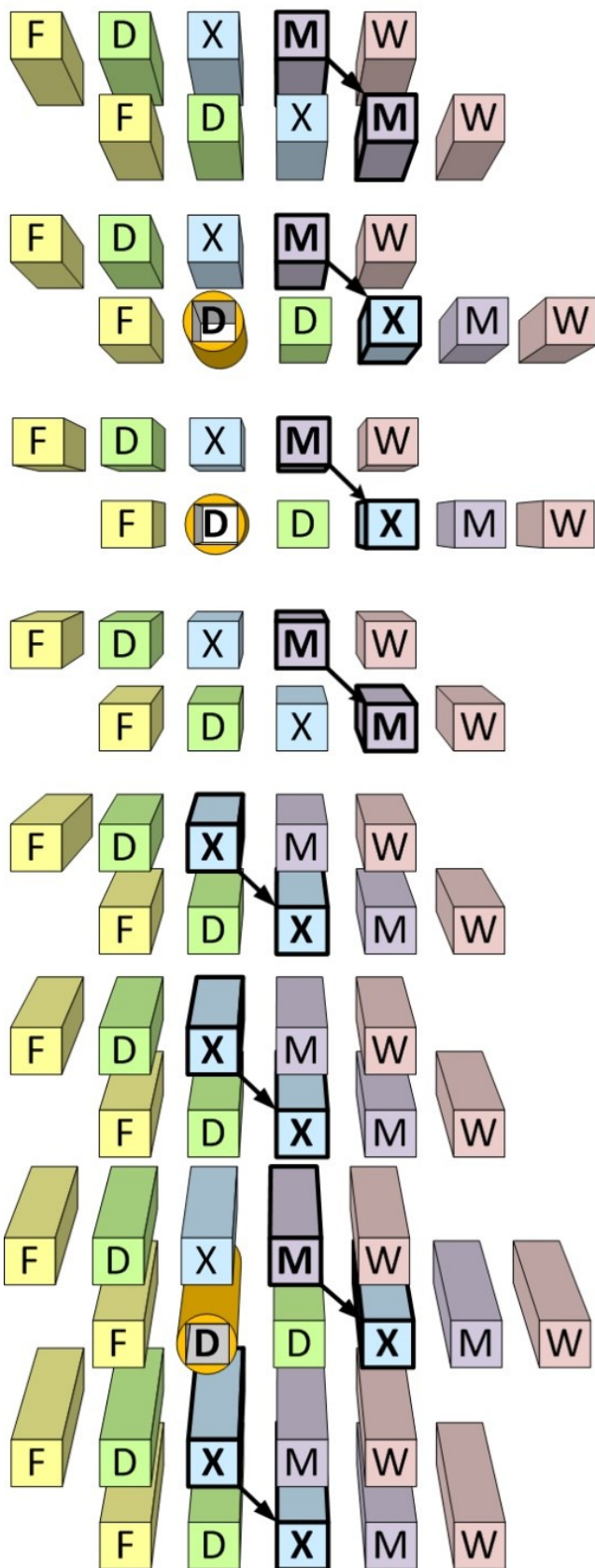
Info

En el simulador WinMIPS64, en la ventana *Register*, veremos los registros con colores:

- Gris: el registro se encuentra en proceso de ser escrito por alguna instrucción.
- Si se muestra algún registro color, significa que el registro se encuentra disponible para adelantar a otra instrucción, desde la etapa del cause que le corresponda al color:
 - Amarillo → IF
 - Cyan → ID
 - Rojo → EX
 - Verde → MEM
 - Violeta → WB

En nanoMIPS se pueden identificar 8 tipos de conflictos por **dependencia de datos** de tipo RAW, clasificados por solución:

- Adelantamiento desde la salida de M a la entrada de M:
 - `lw` → `sw`
 - aritmético-lógica → `sw`
- Adelantamiento desde la salida de M a la entrada de X:
 - `lw` → aritmético-lógica
 - `lw` → `beq`
 - `lw` → `lw`
- Adelantamiento desde la salida de X a la entrada de X:
 - aritmético-lógica → instrucción tipo `r`
 - aritmético-lógica → `beq`
 - aritmético-lógica → `lw`



LW R1, 100(R2)

SW R1, 0(R10)

LW R1, 100(R2)

ADD R3, R1, R8

LW R1, 100(R2)

BEQ R1, R3, etiqueta

ADD R1, R2, R3

SW R1, 0(R10)

ADD R1, R2, R3

SUB R8, R1, R6

ADD R1, R2, R3

BEQ R1, R7, etiqueta

LW R1, 0(R7)

LW R10, 100(R1)

ADD R1, R2, R3

LW R10, 100(R1)

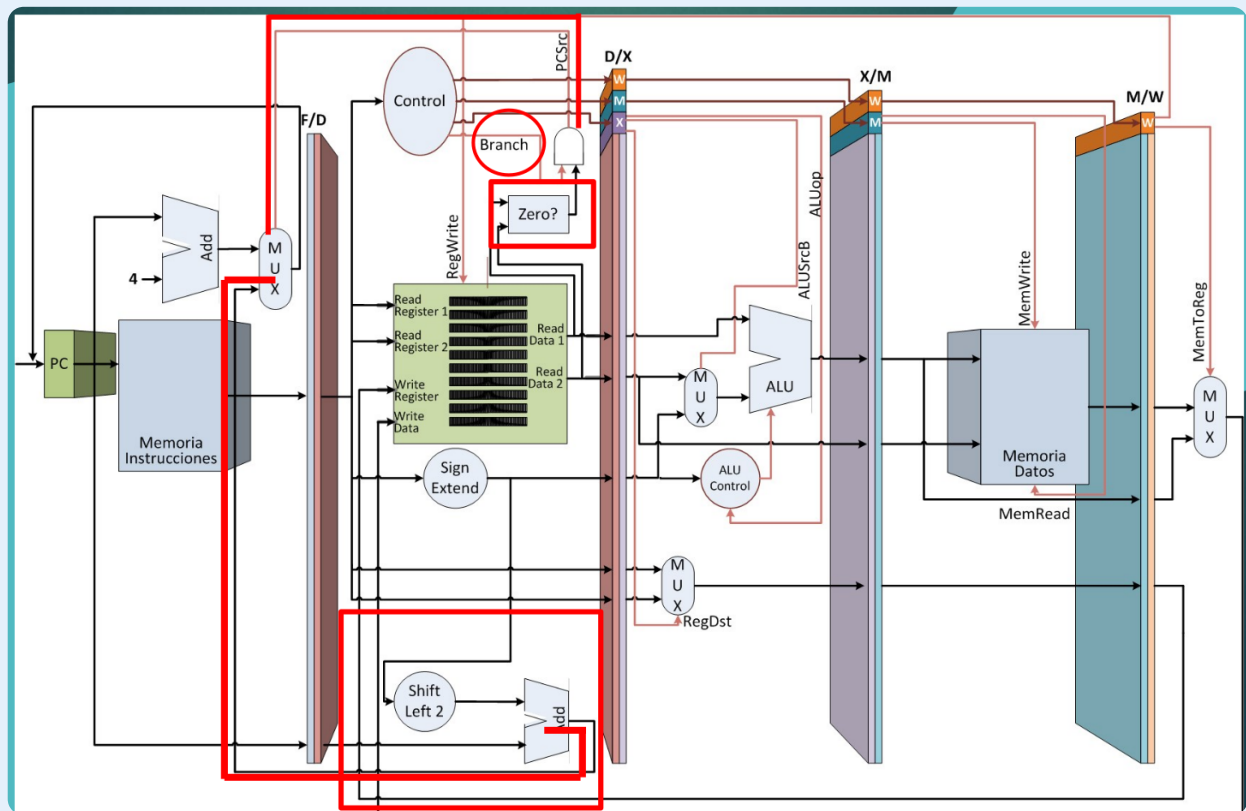
El forwarding puede solventar **conflictos por dependencia de control** sin utilizar retardos.

Riesgo de control y forwarding

Como es en la etapa D en la que se decodifica la instrucción y se sabe que es un salto, en esta misma etapa se añade un restador, el módulo **Zero?**, para evaluar la condición de igualdad del salto, evitando así esperar hasta la etapa X asociada a la ALU.

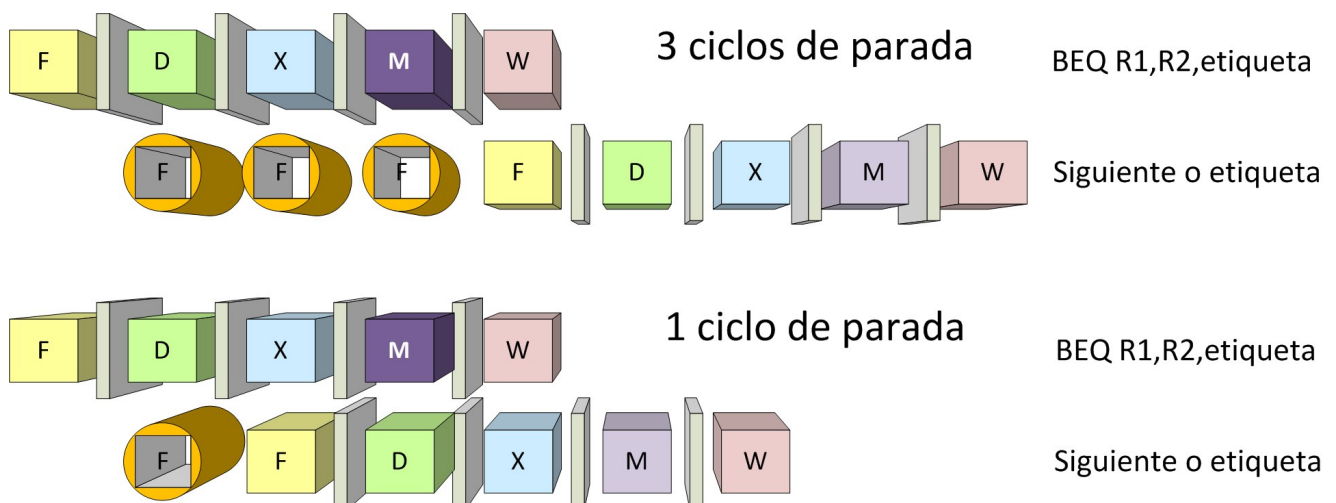
Para calcular la dirección destino del salto se añade un sumador, evitando esperar hasta la etapa X asociada a la ALU.

Aparece un nuevo adelantamiento a la etapa D (cuando hay un riesgo de datos RAW con uno de los registros necesarios para la evaluación de la condición del salto), lo que exige añadir el hardware necesario para realizar los adelantamientos también en esta etapa.



El MUX en el restador a la salida del banco de registros selecciona el valor que se va a cargar en el PC. Una de las entradas es el valor actual del PC con el desplazamiento incluido en la instrucción de BEQ.

Con la ruta de datos modificada para el adelantamiento, la respuesta del nanoMIPS a una instrucción de salto **BEQ** es perder solo un ciclo de reloj, en lugar de los tres que se perdían originalmente.



Predicción de salto, BTB

El nanoMIPS utiliza una técnica de **predicción de salto estática**. Siempre capta la próxima instrucción, o sea, predice que no va a saltar. La instrucción posterior al salto ingresará al cause hasta la etapa IF. En caso de no saltar se produce un **Branch Taken Stalls (BTS)**, y descarta el ciclo de búsqueda IF que no modificó el procesador. Si no salta no se pierde ningún ciclo, si salta se pierde 1 ciclo.

La técnica de **predicción dinámica** que utiliza nanoMIPS se denomina **Branch Target Buffer (BTB)**. Es una pequeña memoria asociativa^[5] que guarda las direcciones de los últimos saltos ejecutados así como su destino. A su vez guarda información que permite predecir si el salto será tomado o no.

En la etapa de fetch se mira si la dirección de la instrucción está en el BTB. Si es así se miran los bits de predicción y se decide si el salto ha de ser tomado o no, dependiendo el resultado de la última vez que se produjo el salto. Si el salto no es tomado o la dirección no está en el buffer, en el siguiente ciclo se hace el fetch de la siguiente instrucción, en orden. Si el salto es tomado en el siguiente ciclo se hace el fetch del nuevo camino de ejecución.

El BTB es conveniente cuando en un bucle la cantidad de iteraciones es alta, y los saltos no son cambiantes. Al predecir el salto condicional en cada iteración, fallará en predecir la/s primeras veces queda deba saltar y fallará en la/s últimas. Cuanto más veces itere el programa, mayor será el impacto positivo que tendrá en reducir el atasco.

Los atascos producto de una predicción errónea se denominan **Branch Misprediction Stall (BMS)**. Cada BMS genera 2 atascos, uno por cargar la instrucción incorrecta, y otro por tener que actualizar la tabla o buffer del BTB.

Ejemplo: loop sin BTB

```
ld    r2, b(r0)
ld    r1, a(r0)
loop: beqz r2, fin
      daddi r2, r2, -1
      dsll r1, r1, 1
      j     loop
fin:   halt
```

Sin branch target buffer, en cada iteración, en b-1 saltos, se cargará la etapa IF de `halt` y será descartada.

Con btb, habrían 4 atascos. 2 BTS en la primera iteración y 2 BMS en la última iteración.

Branch delay slot

Con el delay slot, el nanoMIPS dejará avanzar en el cause la instrucción siguiente al salto, más allá de la etapa IF, que normalmente deja como predicción de saltos estática.

Cambia la forma en que se ejecutan los saltos. Garantiza la ausencia de atascos, pero debemos tomar ciertas precauciones. Usualmente un programa debe ser reescrito para funcionar con delay slot.

Escribir un programa para una máquina con delay slot consiste en reordenar las instrucciones de manera que al ingresar al cause la instrucción siguiente a un salto, no se produzcan otro tipos de stalls, ni produzca un comportamiento no deseado. Si no es posible, se introducen sentencias `nop`.

Modificar programa para funcionar con delay slot

Programa original:

```
loop: dsll r1, r1, 1
      daddi r2, r2, -1
      bnez r2, loop
      halt
```

Corregido con `nop`:

```
loop: dsll r1, r1, 1
      daddi r2, r2, -1
      bnez r2, loop
      nop
      halt
```

Corregido con reordenamiento:

```
loop: daddi r2, r2, -1
      bnez r2, loop
      dsll r1, r1, 1
      halt
```

Siempre será posible corregir con `nop` pero disminuye el rendimiento. Siempre que sea posible se debe utilizar el reordenamiento. Ojo, el reordenamiento puede generar atascos raw.

Ejemplo: problema con delay slot

```
...
seguir: daddi r1, r2, 0
...
bnez r1, seguir
nop
halt
```

Utilizando delay slot, al llegar al salto condicional, también avanzará en el cause la instrucción `nop`. Si no estuviera y quien ingresa al cause luego del `bnez` es `halt`, se detendrá la ejecución del programa, aún cuando el salto evalúe diferente.

Unidades de ejecución y punto flotante

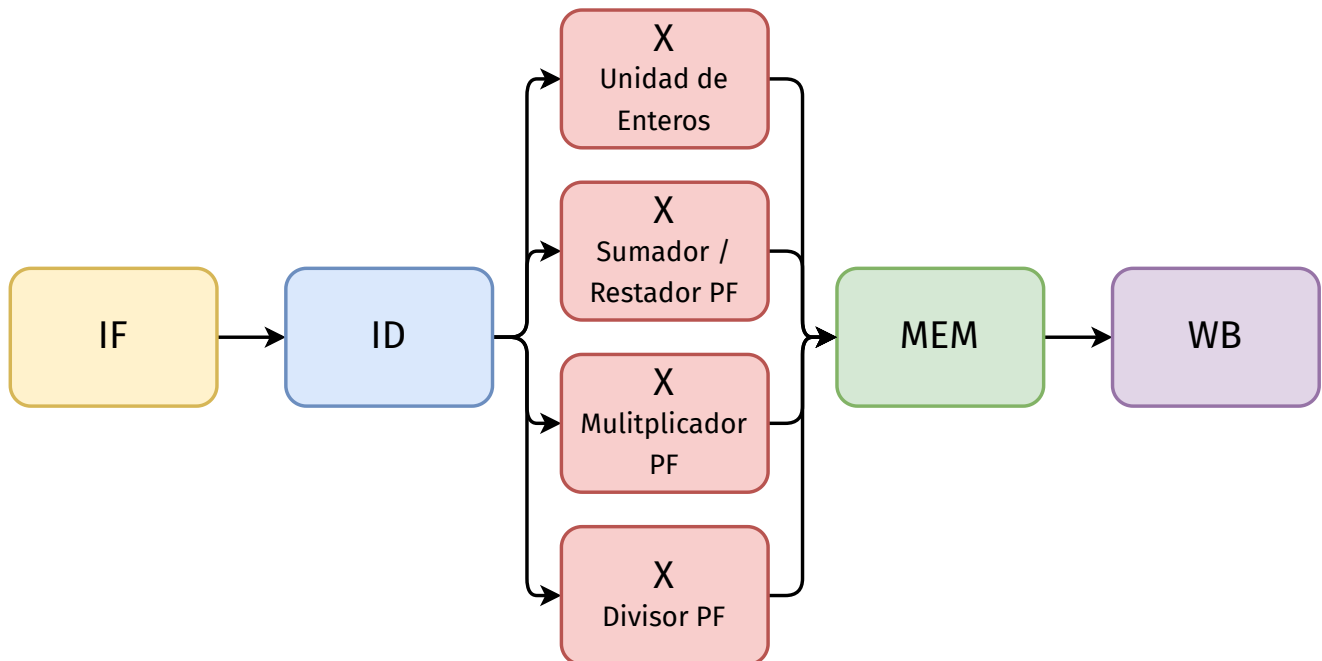
De la aritmética de punto flotante se encarga un chip de hardware separado denominado **Coprocessor 1**. Tiene su propia unidad aritmética y 32 registros dedicados a operaciones con punto flotante (F0 .. F31).

La arquitectura MIPS soporta operaciones en IEEE754 single y doble precision, pero solo usamos doble precisión en la práctica.

Las instrucciones de punto flotante tienen un `s` y terminan en `s` o `d`, dependiendo si son precisión simple o doble respectivamente.

La ruta de datos es **multifuncional** (también llamada **no lineal o bifurcada**), de manera que más de una instrucción puede estar en la etapa de ejecución en un mismo ciclo de reloj, cada una siendo atendida en diferentes unidades funcionales, que llevan a cabo una tarea particular. Pudiendo algunas requerir más de un ciclo en finalizar su operación.

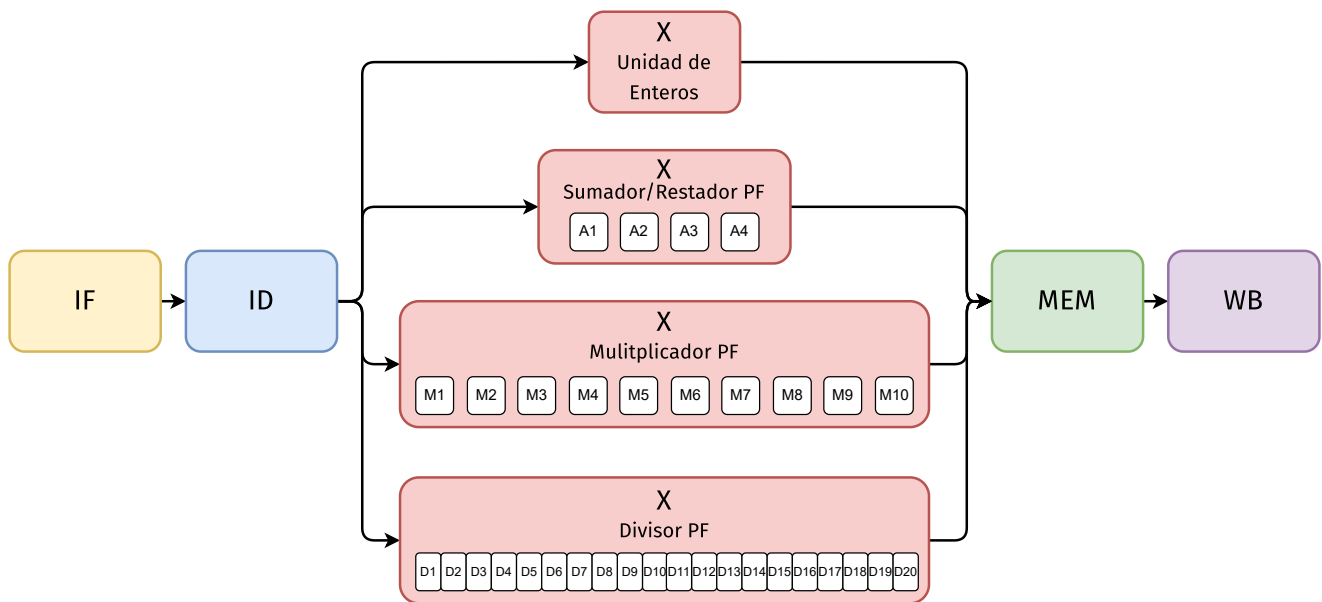
nanoMIPS tiene 4 unidades de ejecución, que pueden operar de manera simultánea en el cauce:



Los tiempos de ejecución de las unidades de punto flotante son muy distintos de los de la unidad de enteros. Introduciendo un problema: dos instrucciones consecutivas en punto flotante requeriría insertar paradas antes de la segunda instrucción, hasta que la primera termine.

La solución es segmentar las unidades funcionales de punto flotante, para poder iniciar nuevas instrucciones, en una misma unidad, sin haber completado la actual.

1. Para instrucciones generales (1 etapa de 1 ciclo).
2. Para sumar punto flotante (4 etapas de 1 ciclo c/u).
3. Para multiplicar punto flotante y enteros (7 etapas de 1 ciclo c/u).
4. Para dividir en punto flotante y enteros (1 etapa de 24 ciclos).



Ejemplo: programa que opera con punto flotante

```

; Convertir el valor de A a punto fijo
; y guardarlo en B
.data
A: .double 5
B: .word 0

.code
l.d f2, A(r0)
cvt.l.d f2, f2 ; convierte de pf a entero en f2
mfc1 r4, f2 ; mueve el valor de un registro fp a uno de proposito general
sd r4, B(r0)
halt
  
```

Subrutinas y pila

La instrucción `jal jump and link` invoca a la subrutina, guarda en el registro `r31` la dirección a la instrucción de retorno (siguiente a `jal`) y carga el `PC` con la dirección correspondiente a la etiqueta de la subrutina.

Para retornar de la subrutina, basta con saltar a la dirección contenida en el registro `r31`, con la instrucción `jr`.

Este mecanismo presenta el inconveniente que al anidar subrutinas se termina sobrescribiendo la dirección de retorno en `r31` para cada subrutina invocada. También puede ocurrir una subrutina haga uso de un registro que otra esté usando.

Se les da nombres más convenientes a los registros de propósito general, y un uso particular. Permitiendo que cualquier subrutina pueda ser usada junto a otras:

Registros	Nombres	Usos	Preservado
<code>r0</code>	<code>\$zero</code>	Siempre tiene 0	

Registros	Nombres	Usos	Preservado
r1	\$at	<i>Assembler Temporary</i> - Reservado para ser usado por el ensamblador.	
r2 - r3	\$v0-\$v1	Valores de retorno de la subrutina llamada.	
r4 - r7	\$a0-\$a3	Argumentos pasados a la subrutina llamada.	
r8 - r15	\$t0-\$t7	Registros temporarios. No son conservados en el llamado a subrutinas.	
r16 - r23	\$s0-\$s7\$	(<i>saved</i>) Registros salvados durante el llamado a subrutinas. Normalmente variables locales.	✗
r24 - r25	\$t8-\$t9	Registros temporarios. No son conservados en el llamado a subrutinas.	
r26 - r27	\$k0-k1\$	Para uso del kernel del sistema operativo.	
r28	\$gp	<i>Global Pointer</i> - Puntero a la zona de la memoria estática del programa.	✗
r29	\$sp	<i>Stack Pointer</i> - Puntero al tope de la pila.	✗
r30	\$fp	<i>Frame Pointer</i> - Puntero al marco actual de la pila.	✗
r31	\$ra	<i>Return Address</i> - Dirección de retorno en un llamado a una subrutina.	✗

Los registros marcados con una ✗, EN CASO DE SER NECESARIO ALTERARLOS, deben ser salvados por la subrutina y restaurados antes de finalizar.

Ejemplo de invocación a una subrutina, sin pila:

```

.data
result: .word 0

.text
daddi $a0, $0, 10
daddi $a1, $0, 20
jal sumar
sd $v0, result($0)
halt

sumar: dadd $v0, $a0, $a1
jr $ra

```

Toda subrutina se dividirá siempre en tres partes:

- El prólogo: se resguarda en la pila todos los registros que deban ser preservados.
- El cuerpo: se escribirá el código propio de la subrutina.
- El epílogo: se deberá deshacer todo lo hecho en el prólogo.

Los valores que debemos guardar, serán almacenados en una **pila**. El registro r29 (\$sp por convención) se usará como puntero al tope de la pila.

Dado que la pila será implementada por el programa, se tiene la libertad de elegir como trabajar con la misma:

- Se puede definir una pila que comience apuntando a una dirección de memoria dada y cada vez que se apile un valor, el `$sp` se incremente.
- Se puede definir una pila que al estar vacía apunte al tope de la memoria y cada vez que se apile un valor, el `$sp` se decremente. Salvo que se indique, toda subrutina asumirá que el registro `$sp` apunta al tope de la pila. En particular, dado que la última dirección en winMIPS es `0x400`, podemos inicializar la pila en este valor y que crezca hacia direcciones menores, como en el `vonsim`.

Ejemplo: inicializar una pila, push y pop

```
.code
; inicializar pila
daddi $sp, $zero, 0x400

; cargar valor en $t0
daddi $t0, $zero, 12

; push, apilar un valor al tope de la pila
daddi $sp, $sp, -8
sd $t0, 0($sp)

; pop, desapilar un valor de la pila
ld $t1, 0($sp)
daddi $sp, $sp, 8
```

Ejemplo: subrutina que preserva registros usando la pila

```
subrutina:
prologo:    daddi $sp, $sp, -tamaño_frame ; reserva espacio en la pila
            sd     $ra, 0($sp)           ; guarda la dirección de retorno
            sd     $s0, 8($sp)          ; guarda el registro $s0
            sd     $s1, 16($sp)         ; guarda el registro $s1
            ...
cuerpo:     ...                         ; cuerpo de la subrutina
            ...
            ...
epilogo:    ld     $ra, 0($sp)           ; recupera la dirección de retorno
            ld     $s0, 8($sp)          ; recupera el registro $s0
            ld     $s1, 16($sp)         ; recupera el registro $s1
            ...
            daddi $sp, $sp, tamaño_frame ; restaura el tope de la pila
            jr     $ra                  ; retorna
```

El tamaño del frame, o la porción de la pila que usará una subrutina en cada llamado, lo calcularemos como:

$$8 \text{ bytes} \times (\text{número de registros a guardar} + \text{número de variables locales a la subrutina})$$

Una vez que se hizo lugar en el tope de la pila, es posible usar `$sp` como registro base e indicar un desplazamiento como valor inmediato, usando un modo de direccionamiento relativo a la base desplazada.

Cabe aclarar que no es necesario guardar registros, aún si pertenece a los que deben ser salvados, si la subrutina no los modifica.

E/S

Utilizaremos el puerto E/S mapeada en memoria, las direcciones de memoria de E/S se encuentran en una porción de la memoria principal, no hay instrucciones específicas sus operaciones.

`CONTROL (0x10000)` y `DATA (0x10008)` son direcciones de memoria fijas para acceder al puerto de E/S. Aplicando distintos comandos a través de `CONTROL`, es posible producir salidas o ingresar datos a través de la dirección `DATA`.

Como el set de instrucciones del procesador MIPS64 no cuenta con instrucciones que permitan cargar un valor inmediato de más de 16 bits (las instrucciones mencionadas son de 32 bits), resulta conveniente definir las en la memoria de datos:

```
CONTROL: .word32 0x10000
DATA:    .word32 0x10008
```

De esa manera, resulta sencillo cargar esas dos direcciones en registros:

```
lwu $t0, CONTROL($zero) ; Carga el valor 0x10000 en $t0
lwu $t1, DATA($zero)   ; Carga el valor 0x10008 en $t1
```

Para escribir un valor:

```
daddi $t3, $zero, 4
sw     $t3, 0($t0) ; control = 4
```

Terminal alfanumérica:

- Imprimir entero sin signo: `CONTROL = 1` $\implies$ `DATA` $\rightarrow$ entero sin signo.
- Imprimir entero con signo: `CONTROL = 2` $\implies$ `DATA` $\rightarrow$ entero con signo.
- Imprimir punto flotante: `CONTROL = 3` $\implies$ `DATA` $\rightarrow$ punto flotante.
- Imprimir string: `CONTROL = 4` $\implies$ `DATA` $\rightarrow$ dirección de una string terminada en cero.
- Limpiar la consola: `CONTROL = 6`.

Pantalla gráfica:

La pantalla gráfica cuenta con una resolución de 50×50 puntos. Siendo (0,0) las coordenadas del punto en la esquina inferior izquierda y (49,49) las del punto en la esquina superior derecha.

- Dibujar un punto en la pantalla gráfica: `CONTROL = 5`, $\implies$ `DATA` $\rightarrow$ es un color RGBA de 4 bytes. `data+5` es la coordenada x y `DATA+4` es la coordenada y .
- Limpiar la pantalla gráfica: `CONTROL = 7`

Entrada:

- `CONTROL = 8` $\implies$ `DATA` $\rightarrow$ un numero leído de teclado.
- `CONTROL = 9` $\implies$, `DATA` $\rightarrow$ un byte (1 caracter ASCII) leído de teclado.

Ejemplo: mostrar en el terminal

Suma `A` y `B`, lo guarda en `C` y lo muestra en el terminal.

```
.data
A: .word 10
B: .word 8
C: .word 0
CR: .word32 0x10000
DR: .word32 0x10008

.text
main: ld r4, A(r0)
      ld r5, B(r0)
      dadd r3, r4, r5
      sd r3, C(r0)
      lwu r1, CR(r0) ; registro control
      lwu r2, DR(r0) ; registro data
      daddi r10, r0, 1
      sd r3, (r2) ; enviar r3...
      sd r10, (r1) ; ...a la terminal
      halt
```

Ejemplo: dibujar en pantalla gráfica

Imprime un punto en (25, 25) de color rojo.

```
.data
control: .word32 0x10000
data: .word32 0x10008

.code
lwu $t0, control($0)
lwu $t1, data($0)
daddi $t2, $0, 7 ; limpiar pantalla
sw $t2, 0($t0)
; cargar color en data
daddi $t3, $0, 255
sb $t3, 0($t1) ; r = 255
sb $0, 1($t1) ; g = 0
sb $0, 2($t1) ; b = 0
sb $t3, 3($t1) ; a = 255
; cargar x e y en data (25, 25)
daddi $t3, $0, 25
daddi $t4, $0, 25
sb $t3, 5($t1)
sb $t4, 4($t1)
```



```

; dibujar punto
daddi $t3, $0, 5
sw    $t3, 0($t0) ; control = 5
halt

```

Ejemplo: dibujar en la pantalla gráfica

Dibujar una línea verde desde (24, 1) hasta (24, 24).

```

.data
control: .word32 0x10000
data:    .word32 0x10008
coorX:   .byte 24
coorY:   .byte 1
color:   .byte 0, 255, 0, 255
.code
lwu $s6, control($zero)
lwu $s7, data($zero)
; coordenada x
lbu $s0, coorX($0)
sb  $s0, 5($s7)
; color
lwu $s2, color($0)
sw  $s2, 0($s7)
; coordenada y
daddi $t0, $0, 5
lbu $s1, coorY($0)
loop: sb $s1, 4($s7)
      sd $t0, 0($s6)
      daddi $s1, $s1, 1
      slti $s3, $s1, 24
      bnez $s3, loop
      halt

```

Ejemplo: dibujar en pantalla gráfica

Cambiar el color de un pixel desde negro, hasta rojo

```

.data
coorX:   .byte 24
coorY:   .byte 10
color:   .byte 0, 0, 0, 255
control: .word32 0x10000
data:    .word32 0x10008
.code
lwu $s6, control($0)
lwu $s7, data($0)
; coordenada x
lbu $s0, coorX($0)
sb  $s0, 5($s7)

```

```

; coordenada y
lbu $s1, coorY($0)
sb $s1, 4($s7)
; color
daddi $t0, $0, 5
lbu $s1, color+1($s0)
lbu $s2, color+2($s0)
lbu $s3, color+3($s0)
lbu $s0, color($s0)
sb $s1, 1($s7)
sb $s2, 2($s7)
sb $s3, 3($s7)
loop: sb $s2, 0($s7) ; r
sd $t0, 0($s6) ; control = 5
daddi $s2, $s2, 32
slti $s3, $s2, 256
bnez $s3, loop
halt

```

Memoria

Introducción

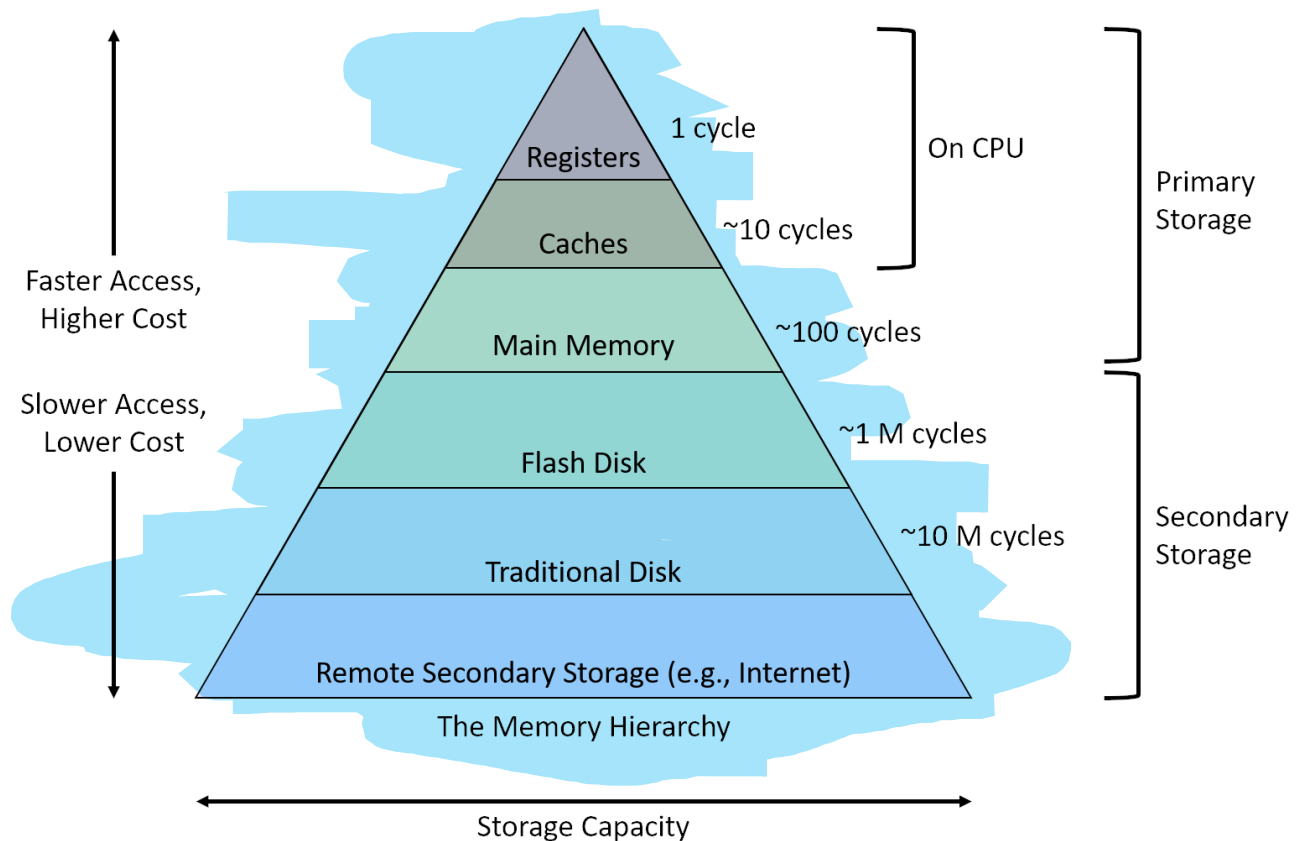
A la hora de diseñar la memoria de un sistema de computo, se debe realizar un compromiso entre **capacidad**, **velocidad** y **costo**. Una memoria ideal sería aquella infinitamente grande, con tiempo de acceso nulo, y costo insignificante.

Históricamente, mientras la velocidad del procesador crecía, según la ley de Moore, la velocidad de la memoria lo hacía también, pero más lentamente. Esto acrecentó la brecha a lo largo del tiempo.

Para compensar este desbalance sin impactar fuertemente en el costo del subsistema, se implementa la memoria como una **jerarquía de memorias organizada en niveles** compuesta por varios tipos de dispositivos.

El sistema de memoria formado por una memoria principal única que se propuso en los primeros modelos de computador quedó descartado rápidamente por sus bajas prestaciones.

Jerarquía de memoria



La memoria correspondiente a cada nivel, cuanto más cercano al procesador, será mas pequeña, más rápida y más cara.

Los principales objetivos de una jerarquía de memoria son:

- **Maximizar tamaño:** idealmente disponer de una “capacidad ilimitada”, equiparada al tamaño del nivel más grande.
- **Optimizar velocidad:** simular que se dispone de un banco de memoria “ultrarápida”, próximo a la velocidad del nivel más rápido
- **Minimizar el costo total:** implementar una memoria a un costo cercano al del nivel más lento.

Hay **dos propiedades** que la jerarquía debe cumplir para que su funcionamiento sea el adecuado:

- **Inclusión:** los datos almacenados en un nivel han de estar también almacenados en los niveles inferiores a él.
- **Coherencia:** las copias de la misma información en los distintos niveles deben almacenar los mismos valores (ser coherentes entre sí).

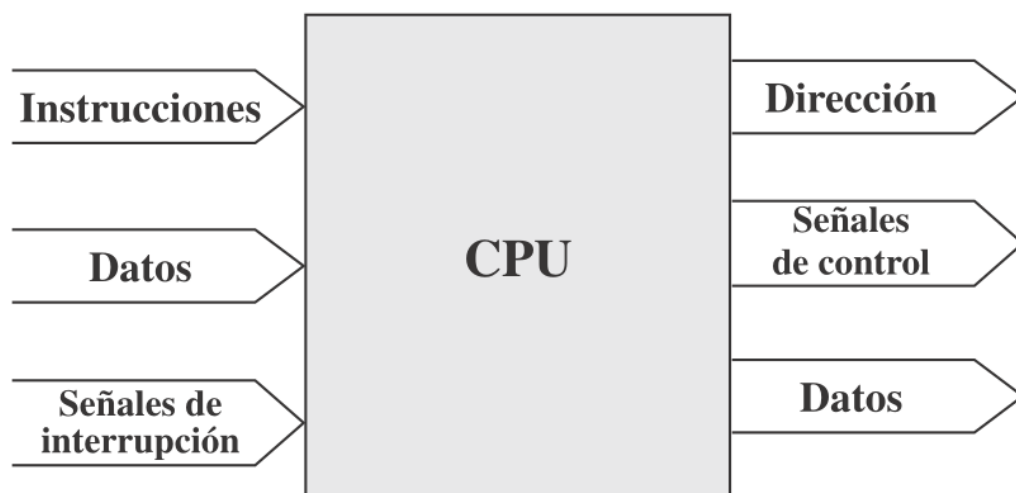
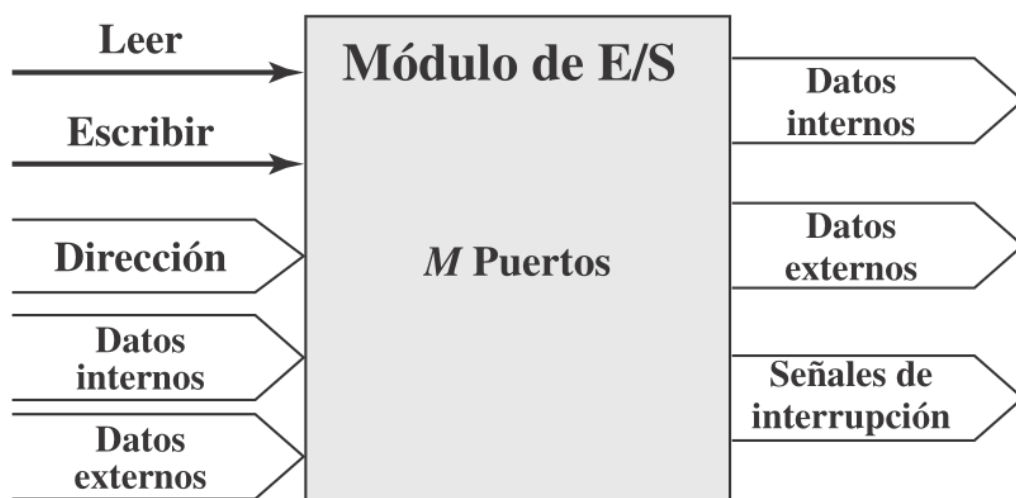
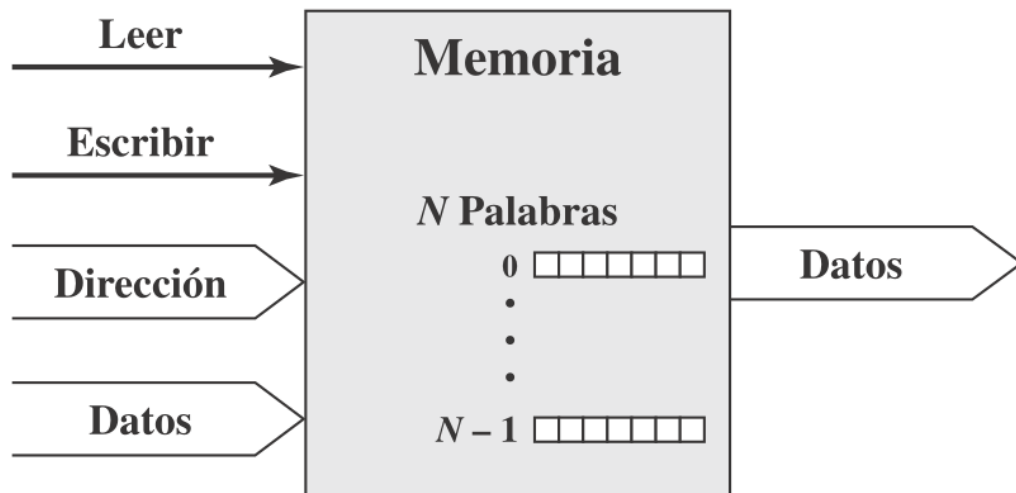
Además, en cualquier jerarquía de memoria debe haber una [correspondencia](#) de direcciones entre los distintos niveles de la jerarquía.

Bus

¿Qué es un bus?

Interconexiones entre módulos

Todas las unidades han de estar interconectadas. Se deben dar distintos tipos de interconexiones para los módulos:



Un **bus** es un camino o medio de comunicación entre dos o más dispositivos (medio de transmisión compartido).

Usualmente, un bus está constituido por varios caminos de comunicación, o líneas. Cada línea transmite una señal binaria. En un intervalo de tiempo, se puede transmitir una secuencia de dígitos

binarios a través de una única línea.

Se utilizan varias líneas del bus para transmitir dígitos binarios simultáneamente (en paralelo). Por ejemplo, un dato de 8 bits puede transmitirse mediante ocho líneas del bus, un bit por línea en simultáneo. Referimos a la cantidad de líneas de un bus como el **ancho del bus**.

Físicamente, el bus es un conjunto de conductores eléctricos paralelos. Cada línea es una línea de metal grabada en una tarjeta (tarjeta de circuito impreso).

¿Cómo funciona?

Cualquier señal transmitida por uno de los dispositivos conectados al bus estará disponible a el resto de dispositivos que estén conectados a él. Si dos dispositivos transmiten durante el mismo periodo de tiempo, sus señales pueden solaparse y distorsionarse.

Una computadora poseen diferentes tipos de buses que proporcionan comunicación entre sus componentes a distintos niveles dentro de la jerarquía del sistema. El bus que conecta los componentes principales del computador se denomina **bus del sistema**.

Bus del sistema - Bus único

Bus de datos: transmite datos. (a este nivel no hay diferencia entre datos e instrucciones). Es un factor clave a la hora de determinar las prestaciones. Por ejemplo, si el bus de datos tiene una anchura de ocho bits, y las instrucciones son de 16 bits, entonces el procesador debe acceder al módulo de memoria dos veces por cada ciclo de instrucción.

Bus de dirección: Identifica la fuente o el destino del dato situado en el bus de datos. Por ejemplo, cuando el procesador desea leer una palabra de una determinada parte en la memoria. El ancho del bus de direcciones determina la máxima capacidad de memoria posible en el sistema. Por ej. el MSX88 tiene un bus de direcciones de 16 bits y cada posición de memoria es de 8 bits, lo que permite direccionar $2^{16} = 65536$ posiciones de memoria, dando una capacidad máxima de $2^{16} * 1 \text{ byte} \div 2^{10} = 64 \text{ KiB}$ de memoria.

Bus de control: se utilizan para controlar el acceso y el uso de las líneas de datos y de direcciones. Transmite información de señales de control y temporización. Las señales de temporización indican la validez de los datos y las direcciones. Las señales de órdenes especifican las operaciones a realizar.

Si un módulo desea enviar un dato a otro debe hacer dos cosas:

1. obtener el uso del bus
2. transferir el dato a través del bus.

Si un módulo desea pedir un dato a otro módulo, debe

1. obtener el uso del bus,
2. transferir la petición al otro módulo mediante las líneas de control y dirección apropiadas y,
3. por último, esperar a que el segundo módulo envíe el dato.

Existen **estándares** que especifican como se debe dar la interconexión entre el sistema de cómputo y las **tarjetas de expansión**. Estas se insertan en **ranuras de expansión** con el fin de expandir las capacidades del computador. Estas ranuras o **slots** están asociados a un **bus de expansión**, que comunica la información entre los periféricos y el resto del sistema.

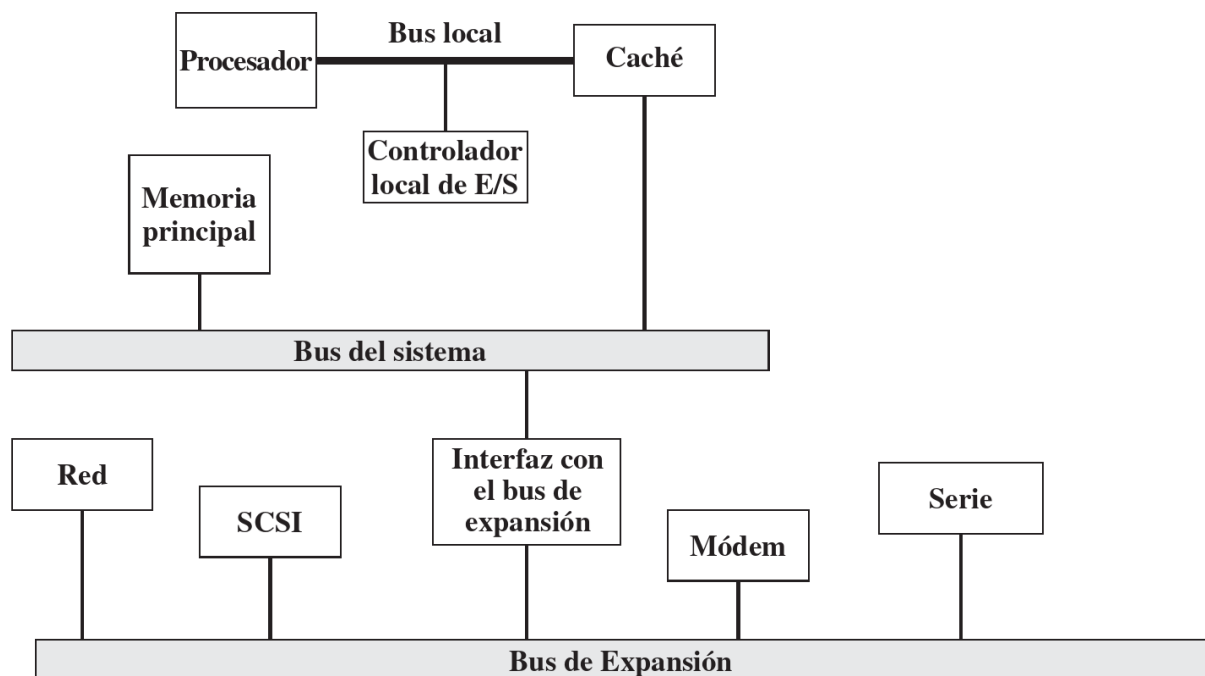
Conectar un gran número de dispositivos a un bus produce **retardos de propagación**. Si el control del bus pasa de un dispositivo a otro, puede afectar sensiblemente a las prestaciones.

El bus puede convertirse en un cuello de botella a medida que las peticiones de transferencia acumuladas se aproximan a la capacidad del bus.

La mayoría de los sistemas utilizan varios buses para solucionar estos problemas, una **jerarquía de buses**.

Jerarquía de buses

Jerarquía de buses tradicional

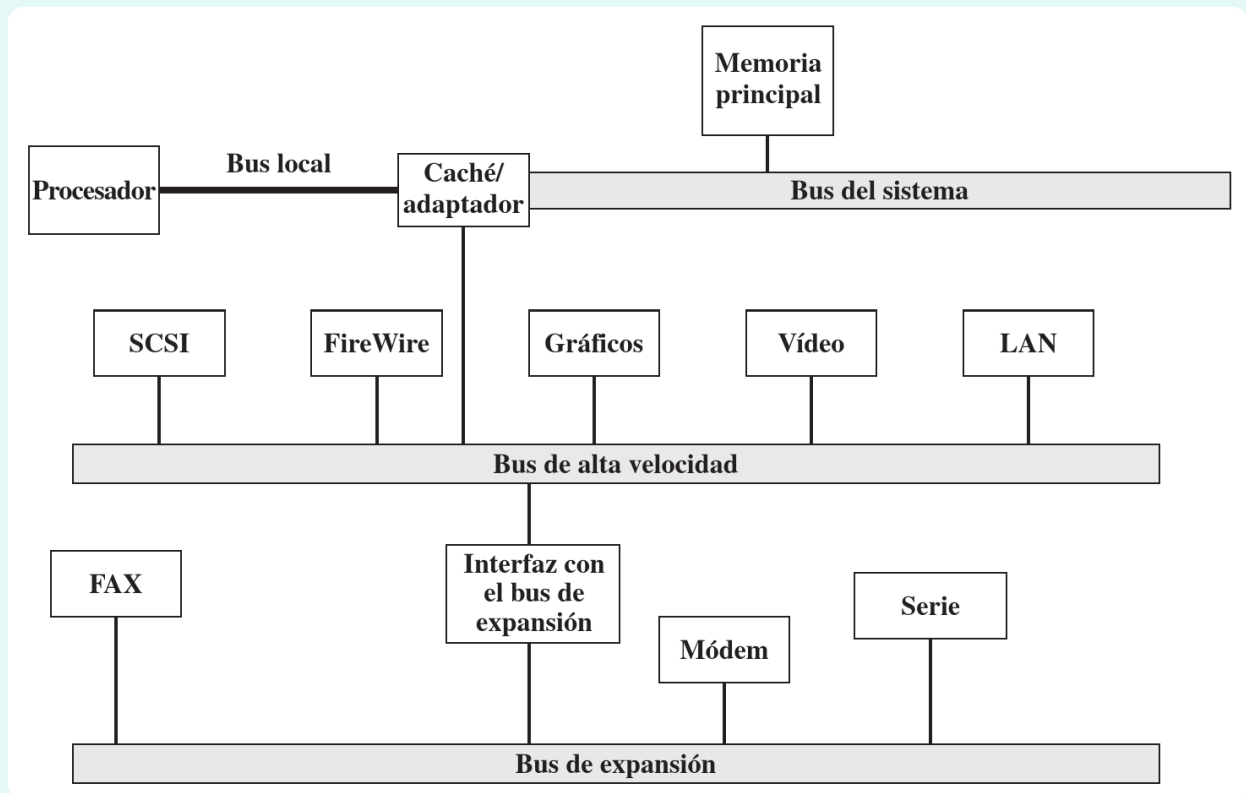


Hay un bus local que conecta el procesador a una memoria caché y al que pueden conectarse también uno o más dispositivos locales. El controlador de memoria caché conecta la caché no solo al bus local sino también al bus de sistema, donde se conectan todos los módulos de memoria principal.

La interfaz del bus de expansión regula las transferencias de datos entre el bus de sistema y los controladores conectados al bus de expansión. Esta disposición permite conectar al sistema una variedad de dispositivos de E/S y al mismo tiempo aislar el tráfico de información entre la memoria y el procesador del tráfico correspondiente a las E/S.

Jerarquía de buses de altas prestaciones

La arquitectura de buses tradicional es razonablemente eficiente, pero muestra su debilidad a medida que los dispositivos de E/S ofrecen prestaciones cada vez mayores. La respuesta por parte de la industria ha sido proponer un bus de alta velocidad que está estrechamente integrado con el resto del sistema, y requiere solo un adaptador (bridge) entre el bus del procesador y el bus de alta velocidad.



Nuevamente hay un bus local que conecta el procesador a un controlador de caché, que a su vez está conectado al bus de sistema que soporta a la memoria principal. El controlador de caché está integrado junto con el adaptador, o dispositivo de acoplo, que permite la conexión al bus de alta velocidad. Este bus permite la conexión de LAN de alta velocidad controladores de trabajo específicos para aplicaciones gráficas y de vídeo, y también controladores de interfaz para buses de periféricos tales como SCSI y Firewire. Los dispositivos de velocidad menor pueden conectarse al bus de expansión, que utiliza una interfaz para adaptar el tráfico entre el bus de expansión y el bus de alta velocidad.

La ventaja de esta organización es que el bus de alta velocidad acerca al procesador los dispositivos que exigen prestaciones elevadas y al mismo tiempo es independiente del procesador. Así, se pueden tolerar las diferencias de velocidad entre el procesador y el bus de altas prestaciones y las variaciones en la definición de las líneas de los buses. Los cambios en la arquitectura del procesador no afectan al bus de alta velocidad, y viceversa.

La interfaz entre los buses es también conocida como **chipset**

Diseño de un bus

Los parámetros a considerar a la hora de diseñar un bus son:

- **Tipo de bus**

- **Método de arbitraje**
- **Temporización**
- **Anchura del bus**
- **Tipo de transferencia de datos**

Tipos de bus

Tipos de buses:

Un **bus dedicado** esta asignado a una única función (ej. bus de direcciones) o a un único subconjunto físico de componentes del computador (ej. bus que conecta módulos de E/S).

Un **bus multiplexado** reutiliza las mismas líneas del bus diferentes propósitos. Requiere de líneas que a través de señales de control identifiquen que tipo de información se está transmitiendo.

La ventaja del multiplexado es el uso de menos líneas, cosa que ahorra espacio y costo. La desventaja es que se necesita una circuitería más compleja en cada módulo.

Ejemplo - tipos de bus:

Dedicados:

- Uso de líneas separadas para direcciones y para datos.
- 16 líneas de direcciones
- 16 líneas de datos
- 1 línea de control de lectura ó escritura (r/w)

Multiplexados:

- Uso de las mismas líneas.
- 16 líneas de direcciones ó datos
- 1 línea de control de lectura ó escritura (r/w)
- 1 línea de control para definir direcciones ó datos (a/d)
- Menos líneas pero mas circuitería

Arbitraje

Más de un módulo puede necesitar el control del bus (ej. CPU y DMAC), puesto que en un instante dado solo una unidad puede transmitir a través del bus, se requiere algún método de arbitraje.

El propósito de un método de arbitraje es designar un dispositivo, el procesador o módulo de E/S como **bus master** o maestro del bus. El maestro podría entonces iniciar una transferencia de datos (r/w) con otro dispositivo, que actúa como esclavo en este intercambio en concreto.

En el **arbitraje centralizado** un único dispositivo de hardware, denominado **controlador del bus o arbitro** es responsable de asignar tiempos en el bus. El dispositivo puede estar en un módulo separado o ser parte del procesador.

En un **arbitraje distribuido** no existe un controlador central, en este cada módulo dispone de lógica para controlar el acceso al bus. Los módulos conectados al bus actúan en conjunto para compartir el bus.

Temporización

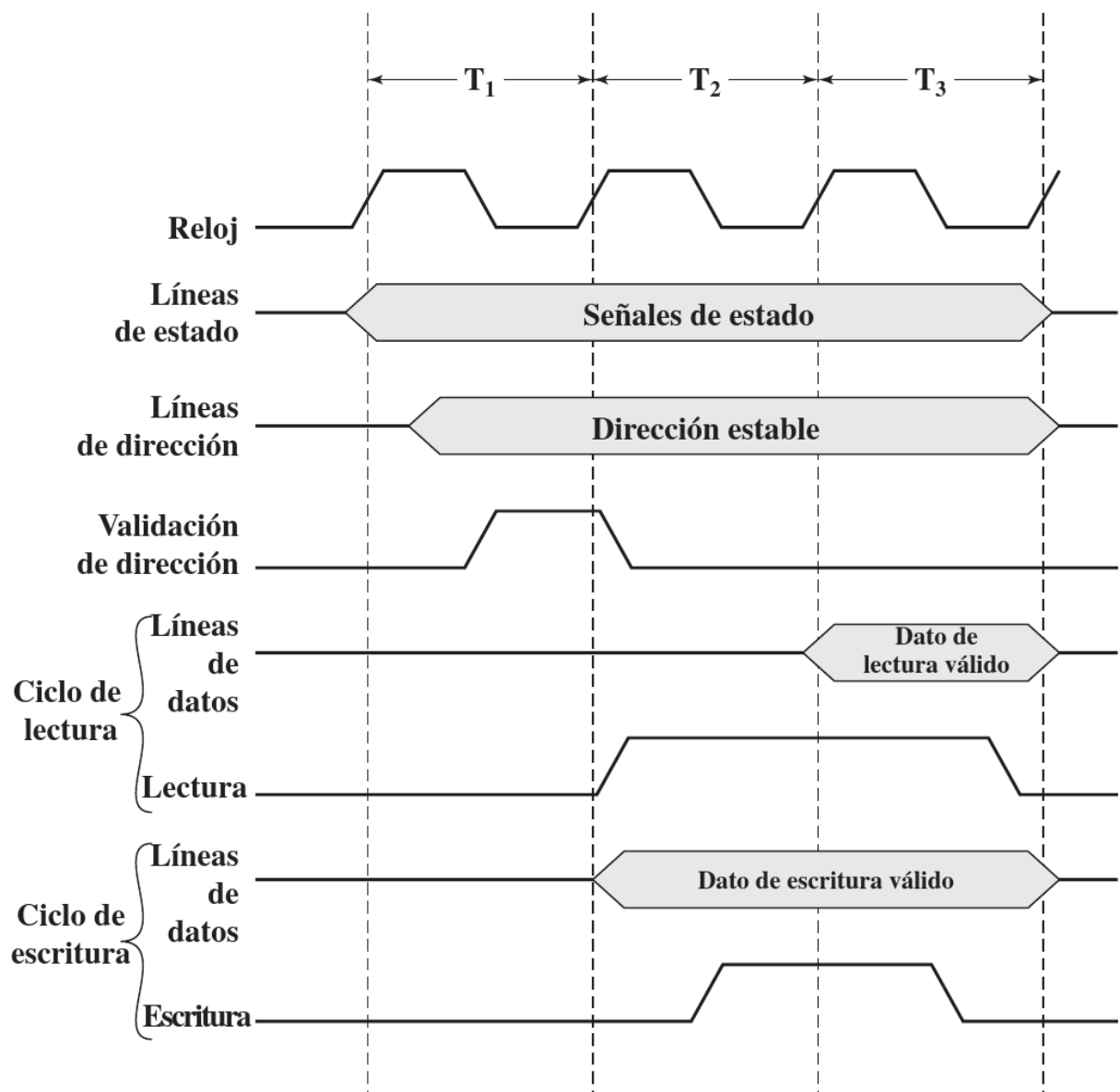
El término temporización hace referencia a la forma en que se coordinan los eventos en el bus.

Con la **temporización síncrona**, la presencia de un evento en el bus está determinada por un reloj. El bus incluye una línea de reloj a través de la que se transmite una secuencia en la que se alternan intervalos regulares de igual duración. Un único intervalo a uno seguido de otro a cero se conoce como **ciclo de bus**. Todos los dispositivos del bus pueden leer la línea de reloj, y todos los eventos empiezan al principio del ciclo de reloj. Es usual que la sincronización ocurra en el flanco de subida. La mayoría de los eventos se prolongan durante un único ciclo de reloj.

Con la **temporización asíncrona**, la presencia de un evento en el bus es consecuencia y depende de que se produzca un evento previo. No hay una señal de reloj. La comunicación entre dispositivos se da mediante una conversación (*handshake*), en base a señales entre el master y slave.

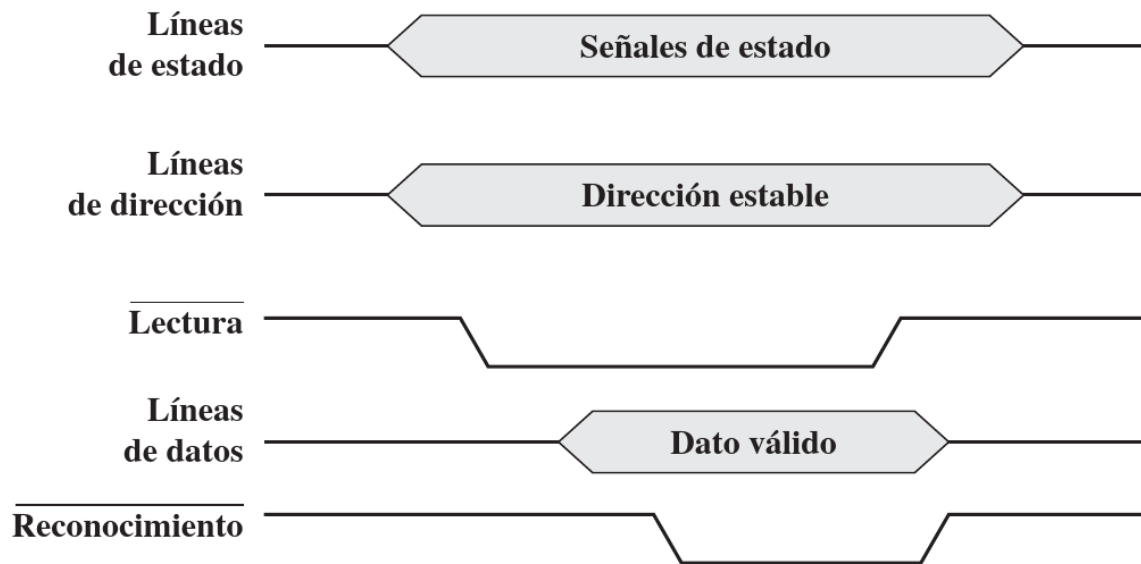
La temporización síncrona es más fácil de implementar y comprobar, pero es menos flexible que la temporización asíncrona. Debido a que todos los dispositivos en un bus síncrono deben utilizar la misma frecuencia de reloj, el sistema no puede aprovechar las mejoras en las prestaciones de los dispositivos. Con la temporización asíncrona, pueden compartir el bus una mezcla de dispositivos lentos y rápidos.

Ejemplo - temporización síncrona de las operaciones del bus



En este ejemplo la CPU activa una señal de lectura y sitúa una dirección de memoria en las líneas de dirección durante el primer ciclo y puede activar varias líneas de estado. Una vez que las líneas se han estabilizado, el procesador activa una señal de inicio para indicar la presencia de la dirección en el bus. En el caso de una lectura, el procesador pone una orden de lectura al comienzo del segundo ciclo. El módulo de memoria reconoce la dirección y, después de un retardo de un ciclo, sitúa el dato en las líneas de datos. El procesador lee el dato de dichas líneas y quita la señal de lectura. En el caso de una escritura, el procesador pone el dato en las líneas de datos al comienzo del segundo ciclo, y pone la orden de escritura una vez estabilizadas las líneas de datos. El módulo de memoria copia la información de las líneas de datos durante el tercer ciclo de reloj.

Ejemplo - temporización asíncrona de las operaciones del bus



En este ejemplo se observa un ciclo de lectura del bus del sistema. El procesador sitúa las señales de dirección y lectura en el bus. Después de un breve intervalo para que las señales se estabilicen, activa la orden de lectura, indicando la presencia de señales de dirección y control válidas. El módulo de memoria correspondiente decodifica la dirección y responde proporcionando el dato en la línea de datos. Una vez estabilizadas las líneas de datos, el módulo de memoria activa la línea de reconocimiento para indicar al procesador que el dato está disponible. Cuando el maestro ha leído el dato de las líneas correspondientes, deshabilita la señal de lectura. Esto hace que el módulo de memoria libere las líneas de datos y reconocimiento. Por último, una vez se ha desactivado la línea de reconocimiento, el procesador quita la información de dirección de las líneas correspondientes.

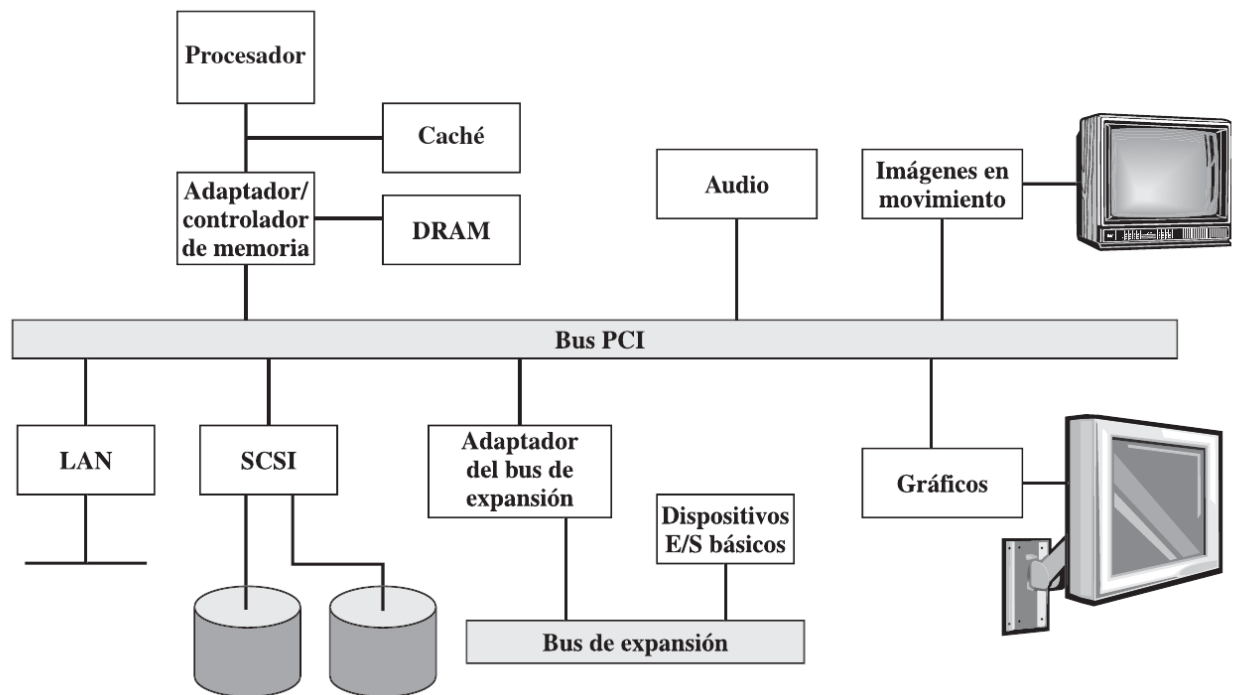
PCI

Peripheral Component Interconnect o **PCI**, es un bus estándar para conectar dispositivos periféricos. Fue desarrollado por Intel y cedió sus patentes al dominio público.

El PCI ha sido diseñado específicamente para ajustarse, económicamente a los requisitos de E/S de los sistemas de la época, se implementa con muy pocos circuitos integrados y permite que otros buses se conecten al bus PCI.

Comandos:

- Transacción maestro-esclavo
- Maestro toma control del bus.
- Determina tipo de transacción (lectura o escritura)
- Fase de direccionamiento.
- Una o más fases de datos.



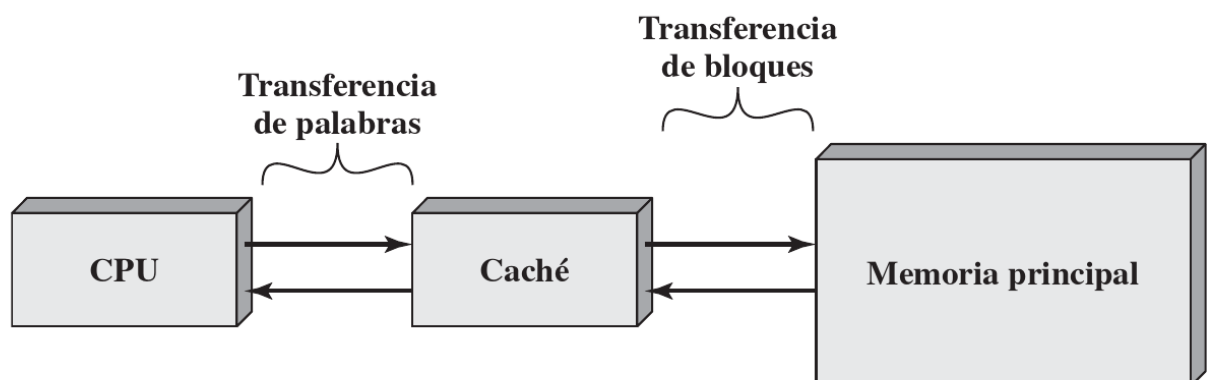
Memoria caché

¿Qué es?

El objetivo de la memoria caché es lograr que la velocidad de la memoria sea lo más rápida posible, consiguiendo al mismo tiempo un tamaño grande al precio de memorias semiconductoras menos costosas.

La memoria caché es una memoria pequeña y muy rápida, que se ubica jerárquicamente entre la memoria principal y la CPU. Físicamente puede localizarse en un chip separado, dentro de la CPU, o en ambos lugares.

La caché contendrá algunos sectores denominados **bloques** o ranuras copiados de la memoria principal. Los bloques son de longitud fija, y están constituidos por un cierto número de palabras.



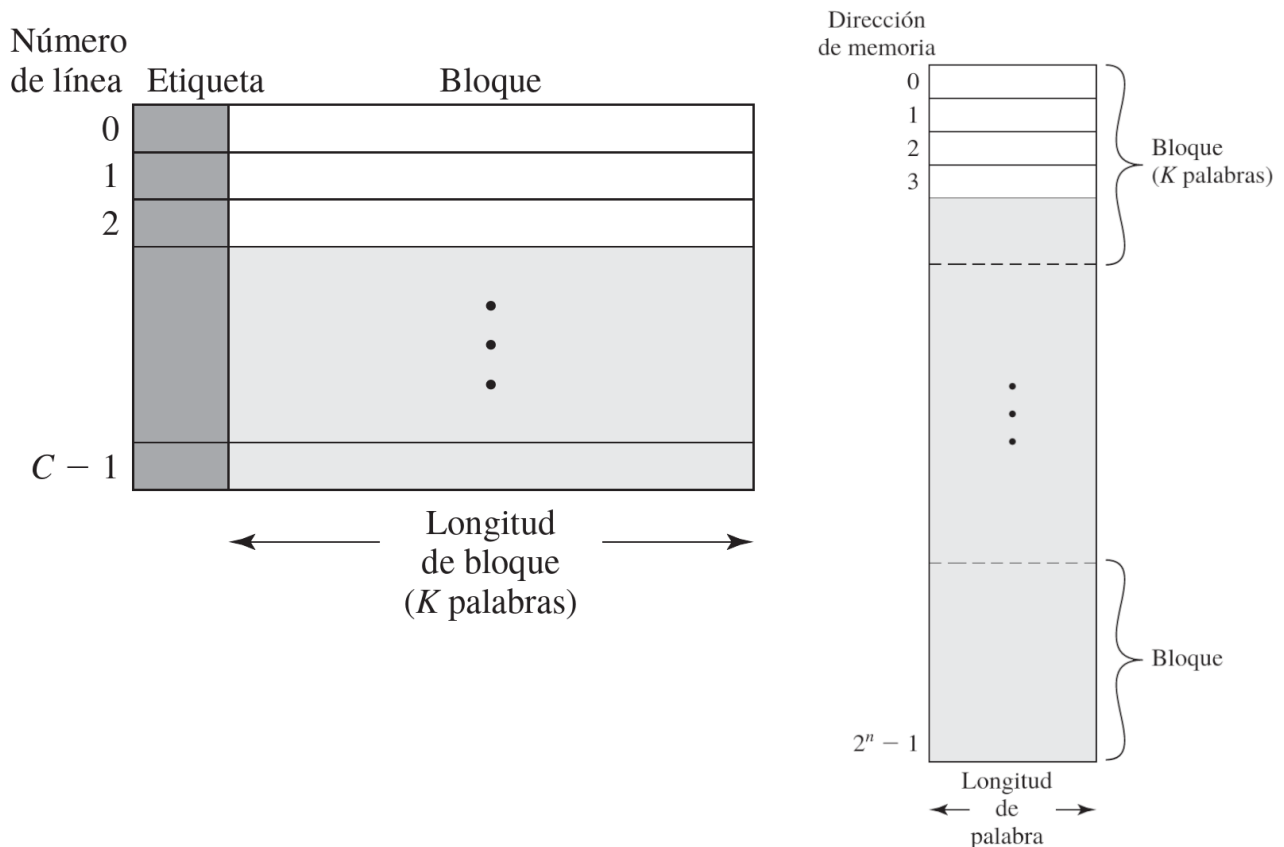
El bloque es la unidad de intercambio de información entre la memoria principal y la caché, mientras que entre la caché y la CPU sigue siendo la palabra.

La cantidad de bloques que podrán ser copiados depende del tamaño de la memoria caché y del tamaño de bloque.

Cada bloque en la caché tiene asociada una **etiqueta**, que identifica a que bloque de memoria principal corresponde.

El conjunto de etiquetas forma el **directorio** de la cache. Este contendrá la información que permite identificar la ubicación de los bloques de memoria principal en la memoria caché.

La agrupación de una etiqueta, con las palabras del bloque al que identifica, forman una **línea** de la cache.



En todo momento, un subconjunto de los bloques de memoria reside en líneas de la caché. Si se lee una palabra de un bloque de memoria, dicho bloque es transferido a una de las líneas de la caché.

¿Cómo funciona?

Un acceso a memoria en una máquina con caché consiste en:

1. El procesador genera la dirección de la palabra que se debe leer o escribir.
2. Se accede a la caché y se determina si la palabra está en ella:
 - Si está (**acierto**, *hit*), se entrega dicha palabra al procesador y finaliza el acceso a memoria.
 - Si no está (**fallo**, *miss*), se busca en cada siguiente nivel de la jerarquía (de caché si los hubiera, y si no de memoria principal) y se copia a la cache el bloque correspondiente. Luego la palabra es entregada al procesador.

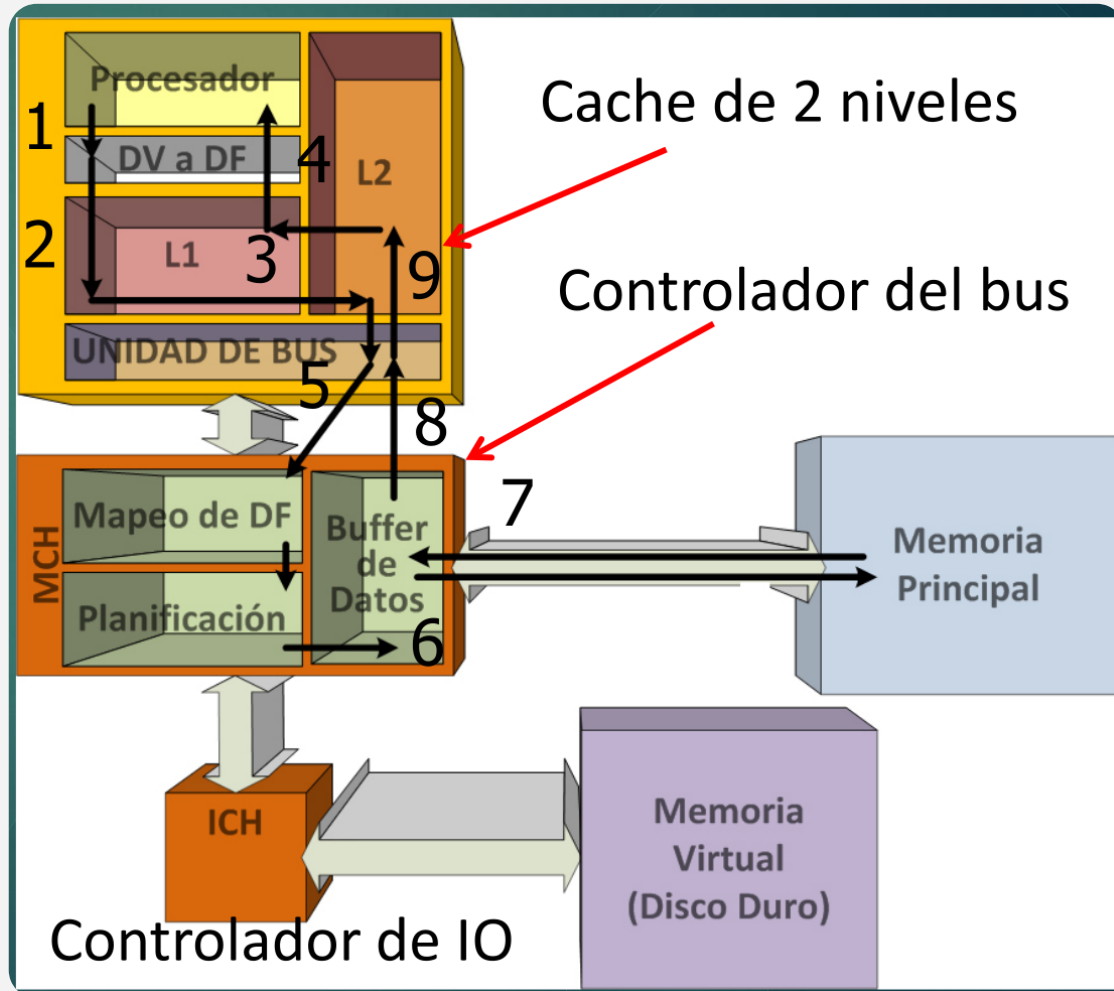
Cuando se produce un acierto de cache, la velocidad de acceso al dato es la de la memoria cache misma (relativamente muy corto).

Los fallos de cache se gestionan mediante hardware y causan que el procesador se detenga hasta que el dato esté disponible, cuestan tiempo. El tiempo para servir un fallo depende de la **latencia** (tiempo necesario para completar un acceso a memoria) y **ancho de banda** (cantidad de información por

unidad de tiempo que puede transferirse) de la memoria principal. Así la eficiencia de uso de la cache dependerá de la tasa de aciertos (cantidad de veces que acierta en promedio).

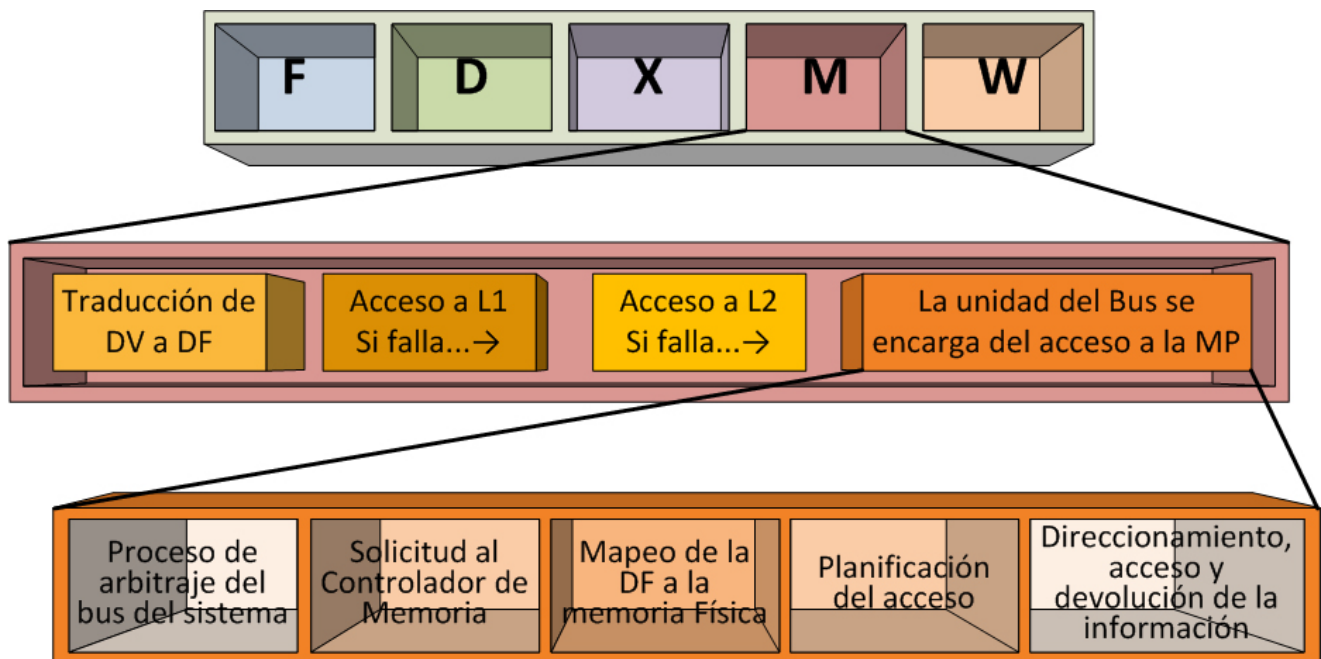
La tasa de aciertos no es proporcional al tamaño de la caché, que es miles de veces más chico que el de la memoria principal. La razón de esto tiene que ver con el comportamiento de los programas.

Acierto/Fallo de caché



- El camino 1-2-3-4 corresponde a un acierto.
- El camino 1-2-5-6-7-8-9-3-4 corresponde a un fallo.

En un procesador con **segmentación del cauce**, se dispone de un ciclo de reloj para el acceso a memoria. Ese tiempo debe ser suficiente para el acceso a la memoria caché. En caso de fallo el acceso a la memoria principal requiere de varios ciclos extra.



Principios de localidad referencial

Los programas manifiestan una propiedad que se explota en el diseño de memoria de los computadores en general y de la memoria caché en particular, la **localidad de referencias**.

Los programas tienen un comportamiento basado en **dos principios** de carácter empírico:

- **Principio de localidad temporal de las referencias**: tendencia del procesador a referenciar elementos de memoria (datos e instrucciones) que han sido referenciados recientemente. *Los programas tienden a reutilizar los datos e instrucciones que utilizaron recientemente.*
- **Principio de localidad espacial de las referencias**: tendencia del procesador a referenciar elementos de memoria (datos e instrucciones) cercanos a los últimos que han sido referenciados. *La mayoría de los programas gastan el 90% de su tiempo de ejecución sobre sólo el 10% de su código.*

¿Cómo se relacionan con el funcionamiento de la jerarquía de memoria?

El principio de localidad temporal asegura que si el procesador ha solicitado una palabra para leer o escribir en ella y se trae esta palabra a memoria caché, es muy probable que esta palabra vuelva a utilizarse recientemente. Se observa, por ejemplo, cuando se ejecutan iteraciones de un bucle.

Por el principio de localidad espacial, si en lugar de traer sólo la palabra solicitada por el procesador a la caché, se trae un bloque de palabras contiguas en la memoria, la probabilidad indica que alguna otra palabra de este bloque se utilizará poco tiempo después. Aumentando la probabilidad de que se produzcan aciertos. Refleja la tendencia del procesador a acceder a las instrucciones secuencialmente, y también la tendencia de los programas a acceder a posiciones de datos consecutivas, como por ejemplo al operar con una matriz o tabla de datos.

En periodos de tiempo largos, las agrupaciones (clusters) en uso cambian, pero en periodos de tiempo cortos, el procesador trabaja principalmente con agrupaciones fijas de referencias a memoria.

Tradicionalmente, la localidad temporal se ha explotado manteniendo en memoria caché las instrucciones y los datos usados recientemente, y aprovechando la jerarquía de caché. Generalmente, la localidad espacial se explota usando bloques de caché más grandes e incorporando mecanismos de precaptación (captando objetos de uso anticipado) en la lógica de control de caché.

Rendimiento

Tiempo de acceso a memoria con una caché

El **tiempo de acceso promedio de la CPU** (t_{CPU}) es el promedio del tiempo que se tarda en obtener los datos buscados en memoria. Esta compuesto por accesos a la caché y accesos a la memoria principal.

$$t_{CPU} = (1 - TF)t_{accesoMC} + TF \cdot PF$$

La tasa de fallos: $TF = \text{num de fallos} / \text{num total de accesos}$

PF es la tasa de penalización por fallo, el tiempo perdido en acceder a la memoria principal, ($t_{accesoMC}$).

Para mejorar las prestaciones se debe reducir el $t_{accesoMC}$, TF y PF.

En un caso *ideal*, donde no hay fallos de cache, el tiempo que se tarda en ejecutar un programa es igual a la velocidad de la CPU. El tiempo de acceso a la caché es despreciable.

Diseño de la caché

A la hora de diseñar una caché, debe tenerse en cuenta:

- Su organización
- Política de asignación (función de correspondencia)
- Política de reemplazo
- Política de escritura

Organización

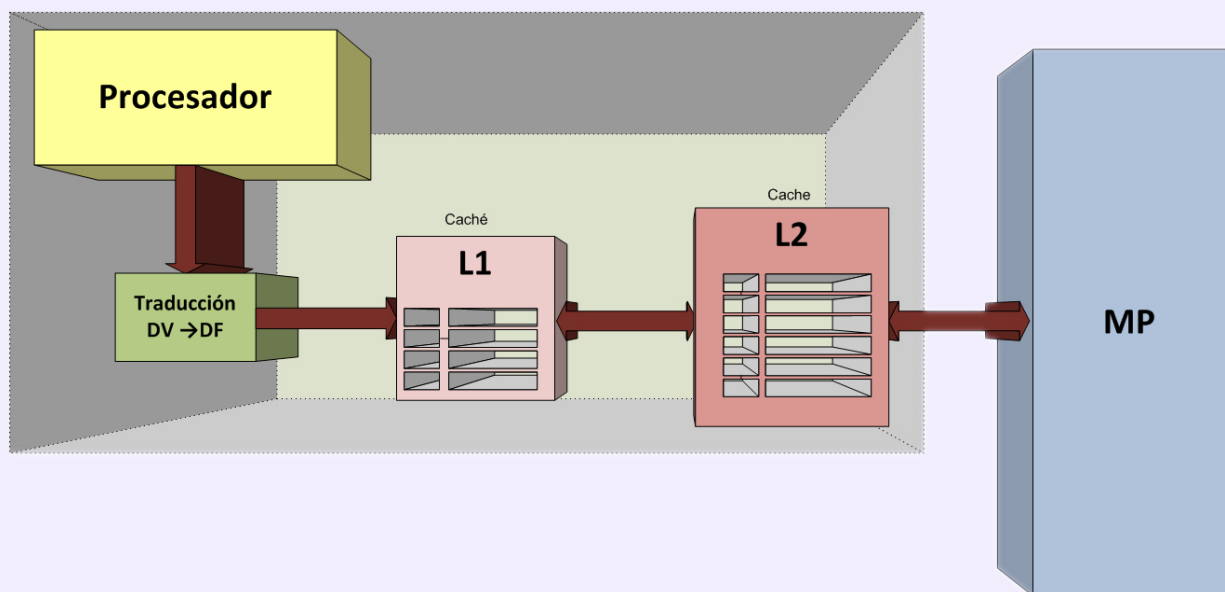
Durante el diseño de una caché, respecto a su organización, hay que tener en cuenta varias consideraciones:

- **Tamaño de la memoria caché:** Debe ser suficientemente grande para contener la mayor cantidad posible de información. Pero no demasiado grande porque impacta en la velocidad ($t_{accesoMC}$).
- **Tamaño del bloque o ranura:** debe ser suficientemente grande para aprovechar al máximo las referencias cercanas. Al aumentar el tamaño mejora la tasa de aciertos, hasta un cierto tamaño de bloque. A medida que lo excede, el aumento comienza a resultar contraproducente, ya que caben menos bloques de memoria principal en la cache, y esto tiende a aumentar la tasa de fallos. Existe un valor óptimo para el tamaño del bloque.
- **Costo:** El costo crece fuertemente a medida que crece el tamaño de la memoria. Otro factor que incrementa el costo significativamente es cuando la caché está incluida en el chip que contiene al

procesador.

- **Niveles:** La memoria caché puede ser una sola, tener solo un nivel, o estar dividida en múltiples niveles. Los múltiples niveles, usualmente, tienen distintos tamaños. La implementación de **cachés multinivel** es uno de los aspectos que más influye en el rendimiento de la jerarquía de memoria.
- **Separación de la caché de instrucciones y operandos:** El mecanismo de acceso a las instrucciones es distinto al del acceso de los datos. Es por ello que las estrategias para obtener una alta tasa de aciertos es distinta en un caso y en el otro. Es posible mejorar la tasa de aciertos general si la caché se divide en una caché de instrucciones y una de datos, con distintas características. Por ejemplo, teniendo ambas caches distintos tamaños, políticas de acceso y de reemplazo.

Esquema de caché con dos niveles



Política de asignación

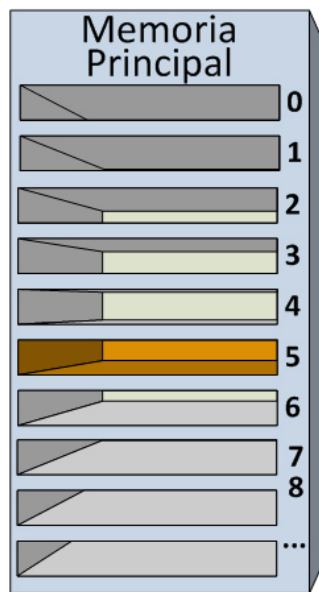
La **política de asignación** (*mapping* o **correspondencia**) define la forma en que se van a asignar los bloques de la memoria principal a una determinada línea de la memoria caché.

Las políticas más empleadas son:

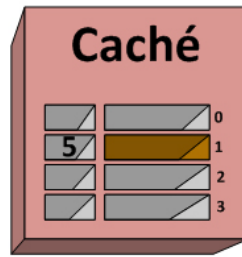
- **Correspondencia directa:** a cada bloque de memoria principal le corresponde una única línea posible en la caché. Una operación determina la línea de caché que le corresponde a un determinado bloque de memoria principal.
- **Correspondencia asociativa:** un bloque puede almacenarse en cualquier lugar o línea de la caché.
- **Correspondencia asociativa por conjuntos:** un bloque puede almacenarse en un conjunto restringido de lugares en la caché. La caché se divide en conjuntos formados por un igual número de líneas. Una operación determina el conjunto en que el bloque debe almacenarse, pero dentro de este conjunto, puede ubicarse en cualquier posición. Es una solución intermedia, entre las dos anteriores.

Ejemplo: política de asignación

Supongamos una memoria caché de sólo cuatro líneas:

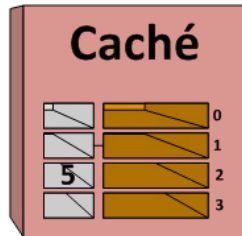


Directa

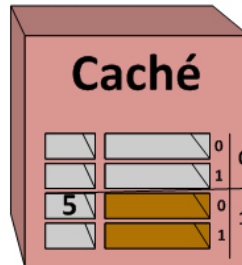


Bloque 5 mod 4 marcos = 1

Asociativa



Asociativa 2 vías



Bloque 5 mod 2 conjuntos = 1

- Directa: una operación de módulo nos indica en qué línea de la caché debe ubicarse el bloque. Es la única posibilidad de emplazamiento.
- Asociativa: el bloque puede ubicarse en cualquier línea.
- Asociativa por conjunto de dos vías: una operación de módulo nos indica en qué conjunto debe ubicarse el bloque.

Correspondencia directa

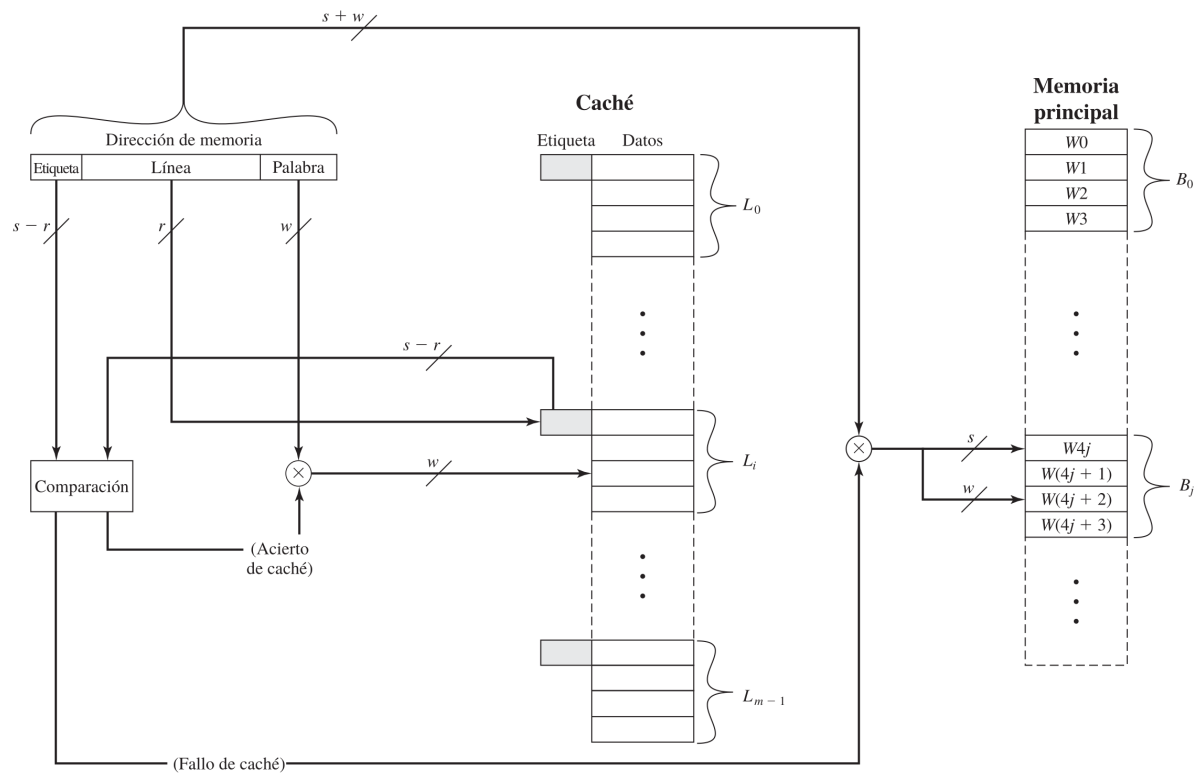
A cada bloque de memoria principal le corresponde una única línea posible en la caché. Esta política de asignación es muy simple de implementar y poco costosa.

La correspondencia entre el número de línea de cache i y el número de bloque de memoria principal j , se expresa como:

$$i = j \bmod m$$

Con m = número total de líneas en la caché

Correspondencia directa



Los w bits menos significativos identifican cada palabra dentro de un bloque de memoria principal. Los s bits restantes especifican uno de los 2^s bloques de la memoria principal. La lógica de la caché interpreta estos s bits como una etiqueta de $s - r$ bits (parte más significativa) y un campo de línea de r bits. Este campo identifica una de las $m = 2^r$ líneas de la caché.

- Longitud de direcciones = $s + w$ bits.
- Número de unidades direccionables = 2^{s+w} palabras o bytes.
- Tamaño de bloque = tamaño de línea = 2^w palabra o bytes.
- Número de bloques en memoria principal = $\frac{2^{s+w}}{2^w} = 2^s$.
- Número de líneas en caché = $m = 2^r$.
- Tamaño de la etiqueta = $s - r$ bits.

La dirección física es interpretada en la cache como formada por tres campos:

- **tag/etiqueta:** número de grupo en la memoria principal.
- **line/línea:** número de bloque dentro del grupo.
- **word/palabra:** número de palabra dentro del bloque.

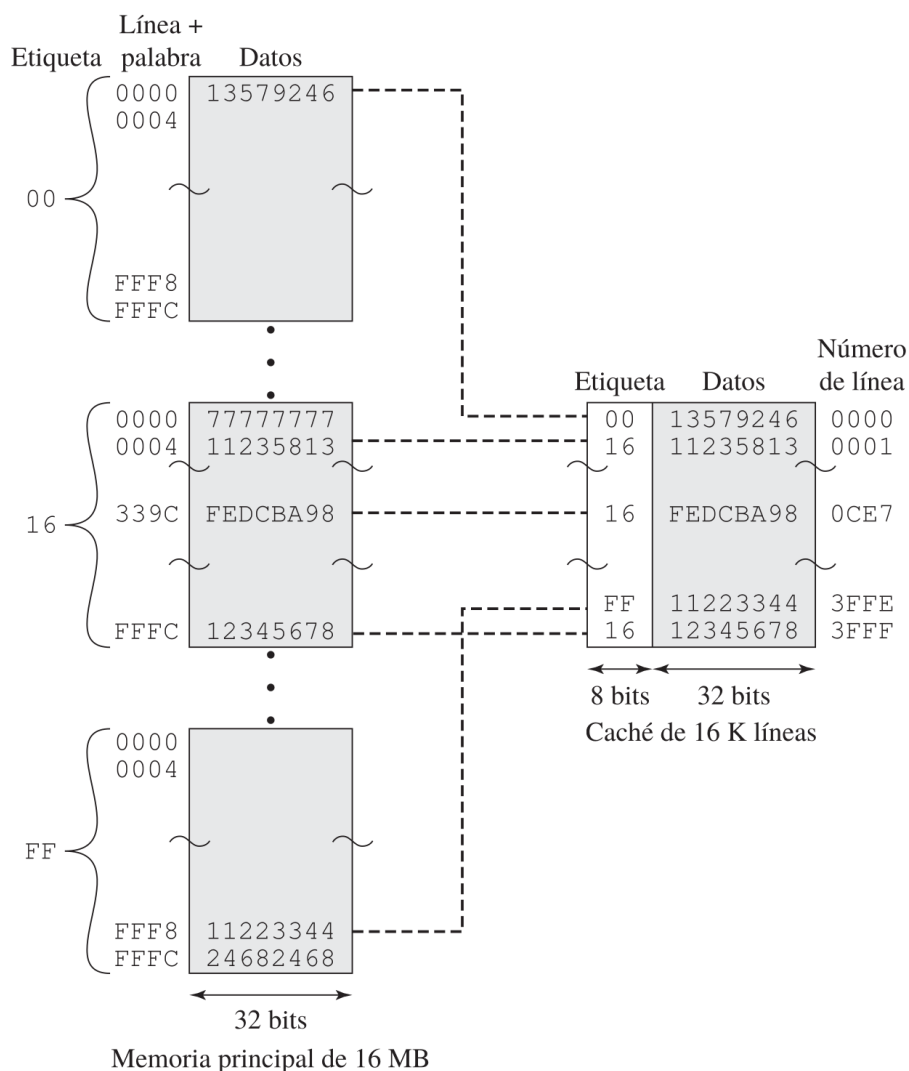
Hay tantas líneas en la caché como bloques por grupo en la memoria principal. La etiqueta de la dirección se compara con la etiqueta de la caché correspondiente. Si la comparación da un acierto, el dato se busca en la caché. Si falla, el dato se trae de memoria principal.

Tiene un rendimiento aceptable, aunque a veces puede ser malo. Su principal desventaja es que hay una posición concreta de caché para cada bloque dado. Por ello, si un programa referencia repetidas veces a palabras de dos bloques diferentes asignados en la misma línea, dichos bloques se estarían intercambiando continuamente en la caché, y la tasa de aciertos sería baja. Esto se denomina **trashing**.

Por ejemplo, si un programa accede a dos bloques que se corresponden a la misma línea (diferentes bloques de memoria principal) de forma repetida, las pérdidas de cache (desaciertos) serán muy grandes.

Ejemplo: correspondencia directa

Memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 8 más significativos son el número de grupo, los siguientes 14 el bloque dentro del grupo y los 2 restantes la palabra dentro del bloque.

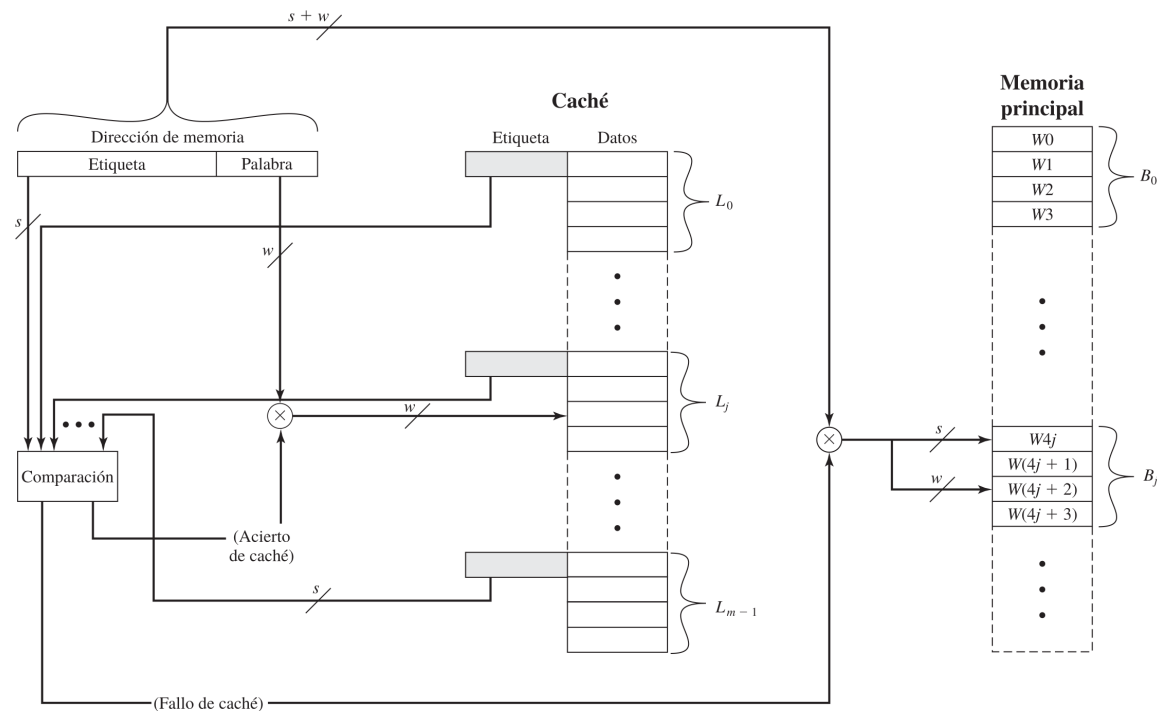


	Etiqueta	Línea	Palabra
Dirección de memoria principal =	8	14	2

Correspondencia asociativa

Supera la desventaja de la directa permitiendo que cada bloque de memoria principal pueda cargarse en cualquier línea de la caché.

Correspondencia asociativa



La lógica de control de la caché interpreta una dirección de memoria como una etiqueta y un campo de palabra. El campo de etiqueta identifica unívocamente un bloque de memoria principal. Para determinar si un bloque está en la caché, su lógica de control debe examinar simultáneamente todas las etiquetas de líneas para buscar una coincidencia

- Longitud de direcciones = $s + w$ bits.
- Número de unidades direccionables = 2^{s+w} palabras o bytes.
- Tamaño de bloque = tamaño de línea = 2^w palabra o bytes.
- Número de bloques en memoria principal = $\frac{2^{s+w}}{2^w} = 2^s$.
- Número de líneas en caché = indeterminado.
- Tamaño de la etiqueta = s bits.

La dirección de memoria caché está compuesta por dos campos:

- **tag/etiqueta:** número de bloque en la memoria principal
- **word/palabra:** número de palabra dentro del bloque

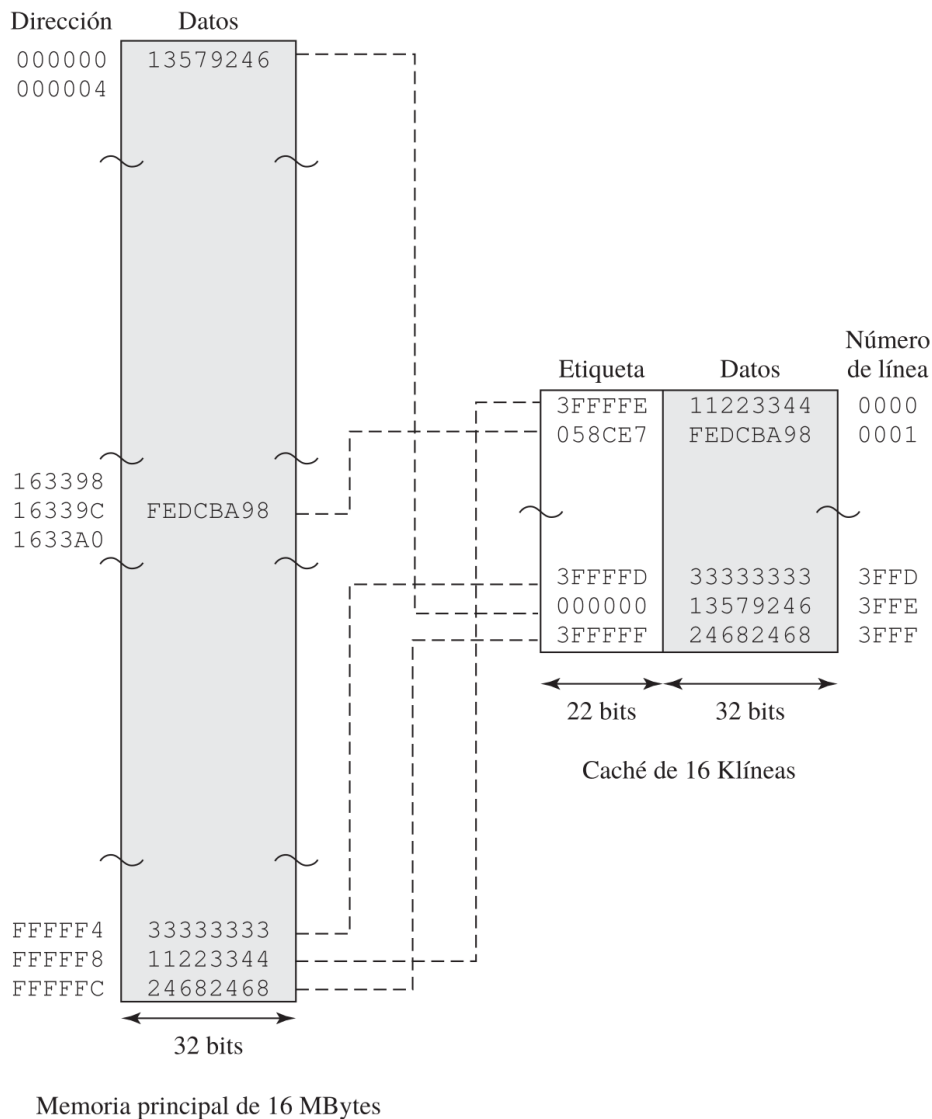
La etiqueta de la dirección que solicita la CPU se compara de manera simultánea con todas las etiquetas de la caché, que identifican el bloque de memoria principal que contiene la línea. Si la comparación da un acierto el dato se recupera de la caché. Si falla, el dato se trae del nivel inferior.

El directorio se implementa utilizando una memoria del tipo asociativa. Su implementación es costosa, requiere circuitería compleja para examinar en paralelo las etiquetas de todas las líneas de la caché. Esta es la principal desventaja.

Esta política permite una libertad absoluta para la asignación y reemplazo de los bloques.

Ejemplo: correspondencia asociativa

Memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 22 bits más significativos se usan para identificar el bloque, y 2 para la palabra dentro del bloque.



	Etiqueta	Palabra
Dirección de memoria principal =	22	2

Correspondencia asociativa por conjuntos

Toma lo positivo de las correspondencias directa y asociativa, sin presentar sus desventajas.

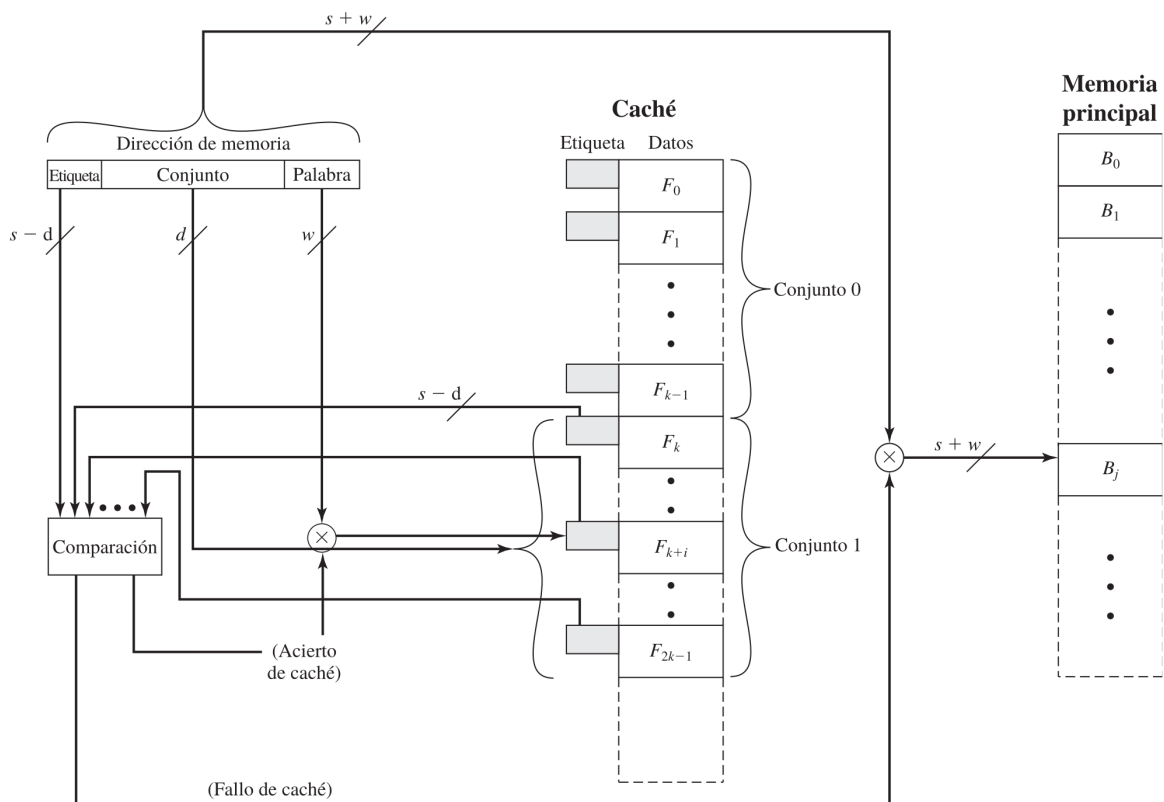
La caché se divide en v conjuntos, cada uno de k líneas. Hay m líneas en total en la caché de manera que:

$$m = v \times k$$

El número de conjunto de caché i al que se corresponde un bloque de memoria principal j , se calcula como:

$$i = j \bmod v$$

Correspondencia asociativa por conjuntos de k vías



El bloque B puede asignarse en cualquiera de las k líneas del conjunto i . En este caso, la lógica de control de la caché interpreta una dirección de memoria como tres campos: etiqueta, conjunto y palabra. Los d bits de conjunto especifican uno de entre $v = 2^d$ conjuntos. Los s bits de los campos de etiqueta y de conjunto especifican uno de los 2^s bloques de memoria principal. La etiqueta de una dirección de memoria es más corta que en la asociativa, se compara solo con las k etiquetas dentro de un mismo conjunto.

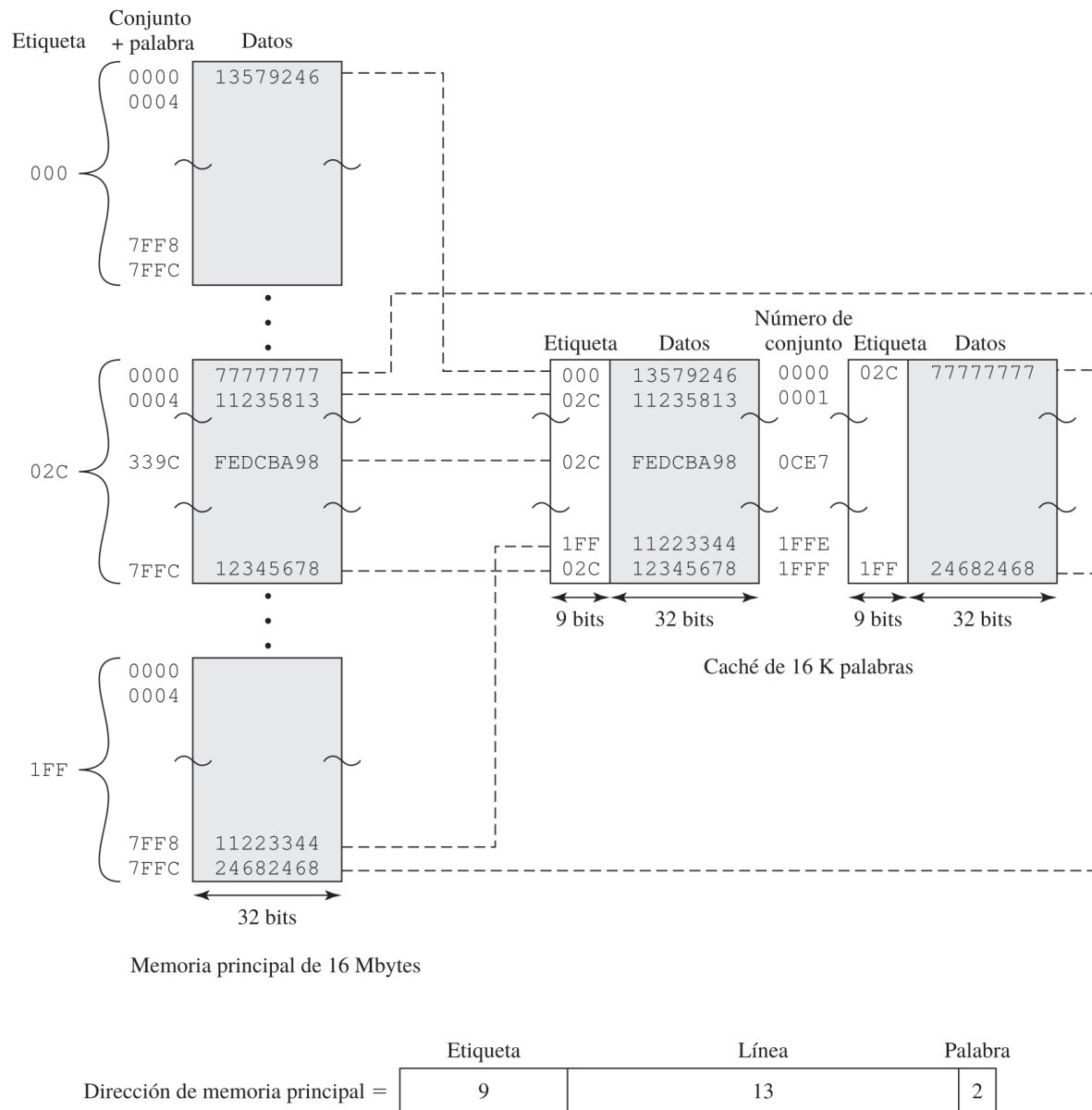
- Longitud de direcciones = $s + w$ bits.
- Número de unidades direccionables = 2^{s+w} palabras o bytes.
- Tamaño de bloque = tamaño de línea = 2^w palabra o bytes.
- Número de bloques en memoria principal = $\frac{2^{s+w}}{2^w} = 2^s$.
- Número de líneas en el conjunto = k .
- Número conjuntos $v = 2^d$.
- Número de líneas en caché $kv = k \times 2^d$.
- Tamaño de la etiqueta = $s - d$ bits.

Un bloque determinado corresponderá a alguna línea o ranura de un conjunto determinado.

En general, la función asociativa por conjuntos combina lo mejor de las otras correspondencias (asociativa y directa).

Ejemplo: correspondencia asociativa por conjuntos de dos vías

Memoria de 16MB y caché de 64KB, con bloques de 4 bytes. De los 24 bits que forman la dirección, los 9 más significativos son el número de grupo, los siguientes 13 el conjunto dentro del grupo, y los 2 restantes la palabra dentro del bloque.



Interpretación de la dirección física desde el punto de vista de la caché:

Directa	Etiqueta		
	Línea		Palabra
Asociativa	Etiqueta		Palabra
Asociativa por conjuntos	Etiqueta		
	Línea	Palabra	

Política de reemplazo

Una vez que se ha llenado la caché, para introducir un nuevo bloque debe sustituirse uno de los bloques existentes. Las diferentes estrategias que determinan qué bloque será sustituido se conocen como **políticas de reemplazo de bloque**.

Para la **correspondencia directa**, solo hay una posible línea para cada bloque y no hay elección posible. Si se requiere traer un nuevo bloque desde la memoria principal, indefectiblemente reemplazará el único bloque que le corresponde.

Para las técnicas **asociativas** se requieren algoritmos de reemplazo. Estos deben implementarse en hardware por razones de velocidad. Cuatro de los más comunes son:

1. **LRU (Least-Recently Used, menos utilizado recientemente):**

- Se sustituye el bloque que se ha mantenido en la caché por más tiempo sin haber sido referenciado.
- Cada línea incluye un bit *USO*. Cuando una línea es referenciada se pone a 1 su bit *USO* y a 0 el de la otra línea del mismo conjunto. Cuando va a transferirse un bloque al conjunto, se utiliza la línea cuyo bit *USO* es 0.
- Requiere controles de tiempos.
- Es fácil de implementar para la asociativa por conjuntos de dos vías.
- Aprovecha la localidad temporal, suponiendo que son más probables de referenciar posiciones de memoria usadas recientemente, por lo que debería dar la mejor tasa de aciertos.
- Es el más efectivo.

2. **FIFO(First-In-First-Out, primero en entrar primero en salir):**

- Se sustituye aquel bloque del conjunto que ha estado más tiempo en la caché, osea, el que antes entró en la caché.
- Puede implementarse fácilmente mediante una técnica cíclica (round-robin) o buffer circular.
- Requiere controles de acceso para identificar el orden en que ingresaron.

3. **LFU (Least-Frequently Used, utilizado menos frecuentemente):**

- Se sustituye aquella línea que ha experimentado menos referencias.
- Requiere controles de uso.
- Podría implementarse asociando un contador a cada línea.

4. **Aleatorio:** se sustituye una línea al azar.

- Se utiliza un generador de números aleatorios para escoger el bloque que se reemplaza.
- Proporciona prestaciones ligeramente inferiores a un algoritmo basado en la utilización.

Política de escritura

Las lecturas son mucho más frecuentes que las escrituras en la memoria caché, pero no hay que descuidar estas últimas ya que suelen ser más costosas.

Las escrituras llevan más tiempo ya que no pueden solaparse con la comparación de las etiquetas como se hace con las lecturas.

La **política de escritura** es el mecanismo utilizado para mantener la coherencia entre la memoria caché y la memoria principal cuando se realizan modificaciones (escrituras).

Cuando la CPU tiene que escribir un resultado en la memoria puede hacerlo:

- **tanto en un acierto** (la dirección donde se va a guardar el dato está en un bloque de la cache)

- como en un fallo (la dirección donde se va a guardar el dato no está en un bloque de la cache).

En cualquiera de las 2 situaciones, se debe evitar una inconsistencia en la jerarquía de memoria (para cumplir la propiedad de **coherencia**), durante los procesos de escrituras. Es decir que, al escribirse el dato en la cache, el correspondiente bloque de la memoria principal debe ser actualizado, en algún momento.

Hay dos problemas que se deben solventar:

- En primer lugar, más de un dispositivo puede tener acceso a la memoria principal. A veces un módulo E/S puede escribir/leer directamente en/de la memoria principal. Si una palabra ha sido modificada solo en la caché, la correspondiente palabra de memoria no es válida. Lo inverso también aplica, si el dispositivo de E/S ha alterado la memoria principal, entonces la palabra de caché no es válida.
- En arquitecturas más complejas con procesadores en paralelo, donde múltiples CPU pueden tener caches locales. Si se modifica una palabra en una de las cachés, podría invalidar una palabra de otras cachés.

Cuando la dirección de memoria a donde tiene que escribir un dato está en la memoria caché, se usan las **políticas de escritura en acierto**. Hay dos estrategias:

- **Write-through** (Escritura inmediata): Se actualizan simultáneamente la posición de la caché y de la memoria principal.
 - Aumenta el tráfico con la memoria principal.
 - Pueden haber retrasos durante múltiples escrituras.
 - Garantiza la coherencia en la jerarquía de memoria y es más fácil de implementar.
- **Write-back** (Post-escritura): La información sólo se actualiza en el primer nivel de caché y se escribe en la memoria principal cuando el bloque es reemplazado.
 - Para no actualizar la memoria con cada reemplazamiento, (innecesario si en ese bloque no se ha llevado a cabo ninguna escritura), suele utilizarse un bit que indica si el bloque que está en caché ha sido modificado (sucio) y por lo tanto debe actualizarse la memoria, o no (limpio).
 - Como la memoria principal se actualiza en el reemplazo, puede contener información errónea en algún momento de tiempo.
 - Permite consumir, por norma general, mucho menos ancho de banda del siguiente nivel de la jerarquía.

Cuando la dirección de memoria a donde tiene que escribir un dato no está en la memoria caché, se usan las **políticas de escritura en fallo**. Hay dos estrategias:

- **No-write allocate** (sin asignación en escritura):
 - Cuando se produce un fallo de escritura, el bloque se modifica directamente en la memoria principal. Sólo se llevan a la caché los bloques que se solicitan para lectura, no para escritura.
 - Habitual con write-through.
- **Write allocate** (con asignación en escritura):
 - Los fallos de escritura se comportan como los fallos de lectura, se trae el bloque que ha provocado el fallo a la memoria caché y se realiza la escritura en función de la política utilizada.
 - Habitual con write-back

Paralelismo a nivel máquina

Paralelismo a nivel máquina

El **paralelismo** es la capacidad para ejecutar varias tareas en el mismo intervalo de tiempo (en paralelo).

Se denomina **grado de paralelismo** al número máximo de instrucciones que se puede ejecutar en paralelo.

Hay dos tipos de paralelismos:

- **Paralelismo a nivel de instrucciones.**
- **Paralelismo a nivel de máquina.**

Paralelismo a nivel de instrucciones

El **paralelismo a nivel de instrucciones** refiere a las posibilidades que tiene un programa para ejecutar instrucciones en paralelo.

El grado de paralelismo de un programa se mide utilizando un parámetro llamado $ILP = IPC = \text{cant instrucciones} / \text{cant ciclos}$ (recíproco al CPI). Refiere a la cantidad promedio de instrucciones que se ejecutan en paralelo en un ciclo.

El paralelismo en las instrucciones depende de la frecuencia con la que se dan dependencias en el código. Las instrucciones podrán ejecutarse en paralelo si son independientes.

Depende también la latencia de operación: el tiempo que transcurre hasta que el resultado de una instrucción está disponible para ser usado como operando de una instrucción posterior. La latencia determina cuánto retardo causará la ocurrencia de un stall.

Paralelismo a nivel máquina

El **paralelismo a nivel de máquina** es la capacidad que tiene una máquina para encontrar y ejecutar tareas en paralelo. En otras palabras, la capacidad del hardware de sacar partido del paralelismo a nivel de instrucciones.

El grado de paralelismo a nivel máquina se mide utilizando un parámetro llamado MPL. Refleja la capacidad que tiene una máquina para aprovechar el paralelismo de un programa, en función de:

- Número de instrucciones captadas por ciclo.
- Número de unidades funcionales.
- Mecanismos para localizar y ejecutar instrucciones independientes.

Relación entre ILP y MLP

Tanto el paralelismo en las instrucciones como el paralelismo de la máquina son factores importantes para mejorar las prestaciones.

Un programa puede no tener el suficiente nivel de paralelismo en sus instrucciones (que tenga muchas dependencias entre ellas), y así no sacar el máximo partido al paralelismo de la máquina. Por

otra parte, una máquina con escaso paralelismo limitará las prestaciones de la ejecución de un programa sin importar la naturaleza del programa.

Para maximizar el paralelismo a nivel de instrucción de un programa, los compiladores realizan ciertas optimizaciones y los procesadores implementan en hardware ciertas técnicas.

Por ejemplo, el empleo de una arquitectura con instrucciones de longitud fija, como en un RISC, aumenta el paralelismo en las instrucciones.

Arquitectura superescalar

¿Qué es?

Se denomina **arquitectura superescalar** a aquella implementación capaz de ejecutar más de una instrucción por ciclo de reloj, utilizando múltiples cauces de instrucciones independientes, de manera concurrente. Pudiendo llevar a cabo dos o más instancias de cada etapa de una instrucción simultáneamente.

El término **superescalar** se emplea como contraposición a la arquitectura escalar, que solo es capaz de ejecutar una instrucción por ciclo de reloj.

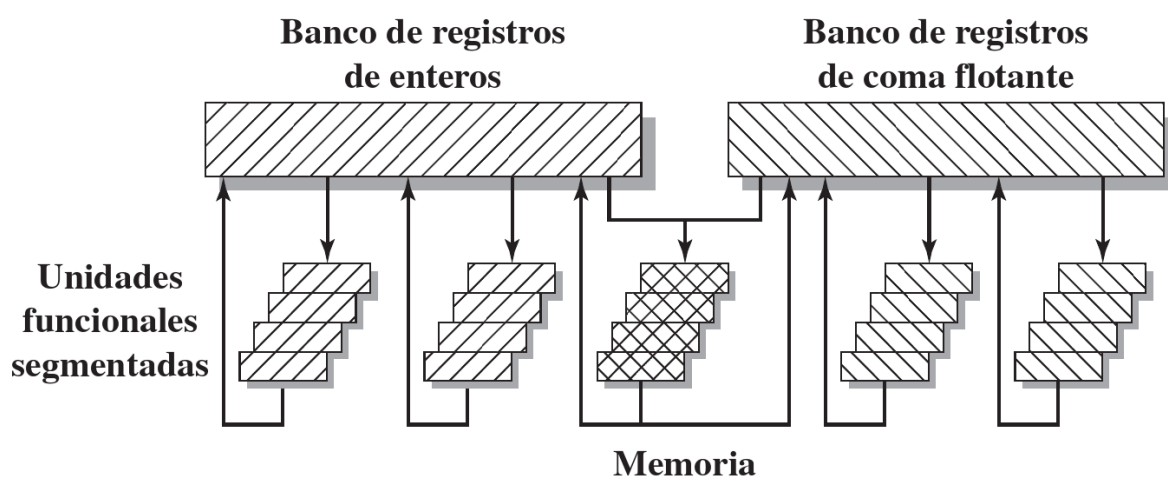
El grado de paralelismo y, por tanto, la aceleración de la máquina aumentan, ya que se ejecutan más instrucciones en paralelo.

El **paralelismo a nivel máquina** no consiste solo en múltiples réplicas de cada etapa del cauce. El procesador además tiene que ser capaz de **identificar el paralelismo en las instrucciones y organizar su captación, decodificación, y ejecución en paralelo.**

Un procesador superescalar capta varias instrucciones a la vez, e intenta localizar instrucciones cercanas que sean independientes entre sí (que no tengan dependencias) y puedan introducirse en el cauce y ejecutarse. Una vez que se han identificado las dependencias, el procesador puede emitir y completar instrucciones en un orden diferente al del código máquina original.

Cuanto más sofisticado es el procesador, menos restricciones impone a estos ordenamientos. Incluso pudiendo alterar cualquier ordenamiento, respecto del orden estrictamente secuencial. La única condición es que el resultado debe ser correcto.

Ejemplo: esquema de la organización superescalar



Se observan las múltiples unidades funcionales, cada una de las cuales está implementada como un cauce segmentado, que admiten la ejecución en paralelo de varias instrucciones.

Si bien se puede realizar una implementación superescalar tanto de una arquitectura RISC como de una arquitectura CISC, la arquitectura de conjunto reducido de instrucciones de una máquina RISC se presta mejor a la utilización de técnicas superescalares.

La aproximación superescalar se convirtió, desde hace muchos años, en algo habitual de implementar en procesadores.

Diseño

El término **emisión de instrucciones** refiere al proceso de iniciar la ejecución de instrucciones en las unidades funcionales (etapa EX). Cuando referimos a su finalización lo mismo, al final de su etapa de ejecución.

La **política de emisión de instrucciones** es el *protocolo* usado para emitir instrucciones. Dicho de otra forma, define la forma en que las instrucciones se captan, ejecutan y como se escriben sus resultados. También puede verse como la forma en que se envían las instrucciones a las diferentes unidades funcionales)

En función del orden en que las instrucciones se captan, ejecutan y escriben sus resultados, existen 3 políticas de emisión de instrucciones en procesadores superescalares:

- Emisión y finalización ordenada.
- Emisión ordenada y finalización desordenada.
- Emisión y finalización desordenada.

Ejemplo: políticas de emisión comparadas

Para analizar las políticas de emisión de instrucciones, vamos a considerar un procesador superescalar con un cauce de 3 segmentos compuesto por:

- Búsqueda y decodificación: IF+D
- Ejecución: Ex
- Escritura de resultados: W

Además, el procesador dispone de:

- 2 unidades de decodificación: D1 y D2.
- 3 unidades de ejecución: E1, E2 y E3.
- 2 copias de la etapa de escritura: W1 y W2.

Esto implica que dos instrucciones pueden ser leídas y decodificadas a la vez. Solo puede leer un nuevo par de instrucciones cuando ambas unidades D1 y D2 se liberan.

Los conflictos se resuelven con paradas.

Se analiza el comportamiento de la siguiente secuencia de 6 instrucciones:

- I1 tarda dos ciclos en ejecutarse.
- I2 tarda 1 ciclo.
- I3 e I4 tardan 1 ciclo y usan (compiten) la misma unidad funcional E3.
- I5 e I6 tardan 1 ciclo y usan (compiten) la misma unidad funcional E2.
- I5 depende del valor producido por I4.

Decodificación		Ejecución			Escritura		Ciclo
I1	I2						1
I3	I4	I1	I2				2
I3	I4	I1					3
	I4			I3	I1	I2	4
I5	I6			I4			5
	I6		I5		I3	I4	6
			I6				7
					I5	I6	8

(a) Emisión en orden y finalización en orden

Decodificación		Ejecución			Escritura		Ciclo
I1	I2						1
I3	I4	I1	I2				2
	I4	I1		I3	I2		3
I5	I6			I4	I1	I3	4
	I6		I5		I4		5
			I6		I5		6
					I6		7

(b) Emisión en orden y finalización desordenada

Decodificación		Ventana	Ejecución			Escritura		Ciclo
I1	I2							1
I3	I4	I1, I2	I1	I2				2
I5	I6	I3, I4	I1		I3	I2		3
		I4, I5, I6		I6	I4	I1	I3	4
		I5		I5		I4	I6	5
						I5		6

(c) Emisión desordenada y finalización desordenada

Observamos:

- a) Emisión y finalización ordenada:
 - Las instrucciones se captan y decodifican de a 2.
 - Las instrucciones I5 e I6 deben esperar hasta que se desocupen ambos decodificadores D1 y D2.
 - Dado que el orden en que se escriben los resultados debe ser el mismo en que se captan, I2 no puede ser escrita hasta que I1 no se haya completado y esté lista para guardarse (W1, W2)
 - I4 tiene que esperar que I3 libere E3. Lo mismo con I5 e I6.

- I5 tiene que esperar que se ejecute I4.
- Se requieren 8 ciclos para ejecutar las 6 instrucciones.
- b) Emisión ordenada y finalización desordenada:
 - Nuevamente, las instrucciones se captan y decodifican de a 2.
 - Dado que el orden en que se escribe puede diferir del orden que la instrucción fue captada, I2 puede ser escrita antes que I1 lo haya hecho.
 - Al permitir la finalización desordenada, I3 puede ser ejecutada con anticipación, ganando un ciclo
 - Ahora se requieren 7 ciclos para ejecutar las 6 instrucciones.
- c) Emisión y finalización desordenada:
 - Las instrucciones se captan y decodifican de a 2, pero ahora la ventana de instrucciones permite, en el ciclo 3, decodificar las instrucciones I5 e I6.
 - Como I6 no tiene dependencias y está lista para ejecutarse, puede ser emitida anticipadamente, incluso antes de I5. La ventana le permite "ver" al procesador más adelante de las instrucciones en conflicto, pudiendo detectar nuevas instrucciones (la I6) que no genera conflicto.
 - Ahora se requieren 6 ciclos para ejecutar las 6 instrucciones.

Política de emisión y finalización ordenada

Es la más sencilla. Ni siquiera los cauces escalares siguen una política tan ingenua. No obstante, es útil considerarla como base para comparar otras aproximaciones más sofisticadas.

Las instrucciones se emiten en el exacto orden en que lo haría una ejecución secuencial, y los resultados son escritos en ese mismo orden.

Para garantizar la finalización en orden, la emisión de instrucciones se detiene temporalmente cuando hay una pugna por una unidad funcional o cuando una instrucción necesita más de un ciclo de reloj para ejecutarse.

Política de emisión ordenada y finalización desordenada

Las instrucciones se emiten en el exacto orden en que lo haría una ejecución secuencial. Los resultados se escriben en "cualquier orden".

La finalización desordenada implica que, en un momento dado, pueden haber tantas instrucciones en la etapa de ejecución como unidades funcionales hayan en la máquina, alcanzando el máximo grado de paralelismo de la máquina.

La emisión de instrucciones se detiene temporalmente cuando hay una pugna por un recurso, una dependencia de datos o una dependencia de saltos.

Con la finalización desordenada aparece la posibilidad de que ocurran dependencias de salida (WAW).

Requiere detectar las posibles nuevas dependencias, y evitar que se ejecuten instrucciones forma desordenada que alteren el resultado del programa. La finalización desordenada necesita una lógica de emisión de instrucciones más compleja que la finalización en orden.

Con la política de emisión ordenada, el procesador decodifica instrucciones hasta el punto de dependencia o conflicto, donde espera hasta que sea resuelto. En otras palabras, al encontrar un stall, no analiza si hay nuevas instrucciones independientes de las que están en el cauce y que podrían introducirse provechosamente en este.

La finalización desordenada se usa en los procesadores RISC escalares para mejorar la velocidad de las instrucciones que necesitan ciclos.

Política de emisión y finalización desordenada

La política de emisión desordenada permite que el procesador decodifique instrucciones aún al hallar una dependencia o conflicto. Para esto se requiere desacoplar las etapas del cauce de decodificación y ejecución, entre estas intercalar un buffer llamado **ventana de instrucciones**.

Con esta organización, cada instrucción decodificada se coloca en la ventana de instrucciones, mientras que este buffer no se llene, el procesador continúa captando y decodificando nuevas instrucciones. A medida que se liberan unidades funcionales de la etapa de ejecución, se emiten las instrucciones, siempre y cuando la instrucción a emitir no posea ningún conflicto o dependencia que la bloquee.

El procesador gana capacidad de anticipación, buscando instrucciones independientes que pueden introducirse en la etapa de ejecución, sin importar la secuencialidad original, siempre y cuando el programa funcione correctamente.

La ventana no es una etapa adicional del cauce. Que una instrucción este en el buffer, implica que el procesador tiene suficiente información sobre la instrucción para decidir si puede emitirla.

Además surge una nueva dependencia, la [dependencias de salida](#) (WAR).

Renombrado de registros

Permitir la emisión desordenada de instrucciones y/o la finalización desordenada puede dar origen a dependencias de salida y antidependencias respectivamente. Surgen porque los valores de los registros no pueden reflejar ya la secuencia de valores dictadas por el flujo del programa (instrucciones que compiten por un mismo registro).

Una técnica que implica la duplicación de recursos, para resolver estos conflictos sin introducir retrasos, es el **renombrado de registros**.

Los registros en las instrucciones del programa son "lógicos", los registros físicos (los reales) los asigna el procesador. Cada vez que se guarda un dato en un registro, el procesador le asigna un nuevo registro físico, de manera que las instrucciones próximas en referirlo puedan acceder al dato correcto.

De esta manera se eliminan las dependencias WAW y WAR (antidependencia y dependencia de salida) quedando únicamente la dependencia verdadera RAW.

Interrupciones

En los procesadores superescalares el tratamiento de [interrupciones](#) internas o excepciones y externas es más complicado que en los procesadores convencionales.

La razón es que en el momento en que se produce una interrupción hay varias instrucciones en ejecución (y además, desordenadas).

- **Interrupciones internas o excepciones:**
 - **Existen 2 estrategias para el tratamiento de excepciones:**
 - **Excepciones precisas:** las instrucciones que preceden a la que produjo la excepción se completan, y las que le suceden se reinician. Es decir, se comporta como si no hubiera segmentación.
 - **Excepciones imprecisas:** no se respetan las condiciones exactas de la máquina no segmentada.
 - Para garantizar un estado consistente (preciso) se requiere:
 - Instrucciones anteriores terminan correctamente
 - La que origina la excepción y siguientes se abortan
 - Tras la rutina de tratamiento se comienza por la que originó la excepción
 - Para que las interrupciones sean precisas los resultados se deben almacenar en el orden en que aparecen las instrucciones.
 - Para ello, se requiere retardar las escrituras de los resultados de tal manera que queden en el orden en el que estaba el programa secuencial.
- **Interrupciones externas:**
 - Las interrupciones externas pueden tener distinto grado de "precisión", dado que no necesariamente tienen que ver con las instrucciones en ejecución (por ejemplo operaciones de I/O).
 - **Interrupciones imprecisas:** se completa lo que estaba en ejecución.
 - **Interrupciones "algo" imprecisas:** el software (la rutina de servicio de la interrupción) resuelve algunas inconsistencias.
 - **Interrupciones precisas:** solo se completan las instrucciones previas a la interrupción, para lo cual se requiere implementar la escritura con buffer de salida. La unidad de emisión deja de emitir, se cancela la cola, todas las instrucciones pendientes se completan, y comienza el procesamiento de la interrupción.

Conclusiones:

- En los sistemas con finalización desordenada:
 - Excepciones imprecisas
 - Mayor performance
- En los sistemas con finalización ordenada (es decir, con buffer de reordenamiento)
 - Excepciones precisas
 - Menor performance

Supersegmentación

- **Primera generación, ejecución secuencial:** se caracterizó por un proceso de ejecución completamente secuencial de instrucciones. No empezaba una nueva instrucción hasta la finalización de la actual.
- **Segunda generación, ejecución segmentada:** se dio la segmentación del cauce, limitada y en pocas etapas. Nuevas instrucciones se iniciaban mientras otras aún estaban en proceso.
- **Tercera generación, ejecución segmentada aumentada:** se profundizó la segmentación, aumentando la cantidad de etapas.

Una solución alternativa (a la técnica superescalar) para alcanzar mayores prestaciones es la **supersegmentación**.

Consiste en doblar la velocidad de reloj interna para realizar dos tareas en un mismo ciclo de reloj externo. De esta forma saca provecho de que muchas etapas del cauce realizan tareas que requieren menos de medio ciclo de reloj.

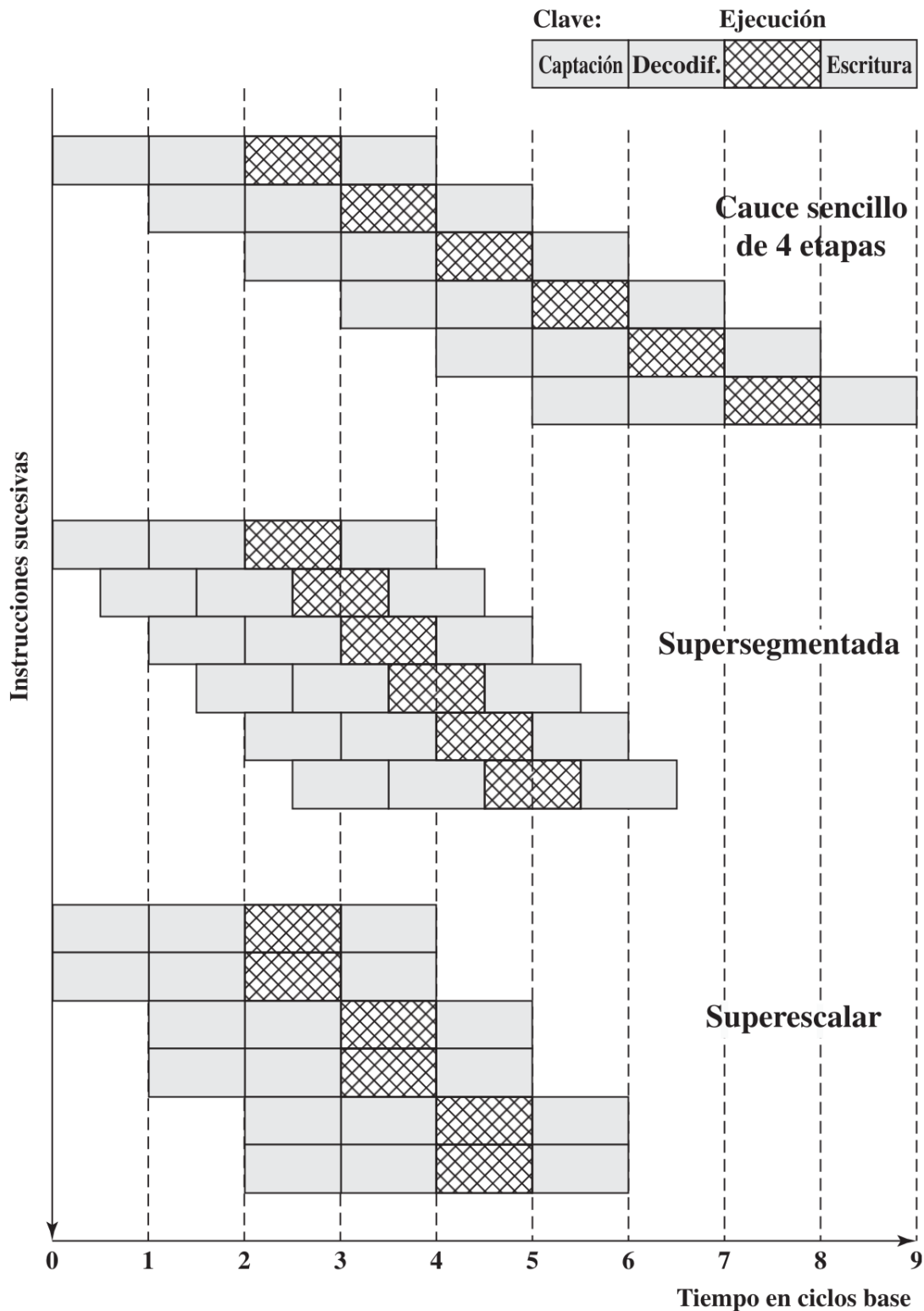
Se segmentan algunas de las etapas del cauce (en concreto las más lentas) del procesador segmentado, en dos o más etapas de forma que se permita que existan dos instrucciones a la vez dentro de la misma etapa (dentro de la misma unidad funcional) sin tener que duplicarla (la réplica constituiría superescalaridad).

Así, un procesador supersegmentado es aquel que aplica dos veces el concepto de segmentación, la primera a nivel global, y la segunda a nivel interno de sus unidades funcionales.

El tiempo para las instrucciones individuales no varía, pero aumenta el grado del paralelismo de la máquina temporal.

El pipeline se hace más profundo, pero está limitado por la tecnología (es decir, por la frecuencia máxima).

Ejemplo: segmentación vs supersegmentación vs superescalar



El cauce de este ejemplo es de cuatro etapas: captación de instrucción, decodificación de la operación, ejecución de la operación y escritura del resultado.

- El cauce base emite una instrucción por ciclo de reloj y puede ejecutar una etapa del cauce en cada ciclo. Solo hay una instrucción en la etapa de ejecución en un determinado instante.
- La implementación supersegmentada es capaz de ejecutar dos etapas del cauce por ciclo de reloj. Una forma alternativa de enfocar esto consiste en que las funciones realizadas en cada etapa se pueden dividir en dos partes no solapadas y que cada una se ejecuta en medio ciclo de reloj. Se dice que una implementación de un cauce supersegmentado que se comporta de esta forma es de grado 2.

- En la implementación superescalar capaz de ejecutar en paralelo dos instrucciones en cada etapa.

También son posibles implementaciones supersegmentadas y superescalares de mayor grado.

El procesador supersegmentado se queda atrás con respecto al procesador superescalar al comienzo del programa y en cada destino de un salto.

Rendimiento

Limites de procesadores segmentados: En general, un procesador de k etapas tiene una productividad teórica máxima igual a k .

- La máxima capacidad teórica es 1 instrucción por ciclo ($CPI = 1$).
- Hay 1 solo pipeline para los diferentes tipos de datos.
- Los atascos en el cauce producen burbujas innecesarias que reducen la productividad.

Rendimientos de procesadores escalares: El rendimiento de un procesador escalar segmentado depende de:

- CPI : instrucciones por ciclo.
- f : frecuencia del reloj
- k : cantidad de etapas del cauce

El tiempo de ejecución se calcula como:

$$CPU_{time} = N_{inst.} \times IPC \times T$$

con T = tiempo de ciclo.

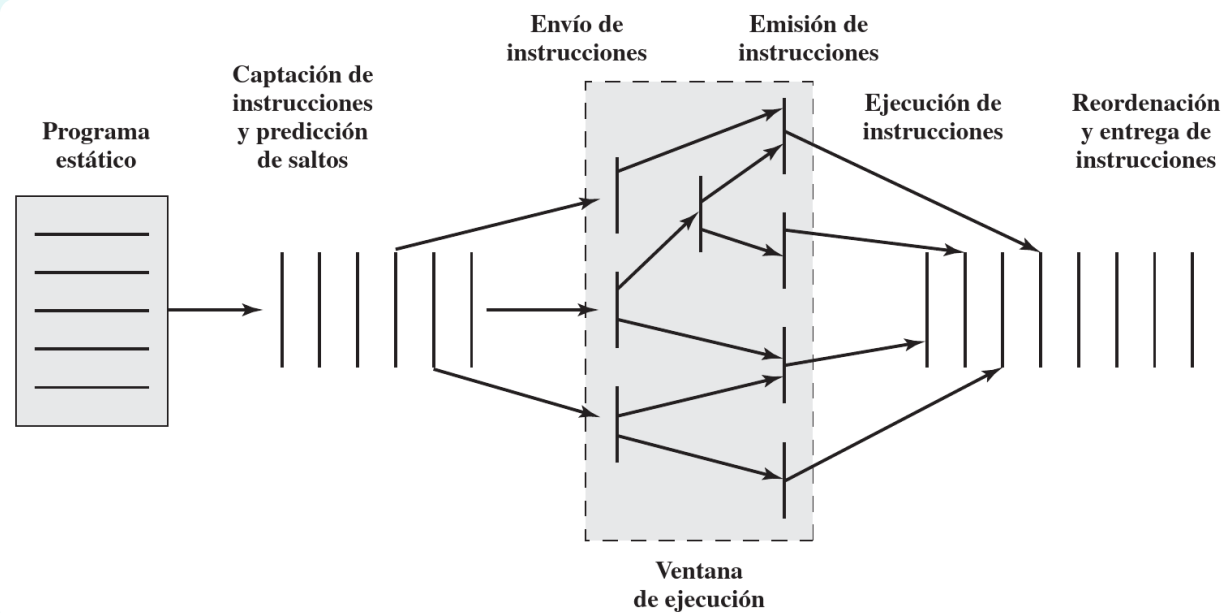
Para estos procesadores, en el mejor de los casos, $IPC = 1$, y solo se puede mejorar si se aumenta f .

Limitaciones en procesadores escalares segmentados: las limitaciones en el rendimiento de un procesador escalar segmentado se originan en dos aspectos:

- El CPI es igual a 1, en el mejor de los casos.
- Los posibles conflictos, que pueden producir atascos en el cauce, se deben a las dependencias ya vistas (estructurales, de datos, de control).

Conclusiones

Ejecución superescalar



Las instrucciones son captadas a partir del programa estático, y decodificadas para la detección temprana de dependencias y predicción de saltos.

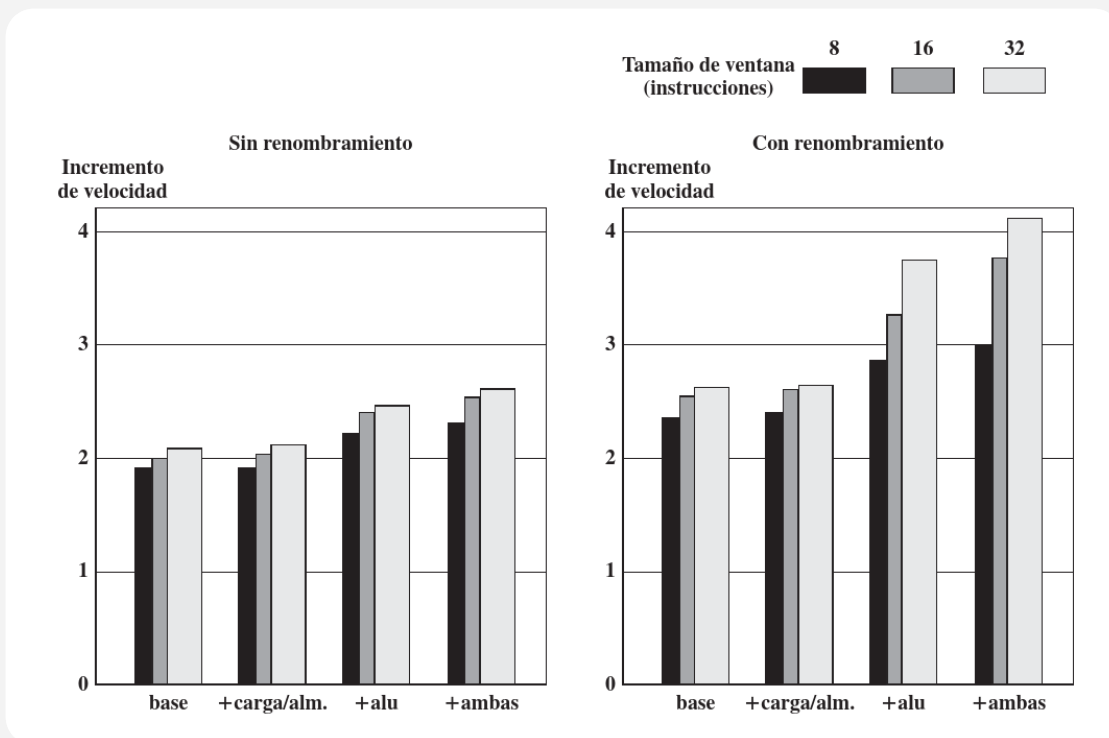
Las instrucciones son enviadas a la ventana de ejecución, donde se reordenan en función de las dependencias verdaderas, esto es, las que están en condiciones de ser ejecutadas o las que necesitan disponer de datos de otras instrucciones.

Las instrucciones son ejecutadas en un orden basado en la disponibilidad de unidades de ejecución y en las dependencias verdaderas. Por último, las instrucciones se vuelven a poner conceptualmente en un orden secuencial y sus resultados se almacenan (commit).

El último paso, llamado commit o entrega, es necesario por la posibilidad de que las instrucciones terminen en un orden diferente a la linealidad del programa inicial, o que se desechen resultados (branch missprediction). El almacenamiento permanente y los registros visibles por el programa no se pueden actualizar inmediatamente después de que las instrucciones finalicen su ejecución. Los resultados han de mantenerse en algún tipo de almacenamiento temporal que sea utilizable por instrucciones dependientes y después convertido en permanente una vez que se determine que el modelo secuencial habría ejecutado la instrucción.

En resumen, la ejecución superescalar involucra:

- Estrategias de captación simultánea de múltiples instrucciones. Estas funciones requieren la utilización de múltiples etapas de captación y decodificación, y lógica de predicción de saltos.
- Lógica para determinar dependencias verdaderas entre valores de registros, y mecanismos para comunicar esos valores a donde sean necesarios durante la ejecución.
- Mecanismos para iniciar o emitir múltiples instrucciones en paralelo.
- Recursos para la ejecución en paralelo de múltiples instrucciones, que incluyan múltiples unidades funcionales segmentadas y jerarquías de memoria capaces de atender múltiples referencias a memoria.
- Mecanismos para entregar el estado del procesador en un orden correcto.



El eje vertical corresponde al incremento de velocidad medio de la máquina superescalar respecto a la máquina escalar. El eje horizontal muestra los resultados para cuatro organizaciones del procesador alternativas. Se muestran resultados para ventanas de 8, 16 y 32 instrucciones, que determinan el grado de anticipación que puede tener el procesador. En la gráfica de la derecha, se permite el renombramiento de registros.

Cuatro tipos de máquinas:

- La máquina base no duplica ninguna de las unidades funcionales, pero puede emitir instrucciones desordenadamente.
- La segunda configuración duplica la unidad funcional de carga/almacenamiento que accede a la caché de datos.
- La tercera configuración duplica la ALU.
- La cuarta configuración duplica tanto la unidad de carga/almacenamiento como la ALU.

La diferencia entre las dos gráficas es que en la segunda se permite el renombramiento de registros. Esto es equivalente a decir que la primera gráfica refleja una máquina limitada por todas las dependencias, mientras que la segunda gráfica corresponde a una máquina limitada solo por las dependencias verdaderas.

Podemos concluir:

Probablemente no merece la pena añadir unidades funcionales sin renombramiento de registros. Hay algunas mejoras de poca importancia en las prestaciones, pero con el coste de una complejidad del hardware aumentada. Con el renombramiento de registros, que elimina las antidependencias y las dependencias de salida, se logran ganancias notables añadiendo más unidades funcionales. Observe, no obstante, que hay una diferencia significativa en la ganancia alcanzable con el uso de una ventana de ocho instrucciones frente a la que se consigue usando una ventana de instrucciones mayor. Esto indica que si la ventana de instrucciones es demasiado pequeña, las dependencias de datos impiden la utilización efectiva de las unidades funcionales

adicionales; es importante que el procesador pueda mirar hacia delante bastante lejos en busca de instrucciones independientes que permitan aprovechar más el hardware.

Paralelismo a nivel proceso

Introducción

La demanda de máquinas de mayor rendimiento es una exigencia que surgió desde la aparición de las primeras computadoras, y continúa en forma permanente.

Existen dos caminos para aumentar la capacidad de procesamiento:

- Mejorar el rendimiento de una máquina **monoprocesador** (con un solo procesador):
 - Explotar el [paralelismo a nivel instrucción](#) (ILP).
 - Optimizar la detección del [paralelismo a nivel hardware](#)/máquina (MLP).
- Disponer de sistemas **multiprocesador** (con varios procesadores):
 - Explotar el **paralelismo a nivel proceso**. Utilizando varios procesadores que puedan ejecutar en paralelo una carga de trabajo dada.
 - Detección del paralelismo a nivel de sistema operativo compilador o programación.

Los **sistemas multiprocesador** explotan el **paralelismo a nivel de proceso**.

Taxonomía de Flynn

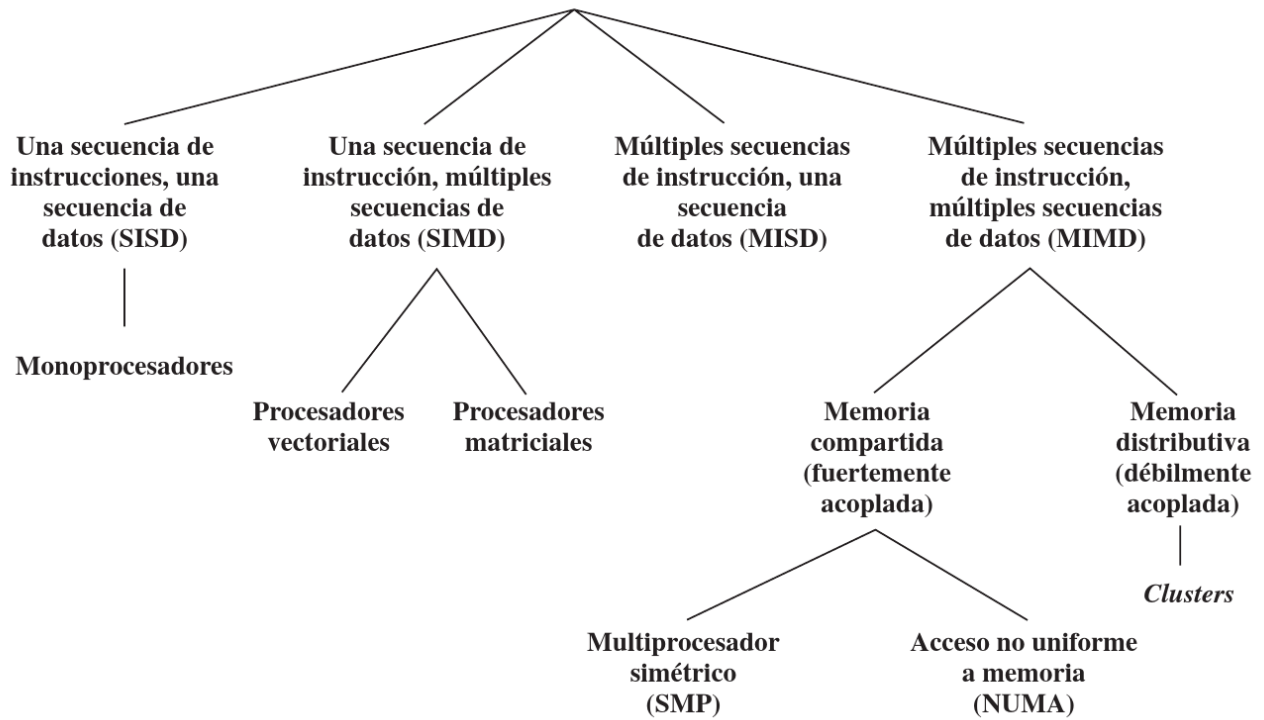
La **taxonomía^[6] de Flynn** es una de las tantas formas de clasificar los sistemas de cómputo. Es la forma más común de clasificarlos según sus capacidades de procesamiento paralelo.

Se utiliza como criterio para la clasificación el **flujo de instrucciones** (*I, instruction*) y el **flujo de datos** (*D, data*).

Flynn propuso las siguientes 4 clases de máquinas:

- **Una secuencia de instrucciones y una secuencia de datos** (SISD, Single Instruction Single Data).
- **Una secuencia de instrucciones y múltiples secuencias de datos** (SIMD, Single Instruction Multiple Data).
- **Múltiples secuencias de instrucciones y una secuencia de datos** (MISD, Multiple Instruction Single Data).
- **Múltiples secuencias de instrucciones y múltiples secuencias de datos** (MIMD, Multiple Instruction Multiple Data).

Organizaciones de computadores



Referencia

Sobre los diagramas en este sección:

- UC: unidad de control
- UP/EP: unidad de proceso/elemento proceso
- SI: secuencia de instrucciones
- SD: secuencia de datos
- UM/ML: unidad de memoria/memoria local

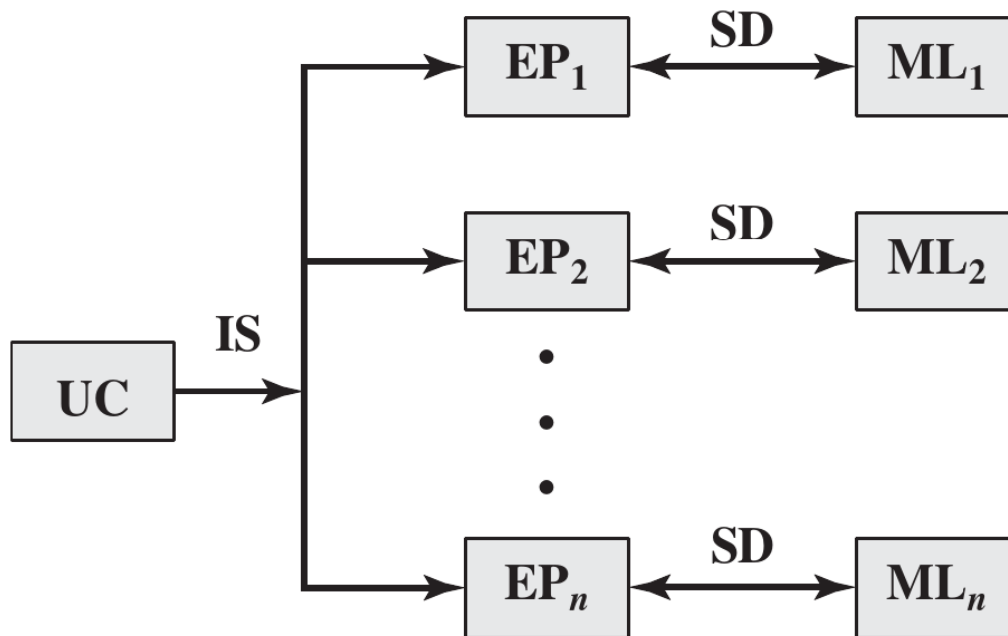
SISD



La unidad de control (UC) interpreta una única secuencia de instrucciones (SI), que la unidad de proceso (UP) resuelve sobre una única secuencia de datos (SD) almacenados en una única unidad de memoria (UM).

Los computadores **monoprocesador** caen dentro de esta categoría.

SIMD



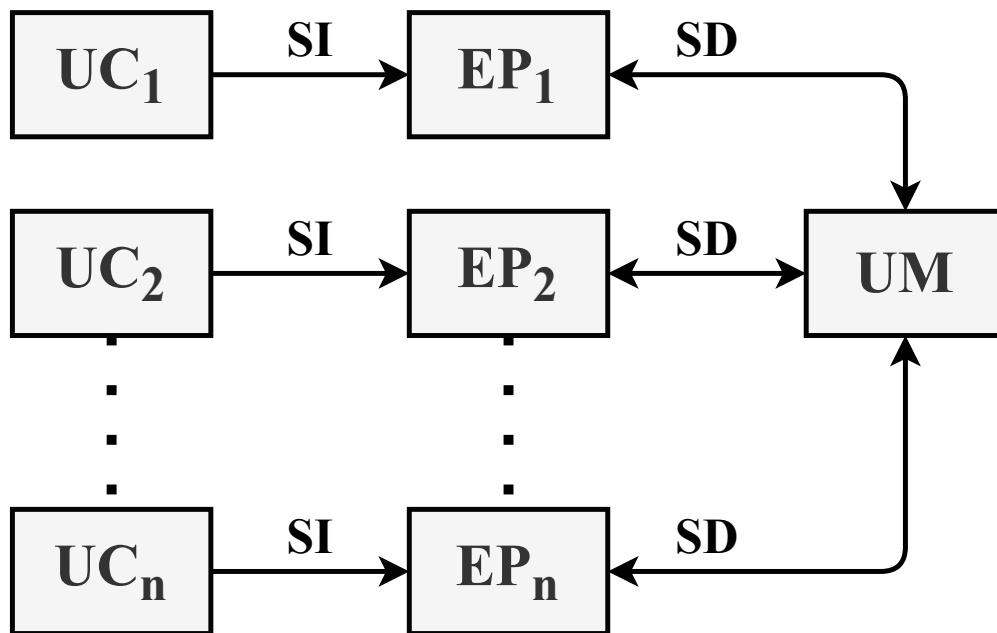
La unidad de control (UC) interpreta una única secuencia de instrucciones (SI) que múltiples elemento proceso (EP) resuelven simultáneamente, cada uno sobre su propia secuencia de datos (SD) proveniente de su memoria local (ML).

Se requieren disponer de:

- Un conjunto ampliado de instrucciones (además de las escalares) para permitir manejar operaciones vectoriales.
- Se deben agregar instrucciones para transferir datos entre unidades de proceso (UP), que es un atributo típico de un "lenguaje paralelo".

Los **procesadores vectoriales** y los **procesadores matriciales** pertenecen a esta categoría

MISD



???? este diagrama está en en el material de la cátedra, es incorrecto

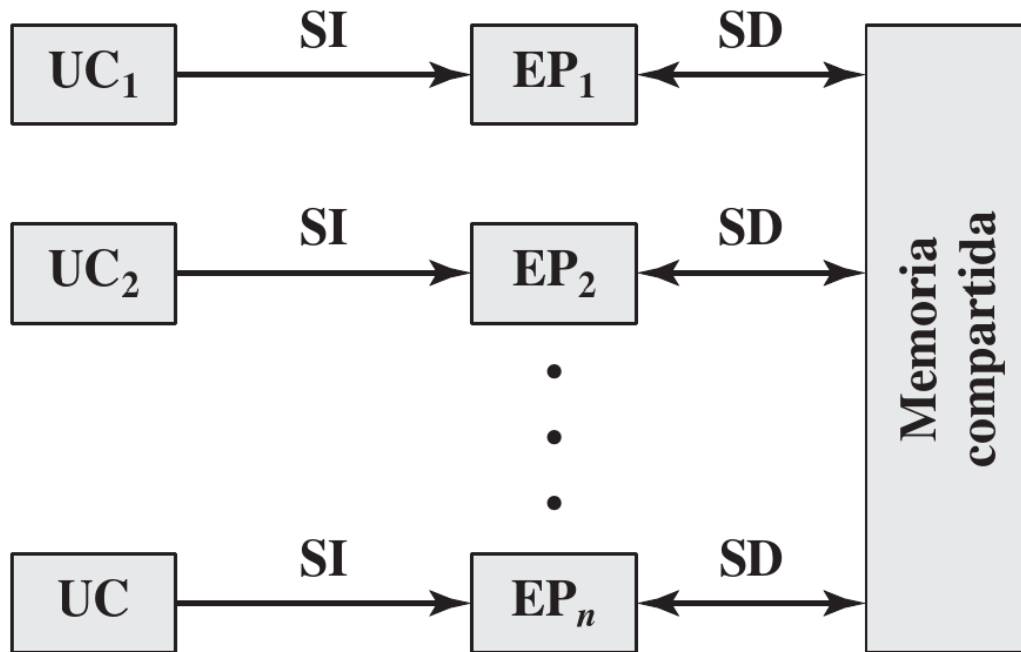
Cada unidad de control (UC) interpreta una secuencia de instrucciones (SI) que cada unidad de proceso (UP) resuelve simultáneamente sobre una única secuencia de datos (SD) proveniente de una sola memoria.

Se transmite una secuencia de datos (SD) a un conjunto de procesadores (UP), cada uno de los cuales ejecuta una secuencia de instrucciones (SI) diferente.

Cada procesador ejecuta una secuencia de instrucciones diferente.

Es un modelo teórico, nunca ha sido implementado.

MIMD - Memoria Compartida



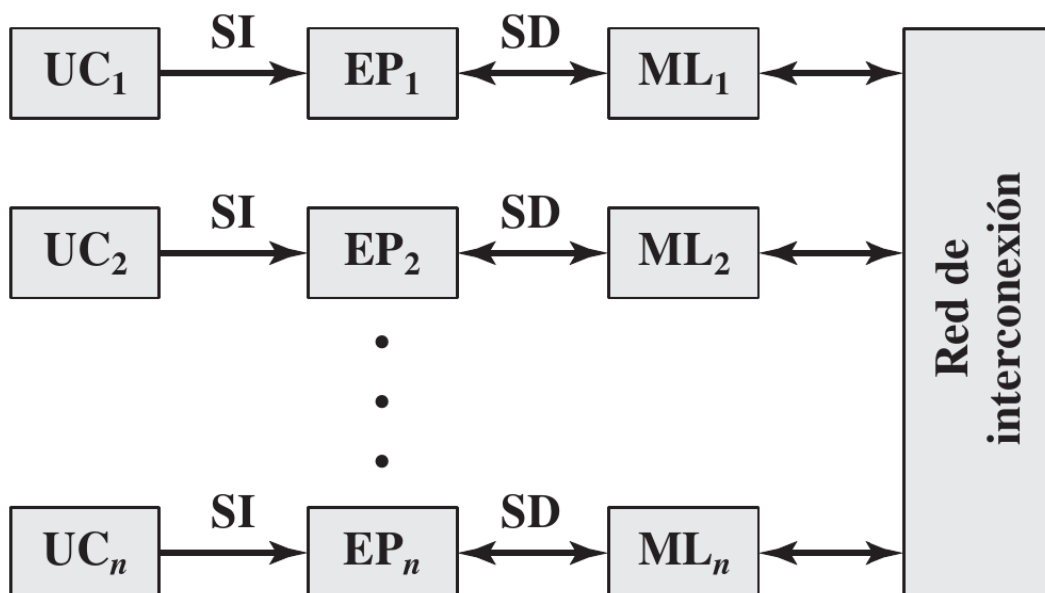
Un conjunto de procesadores ejecuta simultáneamente secuencias de instrucciones diferentes, acceden a estas y los datos almacenados en la memoria compartida, y se comunican unos con otros a través de esa memoria.

Una característica distintiva de estos sistemas es que el tiempo de acceso a memoria principal es aproximadamente el mismo para cualquier procesador (UMA).

Ejemplos de MIMD de memoria compartida son:

- **SMP**
- **Sistemas NUMA** (serían más bien clasificados como un intermedio, compartida/distribuida)

MIMD - Memoria Distribuida

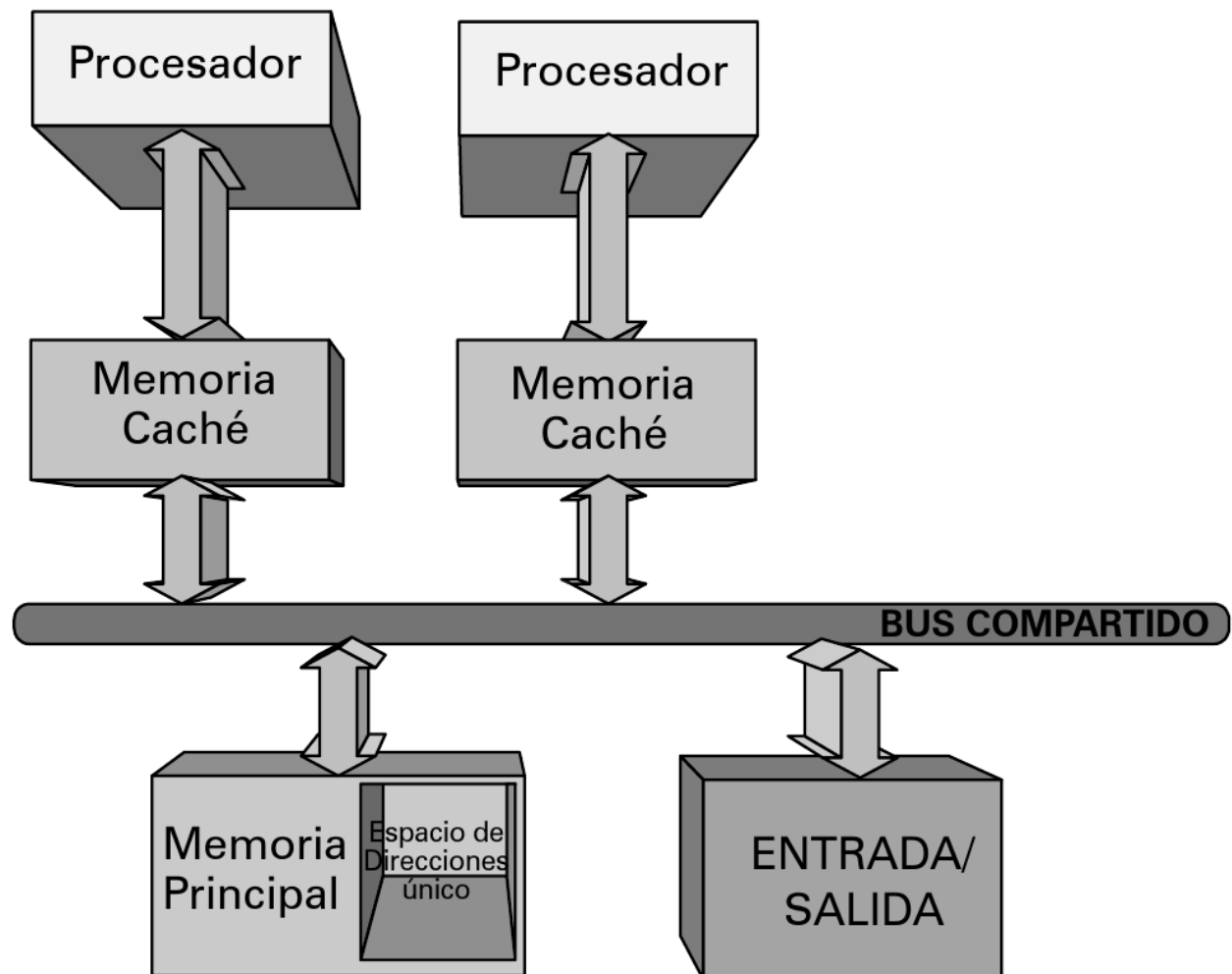


Un conjunto de procesadores ejecuta simultáneamente secuencias de instrucciones diferentes con conjuntos de datos diferentes, ubicados en diferentes memorias. Intercambian información por medio de una red.

Ejemplo de MIMD de memoria distribuida son:

- **Clusters**

Multiprocesadores simétricos SMP



Un **SMP** (*Symmetric multiprocessing*) puede definirse como un computador con las siguientes características:

- **Homogéneos:** Dos o más procesadores idénticos (o muy similares) de capacidades comparables.
- **Fuertemente acoplados:** Memoria principal y E/S única, compartida por todos los procesadores. Interconectada a estos mediante un bus u otro tipo de medio similar.
- **UMA:** Igual tiempo de acceso a la memoria para todos los procesadores.
- **Simétrico:** Todos los procesadores pueden desempeñar las mismas funciones.
- El sistema está controlado por un **sistema operativo integrado**, que proporciona la interacción entre los procesadores y sus programas (diferencia con los clusters).

La existencia de varios procesadores es transparente al usuario. El **sistema operativo** se encarga de la sincronización entre los procesadores y de planificar la distribución de los hilos o de los procesos,

entre los distintos procesadores.

Un sistema SMP tiene las siguientes ventajas con respecto a una arquitectura monoprocesador:

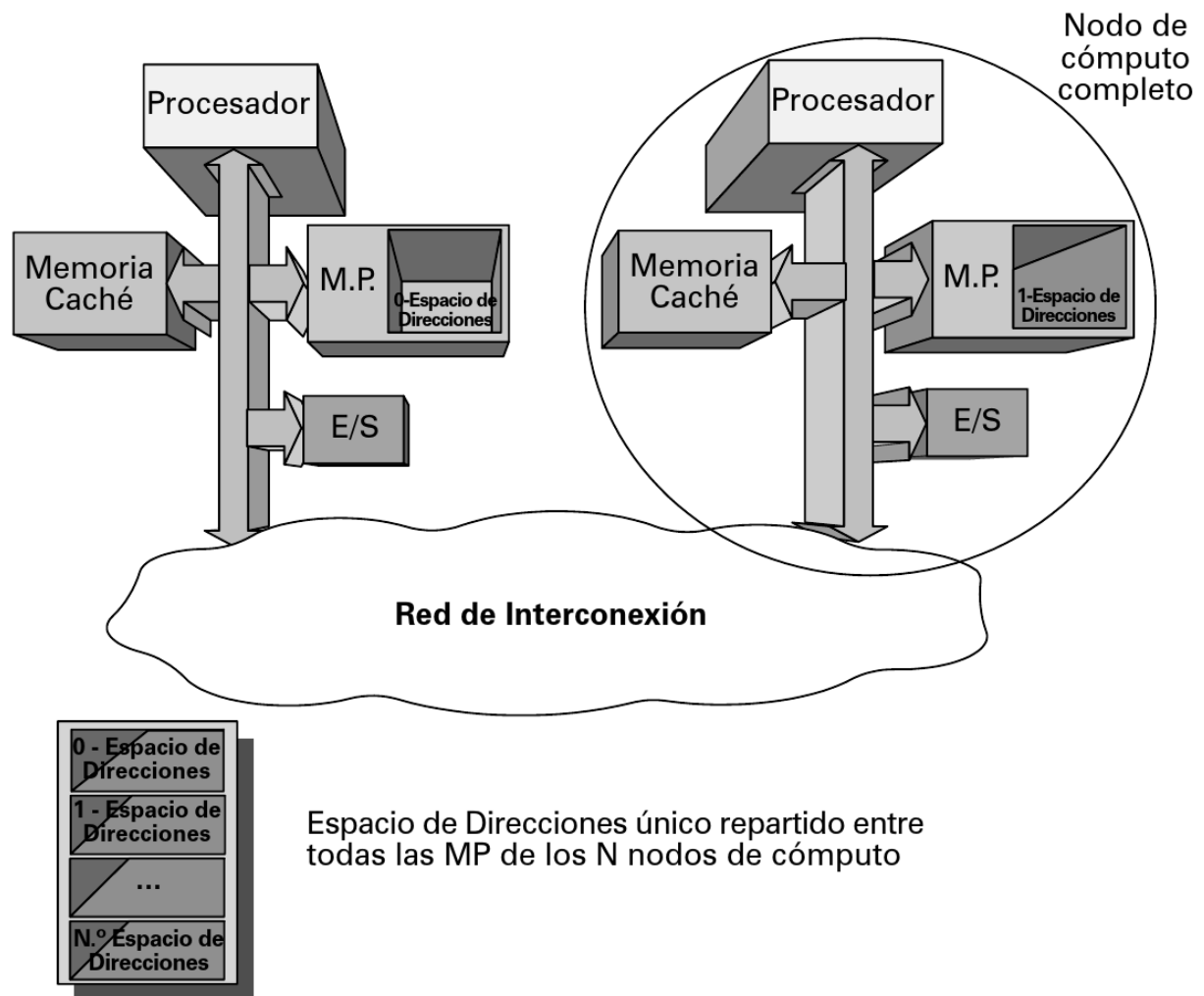
- Prestaciones: si el trabajo a realizar por un computador puede organizarse de forma que partes del mismo se puedan realizar en paralelo, entonces un sistema con varios procesadores proporcionará mejores prestaciones que uno con un solo procesador del mismo tipo
- Buena disponibilidad: debido a que todos los procesadores pueden realizar las mismas tareas, un fallo en un procesador no hará que el computador se detenga.
- Crecimiento incremental: se pueden aumentar las prestaciones del sistema añadiendo más procesadores.
- Escalado: los fabricantes pueden ofrecer una gama de productos con precios y prestaciones diferentes en función del número de procesadores que configuran el sistema.

La compartición de la memoria conlleva a la aparición de tres problemas que deben resolverse para el correcto funcionamiento de la arquitectura: la **coherencia** y **consistencia** de los datos, y la **sincronización** de tareas entre los procesadores.

- Tienen una baja escalabilidad. Todas las referencias a memoria pasan por el bus compartido. Este tiene un ancho de banda limitado y se suele convertir en un cuello de botella al agregar procesadores. En consecuencia, la velocidad del sistema está limitada por el tiempo de ciclo del bus.
- Cada procesador está equipado con una memoria cache, que reduce los accesos a memoria y mejora las prestaciones. Pero introduce el problema de la coherencia, los datos pueden estar en más de una caché. Este problema debe ser resuelto por hardware por razones de velocidad. Se requieren usar protocolos para la administración de los datos en la caché (tipo sondeo o snoopy).

Diseñar arquitecturas de memoria compartida-distribuida, como los sistemas NUMA, contribuyen a superar esta limitación en la escalabilidad.

Multiprocesadores NUMA



Un sistema NUMA (Non Uniform Memory Access) posee dos o más procesadores idénticos (o muy similares) de capacidades comparables (**homogéneos**), que pueden desempeñar las mismas funciones. Los denominamos "**nodos**".

Cada nodo es un procesador con su propia memoria principal y sus propios mecanismos de E/S. Los nodos trabajan independientemente pero están conectados mediante una **red de interconexión** que les permite intercambiar información mediante paso de mensajes.

Se distribuyen estáticamente entre las memorias las direcciones, de manera que exista un **único espacio de direcciones de memoria**, que sea común a todos los procesadores.

El tiempo de acceso a memoria principal de un nodo difiere cuando accede a su memoria local, a cuando accede a la memoria de algún otro nodo. Entonces, el acceso a memoria es **uniforme**, ya que dependiendo la ubicación física del dato, tomará un tiempo diferente accederlo.

Una capa de middleware entre las aplicaciones y el hardware (sistema operativo) hace que la distribución física de la memoria sea transparente a los usuarios. Implica que, por ejemplo, instrucciones como `load` o `store` tengan que ser traducidas a un mensaje de petición por la red.

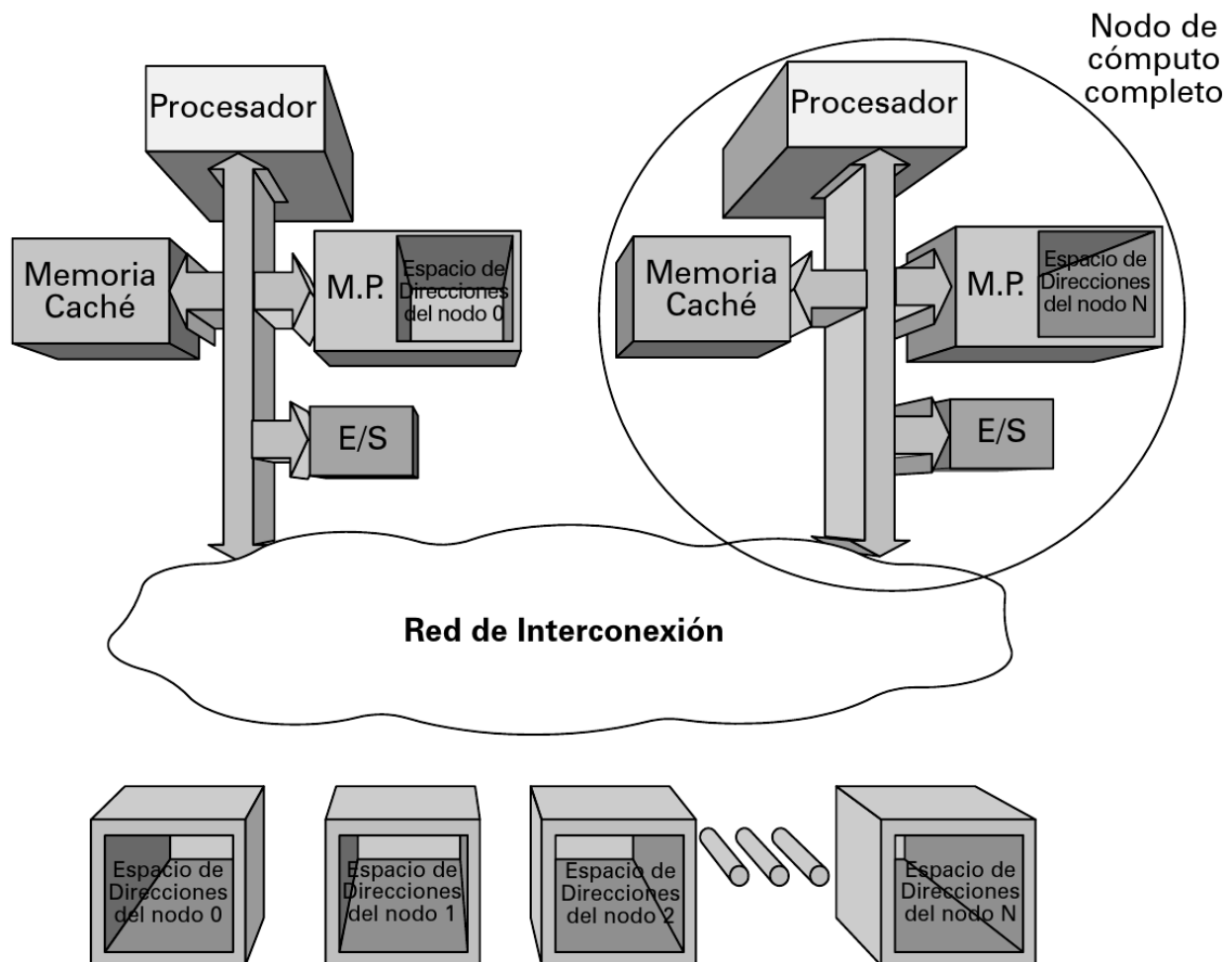
Intenta mantener las ventajas del SMP, especialmente, la similitud con los sistemas monoprocesador a la hora de programar, producto de existir un único espacio de direcciones de memoria. Sin embargo su mayor inconveniente es la escalabilidad.

Los grandes retos en este tipo de arquitectura siguen siendo la resolución de los problemas de coherencia, consistencia y sincronización, que aparecen debido a la compartición de un único espacio

de direcciones, y el middleware que gestione las memorias físicamente distribuidas como si se tratara de una única memoria principal. También existe un reto relacionado con minimizar los efectos que la latencia de la red de comunicaciones añade a los accesos a memoria remotos (entre diferentes nodos).

Para mejorar la escalabilidad aún más, se distribuyen físicamente la memoria entre diferentes nodos y se utiliza una red que no esté basada en medio compartido. Esto ocurre en los cluster.

Cluster



Un conjunto de dos o más computadores monoprocesador independientes o de SMP, que denominamos **nodos**. Los nodos pueden ser homogéneos o heterogéneos, osea, pueden tener características similares (como las arquitecturas ya vistas) o diferir. De hecho, es habitual que sean heterogéneos.

Un nodo es un procesador con su propia memoria principal y sus propios mecanismos de E/S. Si bien cada nodo trabaja de manera independiente, se conecta con otros nodos mediante una **red de interconexión**, con el fin de intercambiar información.

Todos los procesadores pueden desempeñar las mismas funciones. Para las tareas en ejecución se comportan como si fuera una única máquina.

La memoria principal se encuentra dividida, física y lógicamente en **espacios de direcciones físicas independientes**. Por esto recibe el nombre de sistema de memoria distribuida.

Se los considera "**levemente acoplados**", ya que el nivel de interdependencia entre nodos es mínimo, los nodos son más autónomos.

Como cada procesador puede acceder únicamente a su espacio de direcciones local, la comunicación entre procesos debe ser explícita mediante paso de mensajes, a resolver por el programa.

Un middleware entre las aplicaciones y el hardware (sistema operativo) permite que actúe como un único sistema y no como nodos independientes que simplemente están conectados por una red de comunicaciones, proporcionando la interacción entre los procesadores y sus programas.

Las principales ventajas de las arquitecturas de memoria distribuida son su escalabilidad y el uso de comunicación explícita:

- **Escalabilidad absoluta:** dependiendo de la cantidad de nodos incorporados se puede disponer de mayores prestaciones.
- **Escalabilidad incremental:** posibilidad de agregar nuevos nodos fácilmente.
- **Alta disponibilidad:** capacidad de seguir operando con nodos en falla.
- **Mejor relación precio/prestaciones:** porque se usan equipos de cómputo estándar (y, posiblemente, de bajo costo)

Aparecen otros problemas nuevos: la comunicación entre procesos introduce una mayor complejidad en la programación, además introduce una latencia de comunicaciones que puede llegar a ser importante. Por último, habrá que disponer de mecanismos que permitan a los usuarios tener una imagen de sistema único o repartir el trabajo entre los nodos que componen el sistema, por poner sólo algunos ejemplos.

SMP	Cluster
Permiten dar soporte a aplicaciones de alta demanda de recursos	Permiten dar soporte a aplicaciones de alta demanda de recursos
Disponibles comercialmente (más antiguo)	Disponibles comercialmente
Más fácil de administrar y configurar	Superior escalabilidad incremental y absoluta
Cercano a los sistemas de un solo procesador	Superior disponibilidad
La planificación (scheduling) es la diferencia principal	Mayor redundancia
Menos espacio físico / menos consumo de potencia	Cada nodo tiene su propia memoria principal. Las aplicaciones no "ven" la memoria global
Límite práctico en su número de procesadores, entre 16 y 64 por degradación de prestaciones	La coherencia se mantiene por software, no hardware
	Brindan multiprocesamiento a gran escala (Ej existen clusters con 1024 procesadores)

Clasificación según el acceso a memoria

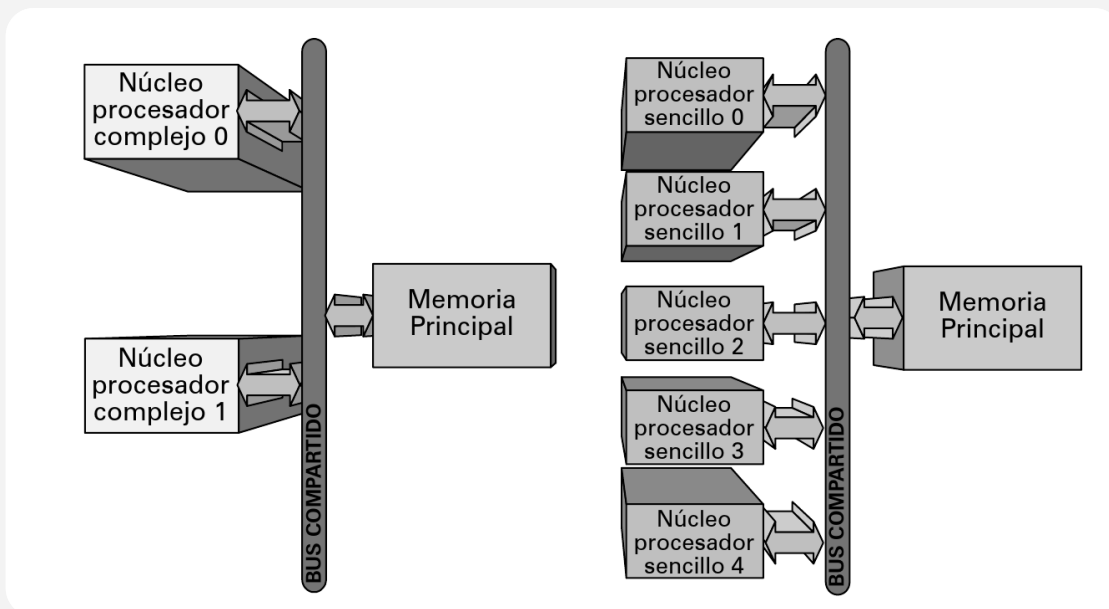
De acuerdo a su acceso a memoria, podemos clasificar a los sistemas multiprocesador como:

1. **UMA**, *Uniform Memory Access*: Igual tiempo de acceso a memoria, y todas sus regiones, para los diferentes procesadores. Por ejemplo SMP.
2. **NUMA**, *Non Uniform Memory Access*: El tiempo de acceso de un procesador difiere, dependiendo de la región de memoria que accede. Diferentes procesadores acceden a diferentes regiones de memoria a diferentes velocidades. Ej: *Cluster*
3. **CC-NUMA**, *Cache-Coherent Non Uniform Memory Access*: Es un NUMA que mantiene coherencia de cache entre las cache de los distintos procesadores, solventando el mayor de sus inconvenientes. Ej. sistemas con memoria compartida/distribuida. ?

Arquitecturas on-chip

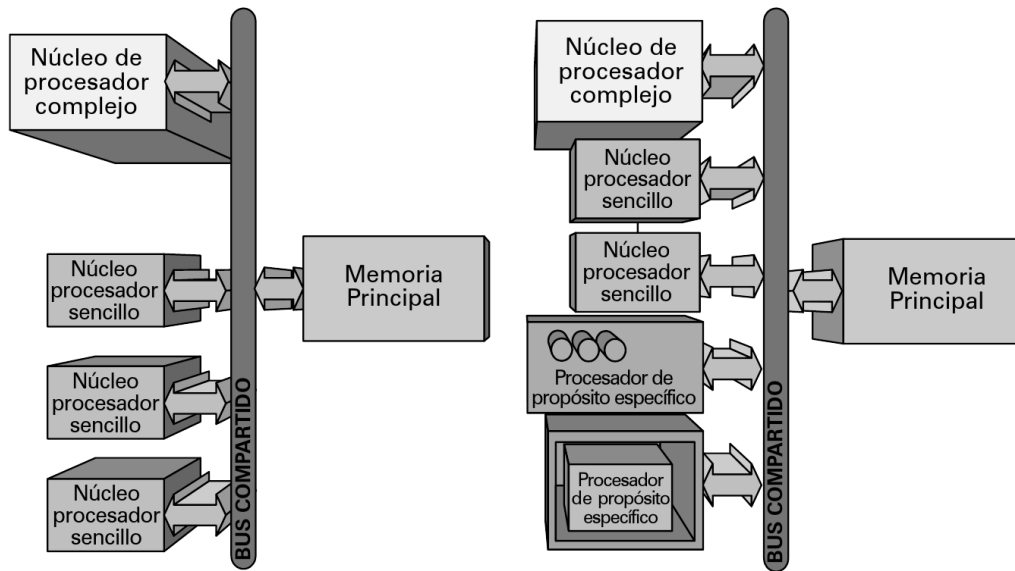
Tanto las arquitecturas on-chip con memoria compartida, como distribuidas pueden ser **homogéneas** o **heterogéneas**.

Arquitectura homogénea on-chip con memoria compartida



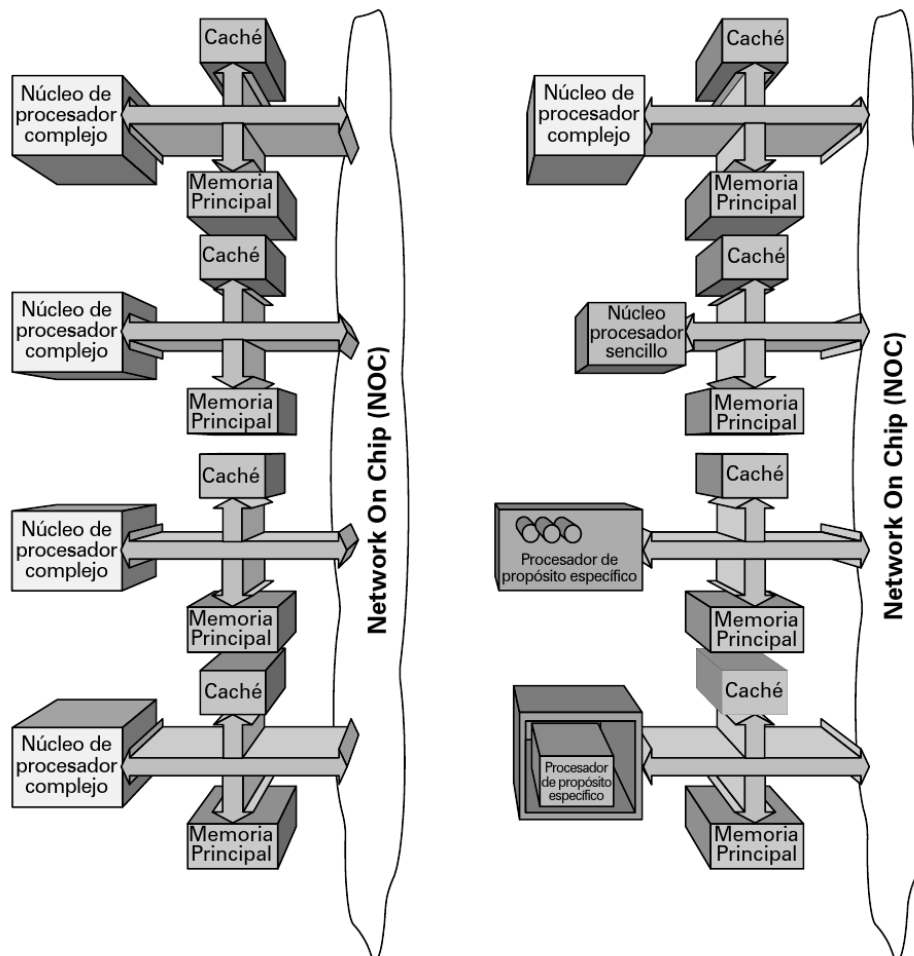
Todos los procesadores son idénticos

Arquitectura heterogénea on-chip con memoria compartida



Los procesadores difieren en prestaciones.

Arquitectura on-chip con memoria distribuida



Multithreading

La velocidad a la que se ejecutan instrucciones es una de las más importantes características de una CPU. Los diseñadores han buscado su mejora en dos frentes:

- incrementando la frecuencia de reloj
- e incrementando el número de instrucciones que se completan en un ciclo de reloj, con el paralelismo de instrucciones (ILP)

Otra alternativa, que permite un paralelismo entre instrucciones elevado sin incrementar ni la complejidad de los circuitos ni el consumo de potencia, es el **procesamiento multihebra (multithreading)**. Esta técnica aprovecha el **paralelismo a nivel hilo (TLP, Thread Level Parallelism)**.

Un **proceso** es un programa en ejecución en un computador, con las siguientes características:

- Un proceso es **propietario** de sus recursos. Dispone de un espacio de direcciones virtuales para almacenar la imagen del proceso, que consta del programa, los datos, la pila y demás atributos que definen el proceso.
- La **ejecución** de un proceso sigue un camino de ejecución (traza) a través de uno o más programas. Esta ejecución puede entremezclarse con la de otros procesos. Así, un proceso tiene un estado de ejecución (preparado, en ejecución, etc.) y una prioridad de asignación. La entidad encargada de **planificar** y de la **conmutación entre procesos** (asignar tiempo de CPU a cada proceso) es el sistema operativo.

Un **hilo, hebra o thread** es una unidad de trabajo dentro de un proceso se puede asignar al procesador. Un proceso tener múltiples hilos. Cada hebra:

- Tiene su propio contexto de procesador (incluido PC y SP) y su propia área de datos para su pila (para permitir la llamada a subrutinas).
- Se ejecuta secuencialmente y puede interrumpirse para que el procesador pase a ejecutar otra hebra dentro del mismo proceso (**conmutación entre hebras**).
- Comparte con otras hebras los recursos pertenecientes al proceso.
- Es relativamente fácil de interrumpir, el procesador cambia a otra hebra rápidamente si lo necesita.

Mientras que las hebras aparecen involucradas en la planificación y ejecución, los procesos tienen que ver tanto con la planificación/ejecución como con la asignación de recursos.

Utilizar esta técnica involucra directamente al sistema operativo y al compilador.

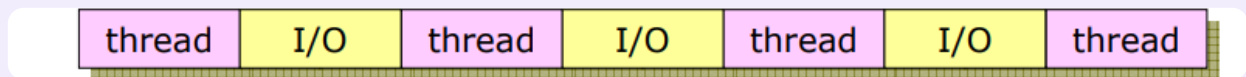
El **multithreading** consiste en ejecutar al mismo tiempo dos o más threads de un programa, permitiendo que cada uno de estos hilos sea planificado de la manera más conveniente en el procesador, es decir, aprovechando al máximo todos los recursos disponibles. Es equivalente a tener dos o más procesadores lógicos en lugar de uno sólo.

La secuencia de instrucciones se divide en secuencias más pequeñas, que pueden ejecutarse en paralelo en diferentes hebras. Ejecutando más de una hebra de manera simultánea se mejoran las prestaciones. Si hay recursos libres durante la ejecución de una hebra, ejecutando varias se los aprovecha mejor.

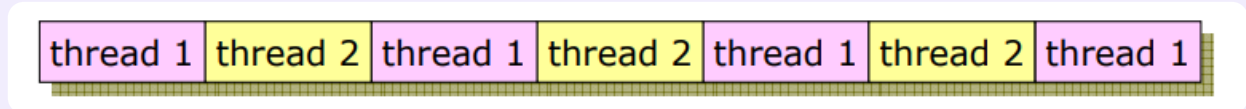
Ejemplo:

El multithreading consiste en ejecutar 2 o más hebras en forma concurrente.

Si se está ejecutando una única hebra, y el procesador queda inactivo durante los períodos de tiempo donde se ejecutan acciones de I/O:



Pero si se pueden ejecutar múltiples hebras, durante los períodos de I/O



#todo

A partir de acá está copiado y pegado de las diapositivas, no completé nada, no entra en el examen teórico de promoción y no tengo tiempo :c

La ejecución de múltiples hebras puede ser de 2 tipos:

- Multihebras explícitas: cuando son definidas en el programa, tanto a nivel de usuario (aplicaciones) como a nivel de Sistema operativo (núcleo).
- Multihebras implícitas: cuando son definidas por el compilador (estático) o por hardware (dinámico).

Para poder implementar procesamiento multihebra es necesario, al menos:

- Un PC (contador de programa) distinto para cada hebra que pueda ejecutarse concurrentemente.
- HW para ejecución concurrente.

El contexto de una hebra está formado por:

- El contador de programa (PC)
- Registros de datos, de direcciones (punteros), de estado, control, de segmentos
- Datos propios en memoria (eventualmente)
- Datos en caché (eventualmente)

Cuando se cambia de una hebra a otra, se produce un cambio de contexto (de hebras).

Los cambios de contexto de hebras pueden ser de 2 tipos:

- tradicional
- rápido.

Tradicionalmente, el cambio de contexto lo produce el SO mediante una interrupción, típicamente asociada a un timer (asignación temporal de hebras):

- La interrupción detiene la hebra en ejecución.
- El SO salva el contexto de la hebra en ejecución.
- El SO recupera el contexto de la hebra detenida anteriormente.

- La hebra detenida anteriormente se reinicia mediante un retorno de interrupción.
- La hebra detenida no se entera que fue interrumpida.

El cambio de contexto puede darse más rápido si se implementa por hardware:

- Se produce porque la hebra queda en espera por alguna razón, típicamente por un fallo de acceso a la caché (tener en cuenta que la PF de acceso a la caché es de varios órdenes de magnitud respecto de un acceso a la caché).
- Si el procesador tiene duplicados, al menos, el PC y los registros (todos o gran parte), entonces el procesador conmuta de hebra sin necesidad de salvar el contexto de la hebra detenida por el fallo (que consume tiempo).
- Este cambio de contexto es mucho más rápido que el tradicional (del SO).

Las hebras en ejecución usan recursos. Los recursos pueden ser asignados de 2 formas:

- Estática: el particionado de recursos es fijo.
- Dinámica: el particionado de recursos cambia por algún mecanismo.

Existen 4 políticas de ejecución multihebra (MT):

- CMP (Chip Multi Processor)
- FGMT (Fine Grained Multi Threading)
- CGMT (Coarse Grained Multi Threading)
- SMT (Simultaneous Multi Threading)

Procesamientos multihebra tipo CMP

- El procesador dispone de 2 o más núcleos. A cada núcleo se le asigna una hebra en forma fija.
- La política de partición CMP es de tipo estática espacial.
- Típico de procesadores SMP de 2 o más núcleos.

Procesamientos multihebra tipo FGMT

- El procesador conmuta de una hebra a otra en cada ciclo. La conmutación es rápida, 1 sola hebra por vez. El procesador dispone de 2 o mas contextos, para ejecutar 2 o más hebra concurrentemente (Ej: Cray CDC 6600 y Denelcor HEP).
- La política de partición FGMT es de tipo estática temporal.
- Al ser una conmutación por ciclo se dice que es de “grano fino” (FGMT : Fine grained Multi-threading)
- Reduce las latencias en memoria, y evita y resuelve dependencias de datos.
- Requiere que el compilador encuentre muchas hebras independientes.

Procesamientos multihebra tipo CGMT

- El procesador conmuta de una hebra a otra cuando una hebra se detiene por algún evento de gran latencia (por ejemplo, fallo de acceso a la caché).
- La política de partición CGMT es de tipo dinámica temporal.
- La conmutación por algún fallo se dice que es de “grano grueso” (CGMT : Coarse grained Multi-threading)
- En general requiere un manejo más complejo porque la conmutación es asincrónica.

- Se deben replicar registros para conmutar rápido.
- Se debe hacer una distribución equitativa de las hebras.

Procesamientos multihebra tipo SMT

- Hay varias hebras ejecutándose en varias unidades funcionales de un procesador superescalar (1 solo núcleo).
- La política de partición SMT es de tipo dinámica espacial. Ej: Pentium 4, core i3, i5, i7 (Intel lo llama Hyper-threading)

Ejemplo: Pentium 4

El Pentium 4 es un procesador con una arquitectura Multithreading (Hyper-threading o HT como la llama Intel) híbrida, que permite disponer de 2 procesadores lógicos compartiendo algunos de los recursos de ejecución del procesador físico (es decir, 1 solo núcleo).

El SO reparte las tareas entre ellos. Cada procesador lógico puede actuar individualmente. La mejora depende fuertemente de grado de paralelización de las tareas. En algunos casos puede llegar a un 30% de mejora. En otros puede hacer la ejecución más lenta (por ello se puede deshabilitar el "modo HT").

El Pentium 4 tiene distintas políticas de tratamiento de las hebras dependiendo de las unidades funcionales involucradas (por eso es un híbrido). En la imagen siguiente se pueden ver las estrategias usadas en cada etapa.

- Etapas Fetch-Decode-Dispatch-Issue 1: FGMT, hasta que una hebra se frena, pasando a CGMT.
- Issue 2 – Ex – Mem: SMT, las hebras pueden mezclar instrucciones.
- Retire: FGMT

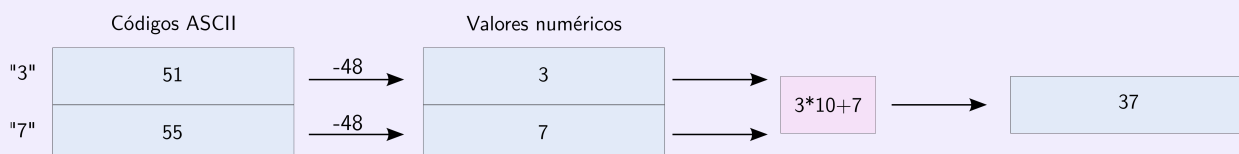
Notas de ensamblador

E/S, manipular números

Al leer un número necesitamos un byte por dígito, debido a su codificación ASCII.

Si queremos almacenarlo en BSS, podemos restarle 30h (48d) a cada dígito. Y luego multiplicar cada dígito por 10^n con n siendo la posición del dígito en el número.

Ejemplo: si leímos $125 = 1 \times 10^2 + 2 \times 10^1 + 5 \times 10^0$



De la misma manera, para imprimir un número, lo dividimos por 10, almacenamos el resto + 30h y repetimos la operación con el resultado de la previa, hasta que la operación de división de como resultado 0.

Ejemplo:

- $125 \div 10 =$ cociente: 12, resto: 5
- $12 \div 10 =$ cociente: 1, resto: 2
- $1 \div 10 =$ cociente: 0, resto: 1.

A la hora de implementarlo, observar que nos da los números en orden inverso. Una forma de circunvenir esta situación es: primero realizar la operación detallada pero contando la cantidad de dígitos, para luego realizar la operación nuevamente mientras se guardan al revés codificados en ASCII, generando así el string.

Imprimir números con signo en MSX88

Los números negativos en el MSX88 están codificados en CA2. Si el número es positivo tendrá el primer bit 0, se imprime como siempre. Si es negativo tendrá el primer bit en 1, para imprimirlo debemos:

- Imprimir un "-"
- Obtenemos el valor absoluto del número (BSS) utilizando la instrucción `neg` (complemento).

WINMIPS: Trabajar con direcciones en vez de valores

Para cargar en un registro una dirección de memoria, en vez del valor, podemos utilizar:

```
daddi $a1, $zero, TABLA
```

Para recuperar el valor desde una subrutina:

```
daddi $t1, $a1, 0
ld    $t2, 0($t1)
```

WINMIPS: Trabajar con ascii

Un caracter en ASCII ocupará 1 byte. Lo declaramos como:

```
.data
caracter: .ascii "a"
```

Para cargar el valor en un registro recordar utilizar `lbu` (*load byte unsigned*) y no `lw`:

```
lbu $t0, caracter
```

Glosario

1. Una **palabra** es una cadena finita de bits que el procesador puede manipular como unidad. El tamaño o longitud de una palabra hace referencia al número de bits contenidos en ella. Esto suele ver reflejado en varios aspectos del diseño de una CPU. En general, es el tamaño de el mayor dato que puede ser transferido desde y hacia la memoria en una misma operación.↵
2. **PSW** (program status word) o registro de flags. Sus bits son utilizados de manera lógica o individual, para indicar el estado del procesador. Es escrito de manera implícita al ejecutarse ciertas instrucciones. Permite que una instrucción realice una acción en consecuencia del resultado de una instrucción que corrió previamente.↵
3. Un **escalar** es una constante o variable que contiene un dato atómico y unidimensional. En contraposición al concepto de escalar, están los conceptos de array (vector), lista y objeto, que pueden tener almacenado en su estructura más de un valor.↵
4. La **integración a escala muy grande** o **VLSI** (*very large-scale integration*) es el proceso de crear un circuito integrado compuesto por millones de transistores en un único chip. Los primeros chip semiconductores contenían solo un transistor cada uno. A medida que la tecnología de fabricación avanzó, se añadieron más transistores, integrando más funciones en un mismo chip. Comenzó a usarse en los años 70. *para más info leer los apuntes de organización de computadoras*↵
5. **Associative memory** is a memory unit whose database can be accessed by the data or content instead of an address or memory location. It is also called content addressable memory or CAM. It helps optimize searching any data as the searching process does not involve addressing, but it works on data.↵
6. **Taxonomía**: clasificación u ordenación en grupos de cosas que tienen unas características comunes.↵